

ARC12 ▪ Architettura dei Calcolatori e Laboratorio

RISC-V sulla piattaforma PRG32

Università degli Studi di Napoli «Parthenope» • A.A. 2026/27

Architettura dei calcolatori

Prima programmare, poi formalizzare

Dalla prima istruzione RISC-V a un gioco pubblicato:
informazione, logica, memoria e processore, insegnati sulla piattaforma PRG32
attraverso Verm1, il progetto che accompagna tutto il corso

Raffaele Montella

Dipartimento di Scienze e Tecnologie
Università degli Studi di Napoli «Parthenope»

Implementazione di riferimento di Verm1: contributo di Giuseppe Coviello

Libro di testo delle lezioni di *Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori* (ARC12),
Corso di Laurea in Informatica, primo anno, primo semestre.

I listati assembly di questo libro sono letti da file che vengono assemblati
e provati automaticamente.

Piattaforma: https://github.com/riscv-prg32/PRG32
Host desktop e debugger: https://github.com/riscv-prg32/PRG32-QT

Architettura dei calcolatori: prima programmare, poi formalizzare.
Libro di testo delle lezioni di ARC12, Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori, Università degli Studi di Napoli «Parthenope», anno accademico 2026/27. Edizione italiana di *Computer Architecture: Program First, Formalise Later*.

Il libro segue, capitolo per capitolo, le venticinque presentazioni del corso: il Capitolo N è la Lezione N. Le presentazioni, i programmi di riferimento del progetto Verm1 e i test che assemblano ed eseguono ogni listato sono conservati insieme ai sorgenti di questo libro; i listati stampati qui sono letti dagli stessi file che i test eseguono. Per questo i listati, con i loro commenti, sono in inglese in entrambe le edizioni: sono il codice provato, non una sua copia.

Il libro è costruito attorno a PRG32, un runtime didattico aperto per giochi in assembly RISC-V e in C, che ha come destinazioni il microcontrollore ESP32-C6, l'emulatore QEMU di Espressif e l'host desktop PRG32-QT. La piattaforma, il suo codice sorgente e la sua documentazione sono distribuiti con licenza MIT dai loro autori: Raffaele Montella (responsabile scientifico e del progetto), Ivan Cafiero e Simone Boscaglia (studenti contributori), Università degli Studi di Napoli «Parthenope». A chi usa PRG32 in esercitazioni, relazioni, tesi o materiale didattico si chiede di citarlo attraverso i metadati `CITATION.cff` del suo repository (Montella et al., 2026a). Il volume che lo accompagna, *Playing with RISC-V* (Montella et al., 2026b), è il manuale di laboratorio della piattaforma.

L'implementazione di riferimento di Verm1, il progetto che accompagna tutto il libro (i quattro programmi dell'Appendice **??**), è un contributo di Giuseppe Coviello.

Il testo e le figure originali di questo libro sono rilasciati con licenza Creative Commons Attribuzione – Condividi allo stesso modo 4.0 Internazionale (CC BY-SA 4.0). I programmi sono rilasciati con licenza MIT. Le fotografie storiche sono riprodotte dalle slide delle lezioni a scopo didattico e restano di proprietà dei rispettivi titolari.

RISC-V è un marchio registrato di RISC-V International. ESP32, ESP32-C3, ESP32-C6 ed ESP-IDF sono prodotti e marchi di Espressif Systems. GitHub è un marchio di GitHub, Inc. Tutti gli altri marchi appartengono ai rispettivi titolari e sono usati qui al solo scopo di identificazione.

Edizione compilata il 12 ottobre 2026. Composto con LaTeX in Latin Modern.

Indice

Prefazione **vii**

Come usare questo libro **ix**

I Partenza **1**

0 Introduzione: prima programmare, poi formalizzare **3**
0.1 Il corso in breve . . . 3
0.2 Un approccio nuovo . . . 4
0.3 La macchina: PRG32 e RISC-V . . . 5
0.4 Il progetto: Verm1 . . . 8
0.5 Il corso in blocchi . . . 9
0.6 Programma e risultati di apprendimento . . . 9
0.7 Organizzazione e valutazione . . . 11
0.8 Materiale . . . 12
0.9 Primo contatto: chi siete? . . . 13
0.10 Prima del prossimo capitolo . . . 14

1 PRG32 e PRG32-QT: allestire l'officina **15**
1.1 Quattro livelli: installate ciò che vi serve . . . 15
1.2 La piattaforma, in dettaglio . . . 16
1.3 Passo 1: installare PRG32-QT . . . 18
1.4 Passo 2: installare la toolchain . . . 19
1.5 Passo 3: la vostra prima compilazione . . . 23
1.6 Facoltativo: QEMU e il kit . . . 24
1.7 Quando qualcosa non funziona . . . 25
1.8 Verificate di aver capito . . . 26
1.9 Esercizi . . . 26
1.10 Glossario . . . 27
1.11 Riepilogo e letture . . . 27

2 Primo contatto: assembly RISC-V su PRG32 **29**
2.1 La macchina . . . 29
2.2 La vostra prima cartuccia . . . 31
2.3 Leggere ciò che avete scritto . . . 36
2.4 Che cosa viene dopo . . . 38
2.5 Verificate di aver capito . . . 38

2.6 Esercizi . . . 39
2.7 Glossario . . . 39
2.8 Riepilogo e letture . . . 40

II Informazione 41

3 Storia della computazione 43
3.1 Che cos'è l'informatica? . . . 43
3.2 Prima dell'elettricità . . . 44
3.3 Il Novecento . . . 46
3.4 Dagli home computer a RISC-V . . . 47
3.5 L'informatica come scienza applicata . . . 50
3.6 Verificate di aver capito . . . 50
3.7 Esercizi . . . 51
3.8 Glossario . . . 52
3.9 Conclusioni e letture . . . 52

4 Rappresentare l'informazione digitale 53
4.1 L'informazione . . . 53
4.2 Il testo . . . 57
4.3 Strutture dati . . . 59
4.4 Dati e algoritmi . . . 60
4.5 Verificate di aver capito . . . 62
4.6 Esercizi . . . 63
4.7 Glossario . . . 63
4.8 Conclusioni e letture . . . 63

5 Rappresentazione dei numeri: gli interi 65
5.1 Riscaldamento sulla macchina . . . 65
5.2 Basi e conversioni . . . 66
5.3 Numeri naturali e overflow . . . 67
5.4 Interi con segno . . . 69
5.5 Ritorno alla macchina . . . 73
5.6 Verificate di aver capito . . . 75
5.7 Esercizi . . . 76
5.8 Glossario . . . 77
5.9 Conclusioni e letture . . . 77

6 Rappresentazione dei numeri: i reali 79
6.1 Riscaldamento sulla macchina . . . 79
6.2 Altre codifiche per gli interi con segno . . . 80
6.3 Virgola fissa . . . 81
6.4 Virgola mobile . . . 83
6.5 Ritorno alla macchina . . . 86
6.6 Verificate di aver capito . . . 86
6.7 Esercizi . . . 87
6.8 Glossario . . . 88
6.9 Conclusioni e letture . . . 88

III Logica **89**

IV Stato e memoria **91**

V La macchina **93**

VI Pubblicazione e prospettive **95**

VII Appendici **97**

Prefazione

La maggior parte dei corsi di architettura dei calcolatori è costruita dal basso verso l'alto. Si comincia dal bit, si sale attraverso i sistemi di numerazione, l'algebra di Boole e le reti logiche, si assembla un processore a partire dalle porte, e solo nelle ultime settimane si lascia che lo studente scriva un programma per la macchina che i mesi precedenti hanno descritto. L'ordine è logicamente impeccabile, e per molti studenti del primo anno è anche la ragione per cui la materia sembra lontana: per dieci settimane ogni risposta arriva prima della sua domanda.

Questo libro è la forma scritta di un corso che rovescia l'ordine senza rinunciare in nulla al rigore. *Prima programmare, poi formalizzare*: nella prima settimana il lettore installa una piccola console, scrive una dozzina di righe di assembly RISC-V e vede un rettangolo muoversi su uno schermo. Da quel momento ogni argomento formale è introdotto da qualcosa che il programma del lettore ha già fatto. Il complemento a due arriva dopo che una posizione è uscita dal bordo sinistro dello schermo e il debugger ha mostrato `0xFFFFFFFF`. L'algebra di Boole arriva dopo che una maschera è servita a chiedere se un pulsante è premuto. Il flip-flop arriva dopo che un contatore è sopravvissuto da un frame al successivo, e il ciclo dell'istruzione dopo che il lettore ha premuto **Step** qualche centinaio di volte e ha visto il program counter avanzare di quattro.

Dal programma classico non viene tolto nulla. Il lettore troverà la notazione posizionale e i formati IEEE 754, assiomi e teoremi dell'algebra di Boole con le loro dimostrazioni, forme canoniche e mappe di Karnaugh, il transistor come interruttore, sommatori, multiplexer e decodificatori, latch e flip-flop, macchine a stati finiti, registri e bus, gerarchie di memoria e memorie permanenti, il datapath e l'unità di controllo, le interruzioni, la scheda video e le periferiche. Cambia la direzione del ragionamento: ogni capitolo si apre sulla macchina, la lascia per tutto il tempo che la teoria richiede, e vi ritorna con una sezione intitolata *Ritorno alla macchina*, in cui le stesse regole vengono rilette nelle istruzioni di un processore vero.

La macchina

Il processore è un core RISC-V: l'insieme di istruzioni RV32IMAC, così come lo realizza l'ESP32-C6 di Espressif. RISC-V è uno standard aperto, abbastanza piccolo da essere imparato per intero in un semestre e abbastanza reale da trovarsi in miliardi di dispositivi; sostituisce qui il Motorola 68000 delle edizioni precedenti del corso, che sopravvive solo in qualche confronto storico.

L'ambiente è PRG32, una piattaforma didattica aperta sviluppata all'Università degli Studi di Napoli «Parthenope». Un programma PRG32 è una *cartuccia*: un piccolo file di codice macchina nativo che fornisce tre funzioni, una chiamata una sola volta e due chiamate a

ogni frame. La stessa cartuccia gira su un kit su breadboard, sull'emulatore QEMU e su PRG32-QT, un'applicazione desktop il cui debugger integrato mostra i trentadue registri e la memoria mentre il programma viene eseguito un'istruzione alla volta. Per seguire il libro non serve una scheda: basta un portatile.

Il progetto

Una piattaforma usata solo per gli esercizi è una palestra. PRG32 è anche un'officina, e il libro ha un progetto che lo attraversa: Verm1, che si legge *vermone*, un omaggio a Snake scritto a partire da un file vuoto in assembly RISC-V. Dopo ogni blocco di quattro capitoli, un capitolo tutorial aggiunge una parte del gioco usando soltanto ciò che quei quattro capitoli hanno insegnato: prima una testa che attraversa un mondo senza bordi, poi la sterzata, il cibo e un punteggio, poi un corpo che cresce e le regole che chiudono una partita, poi cifre, livelli e suono. Un ultimo tutorial confeziona il gioco finito per il Cartridge Store di PRG32. Ogni tutorial stampa ogni riga del programma, dice quali righe sono nuove e quali sono cambiate, e si ferma a punti di controllo in cui il programma deve girare e comportarsi in un modo dichiarato.

Che cosa significa «verificato» in questo libro

Un listato che non si assembla insegna la lezione sbagliata. Il codice assembly di questo libro non è battuto dentro il testo: è letto da file che vengono assemblati, eseguiti sul QEMU ufficiale e sul runner senza interfaccia di PRG32-QT, e confrontati con i risultati attesi da test automatici; le quattro parti di Verm1 sono giocate da un giocatore programmato che scatta anche le schermate stampate nei capitoli tutorial. Dove un frammento non ha potuto essere eseguito sulla macchina dell'autore (i frammenti CUDA e Verilog dell'ultimo capitolo, i passi di installazione per sistemi operativi diversi da quello usato per scrivere il libro) il testo lo dice.

Ringraziamenti

Mariacarla Staffa tiene la metà di laboratorio del corso, in cui gli esercizi di questo libro si svolgono con matita, carta e tastiera. Ivan Cafiero e Simone Boscaglia hanno contribuito a PRG32 da studenti e hanno mostrato che la piattaforma può essere capita, ed estesa, dalle persone per cui è pensata. Giuseppe Coviello ha contribuito l'implementazione di riferimento di Verm1: i quattro programmi che i capitoli tutorial costruiscono riga per riga, e che i test automatici giocano, sono suoi. Gli studenti delle edizioni precedenti del corso hanno scritto, con le loro domande, la maggior parte dei paragrafi che cominciano con «Attenzione».

Raffaele Montella
Napoli, ottobre 2026

Come usare questo libro

Un capitolo, una lezione

Il libro ha venticinque capitoli, numerati da 0 a 24 come le lezioni del corso, e come quasi tutto il resto in questo libro: bit, registri, righe e colonne partono tutti da zero. Il Capitolo N segue la presentazione della Lezione N: l'ordine delle sezioni è l'ordine delle slide, i listati sono gli stessi file, le domande alla fine sono quelle mostrate in aula. Chi ha seguito la lezione trova qui le frasi per cui in una slide non c'è spazio; chi l'ha persa trova la lezione.

Parte	**Capitoli**	**Poi, il progetto**
Partenza	0 Introduzione, 1 Installazione, 2 Primo contatto	
Informazione	3 Storia, 4 Informazione digitale, 5 Interi, 6 Reali	7 Verm1, parte 1
Logica	8–9 Algebra di Boole, 10 Elettronica, 11 Macchine combinatorie	12 Verm1, parte 2
Stato e memoria	13 Macchine sequenziali, 14 Registri e bus, 15 Memoria, 16 Memoria permanente	17 Verm1, parte 3
La macchina	18 Processore, 19 Istruzioni e interruzioni, 20 Scheda video, 21 Ingresso e uscita	22 Verm1, parte 4
Pubblicazione, prospettive	24 Oltre un solo core	23 Verm1 sul Cartridge Store

La forma di un capitolo di teoria

Ogni capitolo che presenta un argomento ha gli stessi cinque movimenti.

1. **Riscaldamento sulla macchina.** Poche righe di RISC-V su PRG32 che producono un fatto che chiede una spiegazione. Eseguitele prima di proseguire.
2. **La trattazione formale.** Definizioni, teoremi, metodi ed esempi svolti.
3. **Ritorno alla macchina.** Le stesse regole, come le realizza RISC-V.
4. **Verificate di aver capito.** Domande a risposta multipla nello stile della prova intercorso. Le risposte, ciascuna con una riga di spiegazione, sono nell'Appendice **??**: leggetele dopo aver risposto, non al posto di rispondere.
5. **Esercizi, glossario, letture.** Gli esercizi si svolgono prima su carta e poi su PRG32; il glossario affianca a ogni termine inglese quello italiano, perché la documentazione tecnica, i messaggi degli strumenti e i commenti dei listati sono in inglese.

La forma di un capitolo tutorial

I Capitoli 7, 12, 17, 22 e 23 costruiscono Verm1. Sono autonomi e pensati per essere seguiti alla tastiera. Ogni passo mostra poche righe, dice se sono **nuove**, **modificate** o **invariate**, e finisce con un punto di controllo *Eseguite* che dichiara che cosa deve mostrare lo schermo. Se all'inizio di un tutorial un programma si rifiuta di funzionare, ripartite dal file di riferimento della parte precedente (Appendice **??**): nessuno deve restare indietro per un errore.

I riquadri

Concetto

Un riquadro **Concetto** enuncia un'idea da cui dipende il resto del capitolo.

Definizione 0.1 (come appaiono le definizioni)

Definizioni formali e teoremi sono numerati all'interno del capitolo.

Provate voi

Un riquadro **Provate voi** chiede di fare qualcosa alla tastiera, e dice che cosa si deve osservare.

Attenzione

Un riquadro **Attenzione** segnala un errore che si ripete ogni anno.

Ritorno alla macchina

Un riquadro **Ritorno alla macchina** lega un risultato formale a un'istruzione, a un registro o a una schermata del debugger.

Nota storica

Un riquadro **Nota storica** racconta da dove viene un'idea.

Notazione

I numeri senza indicazioni sono decimali, con la virgola decimale e, dove serve, il punto per le migliaia. I numeri esadecimali sono scritti come li vuole l'assemblatore, `0x07E0`; i numeri binari come `0b1010` nel codice e con un pedice, 1010_2, nelle formule. I bit si numerano da destra partendo da zero, così il bit i ha peso 2^i. Registri (`a0`, `sp`), istruzioni (`addi`) e nomi di file sono composti in carattere a spaziatura fissa. Nelle espressioni booleane il complemento di x è $\overline{x}$, la somma è $+$ e il prodotto si scrive per giustapposizione. I multipli del byte seguono i prefissi binari dove la precisione conta: 1 KiB $= 2^{10}$ byte, 1 MiB $= 2^{20}$ byte.

I listati e i loro commenti sono in inglese, come gli identificatori di PRG32, i comandi e le etichette dell'interfaccia di PRG32-QT (**Step**, **Resume**, **Memory**): sono ciò che compare davvero sullo schermo.

Le appendici

L'Appendice **??** è un riferimento completo dell'insieme di istruzioni RISC-V usato nel libro: registri, formati, ogni istruzione di RV32I e delle estensioni M, A, C e Zicsr, pseudo-istruzioni, direttive dell'assemblatore, la convenzione di chiamata e i registri di controllo del modo macchina. L'Appendice **??** è il riferimento di PRG32: il contratto, l'interfaccia di programmazione, il formato delle cartucce, gli strumenti a riga di comando, gli host e il Cartridge Store. L'Appendice **??** è un riferimento di Git e GitHub, gli strumenti con cui la piattaforma, questo libro e il progetto del lettore sono conservati e condivisi. L'Appendice **??** stampa per intero i quattro programmi di riferimento di Verm1; l'Appendice **??** le risposte a tutte le domande; l'Appendice **??** il glossario completo inglese–italiano.

Parte I

Partenza

CAPITOLO 0

Introduzione: prima programmare, poi formalizzare

"Matrix è un sistema, Neo. E quel sistema è nostro nemico."
— *Matrix*, Lana e Lilly Wachowski, 1999

In questo capitolo

- Che cos'è il corso, chi lo tiene e come sono usate le sue novantasei ore.
- Il metodo: prima viene un programma che gira, poi la sua teoria.
- La macchina: PRG32, RISC-V e il debugger PRG32-QT.
- Il progetto: Verm1, un gioco scritto da un file vuoto e pubblicato.
- Le regole: programma, risultati di apprendimento, valutazione, materiale.

Un calcolatore è l'oggetto più complicato che la maggior parte delle persone usa ogni giorno, e quello di cui meno sono incoraggiate a guardare il funzionamento interno. I suoi costruttori hanno passato settant'anni a nascondere la macchina dietro strati ciascuno più facile da usare di quello sottostante: le porte logiche dietro le istruzioni, le istruzioni dietro i linguaggi, i linguaggi dietro le librerie, le librerie dietro le applicazioni, e ora le applicazioni dietro gli assistenti conversazionali. Sono gli strati a rendere utile l'informatica. Sono anche un sistema nel senso del film citato qui sopra: ci si può vivere dentro per una vita senza sapere che sotto c'è qualcosa.

L'architettura dei calcolatori è la disciplina che guarda sotto. Studia come una macchina capace soltanto di accendere e spegnere correnti arrivi a rappresentare numeri, testo, immagini e suono; come una manciata di operazioni logiche, ripetute miliardi di volte al secondo, diventino aritmetica, decisioni e memoria; e come il tutto sia organizzato in modo che una sequenza di ordini semplicissimi, un *programma*, possa farle fare qualunque cosa sia calcolabile. Questo capitolo spiega come il corso affronta questo studio, con quale macchina e attraverso quale progetto, e poi enuncia le regole pratiche.

0.1 Il corso in breve

Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori, codice ARC12, è un insegnamento del primo anno, primo semestre, del Corso di Laurea in Informatica

dell'Università degli Studi di Napoli «Parthenope». Vale 12 CFU, appartiene al settore scientifico-disciplinare INFO-01/A (Informatica) e consiste di 96 ore, divise in parti uguali tra lezioni e laboratorio (Tabella 1).

Tabella 1. Il corso in breve.

Nome	Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori (ARC12)
Collocazione	Primo anno, primo semestre, Corso di Laurea in Informatica
Crediti	12 CFU, 96 ore; settore INFO-01/A (Informatica)
Lezioni	Raffaele Montella, 6 CFU, 48 ore
Laboratorio	Mariacarla Staffa, 6 CFU, 48 ore
Ricevimento	Stanza 420, martedì 14:00–16:00, su appuntamento; anche su Microsoft Teams (codice del team `c7rf33g`)
Crediti aggiuntivi	Accenture SuperSkills! dà 2 CFU come ulteriori conoscenze (https://richiediaccesso.uniparthenope.it)
Prerequisiti	Nessuno

Non ci sono prerequisiti formali. Non si presuppone alcuna esperienza precedente di programmazione, di elettronica o di matematica oltre il programma scolastico: l'unico vero prerequisito è la passione per l'informatica e per le tecnologie che la rendono possibile.

Le due metà del corso sono due viste della stessa materia. Nelle **lezioni** si presentano le idee: informazione, numeri, algebra di Boole, reti logiche, memoria, processore, ingresso e uscita. Ogni argomento si apre con un programma che si può eseguire e viene poi formalizzato con definizioni, metodi e dimostrazioni; ogni lezione si chiude con domande a risposta multipla nello stile della prova intercorso; e ogni quattro lezioni un tutorial aggiunge la parte successiva di Verm1, il gioco del corso. Nel **laboratorio** le stesse idee vengono esercitate: rappresentazione dei numeri e tecniche di conversione; analisi, sintesi e minimizzazione di circuiti logici; programmazione in assembly RISC-V sulla piattaforma PRG32; e, infine, il progetto dello studente, una variante di Verm1 o un gioco di sua invenzione. Questo libro è il testo delle lezioni; i suoi esercizi sono il materiale naturale del laboratorio.

0.2 Un approccio nuovo

Prima programmare. Poi formalizzare.

Fin dai primi giorni lo studente scrive assembly RISC-V che gira su un processore vero. Ogni argomento formale risponde poi a una domanda che il programma dello studente ha già sollevato.

L'ordine tradizionale della materia è deduttivo: la rappresentazione, poi la logica, poi i circuiti, poi il processore, poi, finalmente, un programma. È l'ordine in cui la macchina viene *costruita*, e ha l'eleganza di ogni costruzione dai primi principi. Non è l'ordine in cui la macchina viene *capita*. Uno studente che incontra il complemento a due nella terza settimana di un corso deduttivo non ha mai visto un numero negativo dentro un calcolatore, e non ha alcuna ragione di interessarsi a come si scrive. Uno studente che, la settimana prima, ha spostato un rettangolo verso sinistra finché la sua coordinata, mostrata dal debugger, è passata da `0x00000000` a `0xFFFFFFFF` ha una domanda, e il complemento a due è la risposta.

Il corso è quindi organizzato come una spirale, ripetuta per ogni argomento (Figura 1).

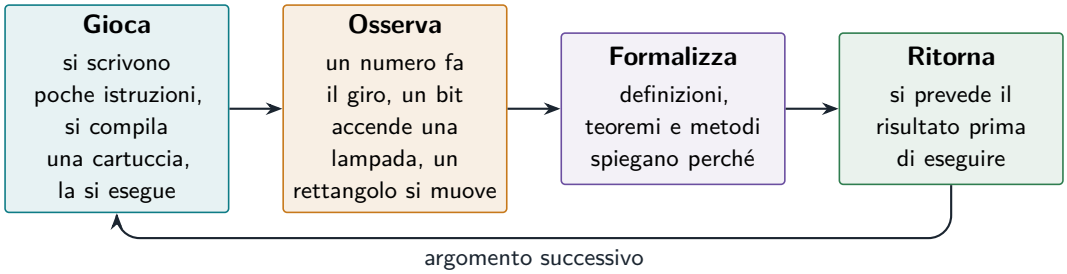


Figura 1. La spirale di apprendimento, ripetuta per ogni argomento del corso.

Prima si *gioca*: si scrivono poche istruzioni, si compila e si esegue una cartuccia. Poi si *osserva*: succede qualcosa che non ci si aspettava, oppure che ci si aspettava e che ora si può vedere. Poi si *formalizza*: definizioni e teoremi spiegano esattamente perché. Infine si *ritorna* alla macchina, e la misura della comprensione è la capacità di prevedere un risultato prima di eseguire il programma. Un voto è un'opinione; una racchetta che esce dallo schermo è un fatto. Il corso usa i fatti, e poi li spiega.

0.2.1 Gli obiettivi del corso

Gli obiettivi sono quelli di ogni corso serio sulla materia; cambia solo il percorso. Il corso

- presenta gli aspetti fondamentali dell'architettura dei calcolatori moderni;
- usa, in laboratorio, i concetti di base necessari a programmarli in modo efficiente con linguaggi di basso livello: linguaggio macchina e assembly;
- tratta con rigore gli strumenti con cui si costruisce un processore, cioè l'algebra di Boole con la sintesi di reti logiche combinatorie e sequenziali, e la rappresentazione binaria con l'aritmetica binaria;
- descrive in dettaglio la struttura funzionale e fisica della CPU, della memoria e del sottosistema di ingresso/uscita;
- sviluppa codice assembly che rende visibile come lavora davvero un processore e come interagiscono le unità fondamentali di un calcolatore.

0.3 La macchina: PRG32 e RISC-V

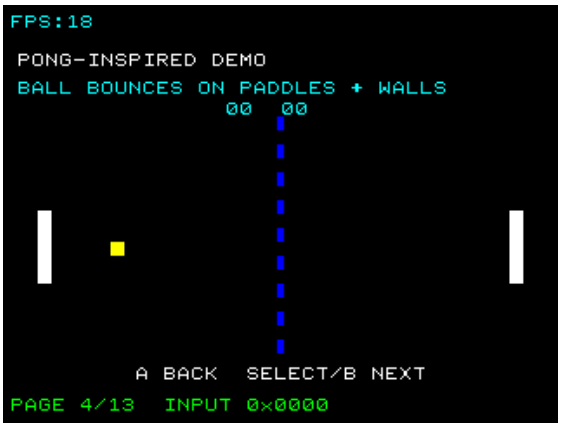
0.3.1 Una console rétro programmata in assembly

PRG32 è un runtime didattico open source per insegnare l'assembly RISC-V e il C, sviluppato all'Università degli Studi di Napoli «Parthenope» (Montella et al., 2026a). All'occhio è una piccola console per videogiochi del tipo che si vendeva nei primi anni Ottanta (Figura 2); per il programmatore è un contratto volutamente sottile.

- Il codice dello studente gira in modo *nativo*, come codice macchina RISC-V a 32 bit dell'insieme di istruzioni RV32IMAC. Non è interpretato da un simulatore didattico: le istruzioni scritte sono le istruzioni eseguite.
- Il contratto consiste di tre funzioni scritte dallo studente: `init` gira una volta; `update` e `draw` girano una volta per frame.
- I giochi viaggiano come piccole *cartucce*, file con estensione `.prg32`.



(a) Il menu del firmware residente.



(b) Pong: codice macchina RISC-V nativo.

Figura 2. PRG32, la macchina del corso.

- La macchina offre un viewport di 320 per 200 pixel, un joystick con pochi pulsanti, e il suono: abbastanza piccola da tenerla tutta in mente, abbastanza ricca da mettere alla prova l'intero corso.

0.3.2 La filosofia di PRG32

Sei principi, presi dal libro di testo della piattaforma (Montella et al., 2026b), danno forma sia a PRG32 sia al modo in cui questo libro lo usa.

Abbastanza piccola da capirla.
Un processore, uno schermo di 320 per 200 pixel, un joystick, un piccolo altoparlante. Nulla è nascosto: l'intera macchina si può tenere in mente.

Vera, non simulata.
Le istruzioni scritte sono quelle che il chip esegue. Ciò che si impara qui è vero su ogni processore RISC-V, da un sensore a un supercalcolatore.

I limiti rendono creativi.
Pochi kilobyte e tre funzioni. I giochi degli anni Ottanta sono nati da limiti come questi: un vincolo è un invito.

Una cosa alla volta.
Si comincia con funzioni che si limitano a ritornare. Si aggiunge una cosa, si esegue, si guarda. `update` cambia lo stato, `draw` lo mostra. Il suono viene per ultimo.

Guardare dentro.
Quando qualcosa sorprende, si ferma il programma e si leggono registri e memoria. Il debugger fa parte del metodo, non è uno strumento d'emergenza.

Fare, poi condividere.
Una cartuccia è un file che si può dare a un amico. Il Cartridge Store è lo scaffale su cui il lavoro finito incontra i suoi giocatori.

La piattaforma ha quindi due usi nel corso. È una *palestra*: ogni capitolo si apre con un breve programma che gira e si chiude tornando alla macchina; un esercizio consiste di una previsione e di una verifica; e la macchina non lusinga mai, perché una maschera sbagliata accende la lampada sbagliata. Ed è un'*officina*: una cartuccia vuota e uno schermo, su cui regole, colori, suoni e livelli sono decisioni di progetto che spettano all'autore. Il progetto del corso comincia guidato e finisce libero, e ciò che è finito si può pubblicare. Gli studenti

del primo anno di rado si aspettano che un corso di architettura chieda loro un'idea; questo lo fa.

0.3.3 PRG32-QT: il debugger del corso

PRG32-QT è PRG32 su un portatile: un'applicazione che esegue le stesse cartucce senza scheda e senza compilatore (Montella, 2026). In modalità debug apre la macchina (Figura 3): il programma può essere messo in pausa, ripreso e fatto avanzare di un'istruzione alla volta; si possono mettere breakpoint su qualunque istruzione di `init`, `update` o `draw`; vengono mostrati i trentadue registri e 128 byte di memoria a qualunque indirizzo; la velocità si può regolare tra 0,01 e 4 volte quella reale; e dei contatori riportano i frame, le istruzioni eseguite e i frame arrivati in ritardo.

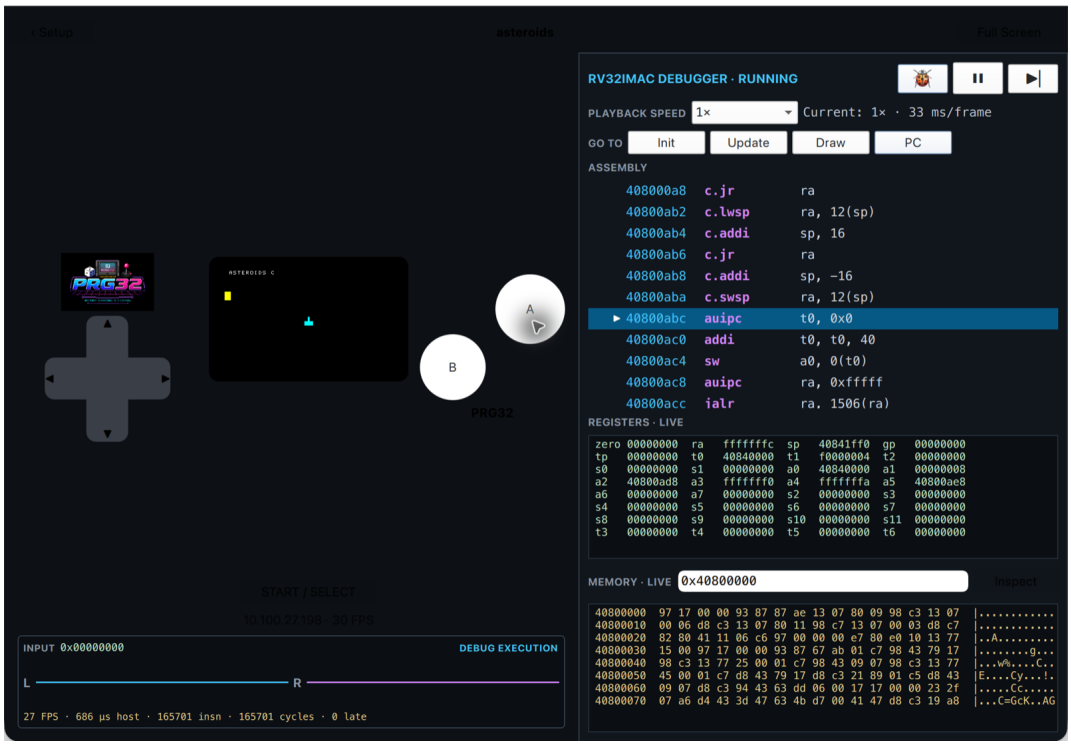


Figura 3. PRG32-QT in modalità debug: a sinistra il gioco, a destra la macchina.

Il debugger si usa in ogni capitolo: per verificare una previsione, per trovare un errore, per misurare. Fornisce anche la definizione operativa di comprensione adottata in questo libro.

Quando si è capito un programma?

Un programma è capito quando si sa dire che cosa conterrà ciascun registro *prima* di premere **Step**.

0.3.4 Perché RISC-V?

La scelta del processore non è neutra, e quattro ragioni hanno portato a RISC-V.

È aperto.
RISC-V è un insieme di istruzioni a standard aperto: chiunque può progettare, costruire

e vendere un processore RISC-V senza pagare una licenza per l'architettura (RISC-V International, 2024).

È piccolo.
L'insieme di istruzioni intero di base ha una quarantina di istruzioni e trentadue registri. Lo si può imparare tutto in un semestre, e l'Appendice **??** lo elenca tutto.

È reale.
`addi`, `lw`, `sw` e `beqz` non sono un dialetto didattico. L'industria consegna core RISC-V a miliardi, ed essi eseguono esattamente queste istruzioni.

È europeo.
L'Europa considera RISC-V una leva per la sovranità tecnologica: lo European Chips Act e progetti come DARE (Digital Autonomy with RISC-V in Europe, un progetto della EuroHPC Joint Undertaking) costruiscono processori su di esso (EuroHPC Joint Undertaking, 2025).

0.3.5 La mappa: ogni argomento si incontra due volte

La Tabella 2 è il corso in una pagina. Ogni argomento formale della colonna di sinistra si incontra prima sulla macchina, nella forma data dalla colonna di destra, e solo dopo nella sua forma generale. Vale la pena di tornare a questa tabella all'inizio di ogni parte del libro.

Tabella 2. Ogni argomento formale si incontra prima su PRG32.

Argomento formale	**Incontrato prima su PRG32 come…**
Notazione posizionale, esadecimale	il colore `0x07E0` e il valore mostrato dal debugger
Complemento a due, overflow	una posizione che va a sinistra oltre lo zero e mostra `0xFFFFFFFF`
Virgola fissa e virgola mobile	una palla che si muove di 0,375 pixel per frame su un chip senza unità in virgola mobile
Algebra di Boole, maschere di bit	`andi t3, t0, 1` per verificare se **LEFT** è premuto
Elettronica, porte logiche	il pulsante che collega un piedino del chip a massa
Reti combinatorie, sommatori	che cosa fa davvero `add` a trentadue bit
Reti sequenziali, registri	il contatore che sopravvive da un frame al successivo
CPU, datapath, ciclo dell'istruzione	l'esecuzione passo-passo di un'istruzione in PRG32-QT
Memoria e ingresso/uscita	`lw` e `sw`, lo stack, il joystick e il display

0.4 Il progetto: Verm1

Verm1, che si legge *vermone*, è un omaggio a Snake: il gioco del cabinato *Blockade* (1976) e dei telefoni Nokia (1997). Un verme si muove su una griglia, mangia, cresce, e non deve mordersi. Lo studente ne scrive ogni riga in assembly RISC-V, senza motore e senza librerie oltre alle chiamate di PRG32. Il gioco è abbastanza piccolo per uno studente del primo anno e abbastanza ricco da richiedere ogni argomento del corso; alla fine è pronto per il Cartridge Store con sopra il nome del suo autore. L'implementazione di riferimento, con cui ogni lettore può confrontare il proprio programma, è un contributo di Giuseppe Coviello (Appendice **??**).

Il progetto è il filo del libro. Si costruisce in quattro parti e una pubblicazione (Tabella 3, Figura 4), e ogni parte usa esattamente ciò che i quattro capitoli precedenti hanno insegnato.

Ogni parte finisce con un gioco che gira: il lettore non torna mai a casa con un programma rotto.

Tabella 3. Verm1: quattro parti e una pubblicazione.

Capitolo	Dopo la parte su	Verm1 acquista	Con
?? Parte 1	informazione e numeri	una griglia, una testa che si muove, un mondo senza bordi	colori, scorrimenti, virgola fissa
?? Parte 2	logica e circuiti	la sterzata, il cibo, un punteggio	maschere, XOR, tabelle
?? Parte 3	stato e memoria	un corpo che cresce, le collisioni, tre schermate	una macchina a stati, vettori
?? Parte 4	la macchina	cifre, livelli, suono, disegno veloce	chiamate, divisione, il frame buffer
?? Lo Store	l'intero corso	un nome, un'icona, i crediti, un bundle	gli strumenti di PRG32



(a) Parte 1: una testa **(b)** Parte 2: cibo, punteggio **(c)** Parte 3: un corpo **(d)** Parte 4: finito

Figura 4. Verm1 cresce con il lettore: lo stesso file, quattro sedute. Le schermate sono prese dai programmi di riferimento dai test automatici del materiale del corso.

Un capitolo tutorial obbedisce a quattro regole. È *autonomo*: mostra l'intero programma, e ogni riga è nuova, modificata, oppure ristampata invariata. Procede *passo per passo*: poche righe alla volta, e dopo ogni passo il programma viene compilato, eseguito e confrontato con un risultato dichiarato. Usa *il debugger*: ogni tutorial ha una sessione in PRG32-QT, con un breakpoint, un registro da osservare e una regione di memoria da leggere. E il suo risultato appartiene al lettore: ogni tutorial finisce con modifiche da fare in autonomia, perché il gioco che viene pubblicato non dovrebbe essere identico a quello stampato qui.

0.5 Il corso in blocchi

L'ordine degli argomenti è stato scelto per l'apprendimento. Prima viene che cosa significa un bit; poi come i bit si combinano; poi come si ricordano; poi la macchina che fa tutte e tre le cose. Il processore è presentato prima dei dispositivi che pilota. Dopo ogni blocco il progetto lo usa. L'ultima lezione è uno sguardo in avanti, tenuta prima di una visita ai sistemi di calcolo del Dipartimento (Tabella 4).

0.6 Programma e risultati di apprendimento

0.6.1 Programma

Organizzazione del calcolatore.

Hardware e software; software applicativo e di sistema; il modello di von Neumann;

Tabella 4. I blocchi del corso. I numeri sono quelli delle lezioni e dei capitoli di questo libro.

Blocco	Lezioni	Poi
Partenza	0 Introduzione; 1 PRG32 e PRG32-QT: installazione; 2 Primo contatto con RISC-V	
Informazione	3 Storia; 4 Informazione digitale; 5 Interi; 6 Reali	7 Verm1, parte 1
Logica	8 e 9 Algebra di Boole; 10 Elettronica; 11 Macchine combinatorie	12 Verm1, parte 2
Stato e memoria	13 Macchine sequenziali; 14 Registri e bus; 15 Memoria; 16 Memoria permanente	17 Verm1, parte 3
La macchina	18 Processore; 19 Istruzioni e interruzioni; 20 Scheda video; 21 Ingresso e uscita	22 Verm1, parte 4
Pubblicazione		23 Verm1 sul Cartridge Store
Prospettive	24 Oltre un solo core: RISC-V superscalare, HPC, GPU e FPGA	la visita ai sistemi di calcolo

principi di funzionamento di un calcolatore.

Algebra di Boole e logica.
Funzioni booleane, tabelle di verità; reti combinatorie e loro minimizzazione; sommatori; macchine a stati finiti; reti sequenziali; flip-flop, registri.

Rappresentazione dei dati.
Codifica; codici ridondanti, rilevazione e correzione degli errori; decodificatori, multiplexer; sistemi posizionali; naturali, interi, reali; aritmetica.

Struttura di un calcolatore.
La CPU: unità di controllo, ALU, registri; il ciclo dell'istruzione; il datapath; la memoria centrale; il sottosistema di I/O; architetture microprogrammate.

Assembly RISC-V.
Insieme di istruzioni; registri, indirizzamento; aritmetica e logica; salti; sottoprogrammi, convenzione di chiamata, stack; memoria e I/O.

Progetto.
Verm1: un gioco in assembly RISC-V su PRG32, costruito in quattro parti guidate, poi variato, documentato e confezionato dallo studente o da un piccolo gruppo.

Il programma dettagliato è pubblicato sul sistema informativo dell'ateneo e sul portale di e-learning, https://elearning.uniparthenope.it.

0.6.2 Risultati di apprendimento

I risultati sono enunciati, come d'uso nell'istruzione superiore europea, secondo i cinque descrittori di Dublino.

Conoscenza e capacità di comprensione.
Una solida conoscenza del sistema binario, della logica digitale e dei principi di funzionamento dei calcolatori. Non si presuppone alcuna formazione informatica precedente.

Capacità di applicare conoscenza e comprensione.
La capacità di applicare la teoria a esercizi e a programmi in assembly RISC-V, e di riconoscere sia le capacità sia i limiti dei calcolatori su problemi reali.

Autonomia di giudizio.
La capacità di scrivere programmi assembly che rappresentano ed elaborano in binario problemi reali, scegliendo rappresentazioni dei dati e strutture di controllo adeguate.

Abilità comunicative.
La capacità di partecipare attivamente a gruppi che progettano, realizzano e documentano applicazioni assembly, usando una terminologia tecnica corretta.

Capacità di apprendimento.
La capacità di collegare rappresentazione, reti logiche e processori in un unico quadro, e di aggiornarsi in autonomia da fonti tecniche affidabili.

0.7 Organizzazione e valutazione

Questa sezione riporta le regole dell'anno accademico 2026/27. Le date non ancora fissate al momento della compilazione del libro vengono comunicate in aula e sul portale di e-learning, che in caso di differenze prevalgono su questo testo.

0.7.1 Orario

Da settembre a dicembre 2026 il corso si tiene quattro volte alla settimana in Aula 1: lezioni il lunedì (14:00–16:00) e il mercoledì (09:00–11:00), laboratorio il martedì e il venerdì (09:00–11:00). Un calendario interno con l'effettiva distribuzione di lezioni e sessioni di laboratorio è condiviso con gli studenti.

0.7.2 Una prova intercorso

C'è una sola prova intercorso, svolta in presenza e su carta in circa 45 minuti: **dieci domande a risposta multipla più due esercizi**. Si tiene di lunedì, 14:00–16:00, in Aula 1, in una data che sarà comunicata; gli studenti possono essere divisi in gruppi e la prova svolta a turni. I candidati portano un documento di identità e una penna, e devono conoscere il proprio indirizzo di posta istituzionale e il numero di matricola.

La prova valuta entrambe le parti, studenti e docenti: serve a sapere dove sta andando il corso. Chiude i primi tre blocchi (Tabella 5); il quarto, la macchina, si esamina all'orale. Negli esempi della prova compaiono istruzioni RISC-V: il materiale del primo contatto del Capitolo 2 si dà per noto, e il codice di Verm1, parti da 1 a 3, è una fonte naturale di domande. Ogni capitolo di questo libro finisce con domande dello stesso tipo, e la preparazione migliore è rispondere settimana per settimana.

Tabella 5. Che cosa copre la prova intercorso.

Blocco	Lezioni	Argomenti
Informazione	da 3 a 6	storia della computazione; informazione digitale; rappresentazione dei numeri: interi e reali
Logica	da 8 a 11	algebra di Boole; elettronica e porte logiche; macchine combinatorie
Stato e memoria	da 13 a 16	macchine sequenziali: latch, flip-flop, macchine a stati; registri e bus; memoria e memoria permanente
Non nella prova	da 18 a 24	il processore, istruzioni e interruzioni, la scheda video, ingresso e uscita, le prospettive: parte dell'esame orale

Niente tecnologia. Niente GPT.

La prova intercorso e l'esame sono *#nogpt*. Non è ammesso alcun aiuto tecnologico o umano di alcun tipo: niente dispositivi, niente ChatGPT o altri modelli linguistici, niente IA generativa. Copiare annulla subito la prova; comportamenti più gravi hanno conseguenze più gravi.

0.7.3 Valutazione

L'esame ha due parti, ed entrambe vanno superate.

- **Teoria (50%)**: un esame orale con domande estemporanee su tutto il programma; alcune risposte richiedono uno schema o un calcolo alla lavagna. L'orale può anche abbassare il voto ottenuto nella prova intercorso.
- **Pratica (50%)**: l'esame di laboratorio con la prof.ssa Staffa.

Il voto finale è la media del voto di teoria e del voto di pratica. I risultati della prova intercorso e degli esami valgono entro l'anno accademico corrente.

Per verbalizzare l'esame serve un voto positivo sia in teoria sia in pratica, e la verbalizzazione avviene solo durante un appello di teoria. L'esame di laboratorio si tiene di solito una settimana prima dell'appello di teoria, ed entrambi si prenotano su https://uniparthenope.esse3.cineca.it. Due domande vengono poste ogni anno. *Si può sostenere l'esame di teoria prima di aver superato quello pratico?* Sì: il risultato vale per un anno accademico, ma l'esame viene verbalizzato solo dopo aver superato la parte pratica. *Ci si può prenotare all'appello di teoria senza la prova pratica?* Sì, il caso è lo stesso.

0.7.4 La settimana della prova e l'evento finale

Durante la settimana della prova intercorso le lezioni sono sostituite da seminari sulle architetture di calcolo emergenti. Alla fine del corso, uno studente o un gruppo che presenta il proprio gioco alla cerimonia di chiusura, una variante di Verm1 o un gioco originale, può ottenere punti bonus per l'esame. La mattinata conclusiva ha due metà: l'ultima lezione, oltre un solo core, da RISC-V superscalare ai supercalcolatori (Capitolo **??**), e una visita ai sistemi di supercalcolo del Dipartimento di Scienze e Tecnologie. I giochi finiti si possono pubblicare sul Cartridge Store di PRG32, come mostra il Capitolo **??**.

0.8 Materiale

Slide.
Le presentazioni che questo libro segue sono sempre disponibili in una cartella condivisa di Google Drive e vengono pubblicate periodicamente su https://elearning.uniparthenope.it. Sono generate da sorgente: correzioni e nuovi esercizi sono benvenuti come contributi.

Piattaforma.
PRG32, libero e open source, su https://github.com/riscv-prg32/PRG32, con il suo libro di testo, *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform* (Montella et al., 2026b).

Libri di testo.
Patterson e Hennessy (2020), *Computer Organization and Design, RISC-V Edition* (in

italiano: *Struttura e progetto dei calcolatori*), e Tanenbaum e Austin (2013), *Structured Computer Organization* (in italiano: *Architettura dei calcolatori. Un approccio strutturale*), sono i due riferimenti del corso. Il programma elenca anche Stallings (2021) e Bucci (2017).

Standard.
La documentazione ufficiale di RISC-V su https://riscv.org (RISC-V International, 2024).

Il materiale didattico vive in una cartella condivisa con gli studenti, che possono contribuirvi; per ottenere l'accesso lo studente invia al docente l'indirizzo di un account collegato al servizio. Ogni altra comunicazione con il docente passa per l'indirizzo istituzionale dello studente.

Cineforum

Trenta film e serie sull'informatica accompagnano il corso, per ricordare che la materia ha una cultura oltre che una tecnica: *The Social Network* (2010), *Steve Jobs* (2015), *Pirates of Silicon Valley* (1999), *The Imitation Game* (2014), *Hackers* (1995), *Ex Machina* (2014), *Her* (2013), *Tron* (1982), *Tron: Legacy* (2010), *The Matrix* (1999), *WarGames* (1983), *Sneakers* (1992), *Jobs* (2013), *Antitrust* (2001), *Blackhat* (2015), *The Fifth Estate* (2013), *The Circle* (2017), *Silicon Valley* (2014–2019), *Source Code* (2011), *Chappie* (2015), *Mr. Robot* (2015–2019), *Ghost in the Shell* (1995), *Minority Report* (2002), *Transcendence* (2014), *Ready Player One* (2018), *A.I. Artificial Intelligence* (2001), *Revolt* (2017), *The Thirteenth Floor* (1999), *Eagle Eye* (2008) e *Terminator 2: Judgment Day* (1991). *Ready Player One* è anche il riferimento culturale del libro di PRG32: la dimestichezza con gli strati fondamentali è un'alfabetizzazione che conserva il suo valore.

0.9 Primo contatto: chi siete?

La prima lezione finisce con delle domande rivolte all'aula, e un lettore che lavora da solo può rivolgerle a sé stesso. Non servono a selezionare, ma a capire da dove cominciare.

- **Hardware.** Avete mai assemblato un calcolatore? Avete giocato con un Arduino o un ESP32?
- **Software.** Quanti linguaggi di programmazione conoscete? Avete mai letto una riga di assembly?
- **Cultura.** Che cosa sono il retrocomputing e il retrogaming? Avete mai visitato un museo dell'informatica?
- **Concetti.** Sapete dire la differenza tra ROM, RAM e memoria di massa? Che cos'è il cloud computing? E l'Internet delle cose?
- **RISC-V.** Avete sentito parlare di RISC-V? Quale processore c'è nel vostro telefono, nel vostro portatile, nel vostro router?

In aula le domande sono seguite da un breve questionario anonimo, il *Cappello Parlante*, su ciò che attrae di più ciascuno studente nel futuro dell'informatica e della scienza; i risultati vengono discussi e usati per calibrare esempi e progetti.

0.10 Prima del prossimo capitolo

Il Capitolo 1 installa tutto, passo per passo, su Windows, macOS e Linux. Si va più in fretta se i download lunghi sono già fatti.

Esercizio 0.1 (preparazione)

1. Procuratevi un portatile su cui vi sia permesso installare programmi.
2. Scaricate l'installer di PRG32-QT per il vostro sistema dalla pagina *Releases* di https://github.com/riscv-prg32/PRG32-QT.
3. Con una buona connessione, avviate il download lungo: ESP-IDF, seguendo il `README` di https://github.com/riscv-prg32/PRG32 per il vostro sistema.
4. Se qualcosa fallisce, non combattete da soli: conservate il messaggio di errore.

Risultato atteso. Cominciate il Capitolo 1 con i download fatti. In due ore avete PRG32-QT in funzione e una cartuccia compilata sul vostro portatile.

CAPITOLO 1

PRG32 e PRG32-QT: allestire l'officina

"Datemi sei ore per abbattere un albero, e passerò le prime quattro ad affilare l'ascia."
— attribuita ad Abraham Lincoln

In questo capitolo

Alla fine di questo capitolo avrete

- un quadro chiaro dei pezzi di PRG32: runtime, cartuccia, strumenti, host, Store;
- PRG32-QT installato sul vostro portatile, per eseguire qualunque cartuccia e guardarci dentro;
- la toolchain RISC-V installata e verificata, per compilare cartucce vostre;
- una cartuccia compilata da voi, in esecuzione in PRG32-QT e fermata nel debugger;
- un elenco di che cosa fare quando qualcosa non funziona.

Il primo lavoro di un artigiano in una nuova officina è mettere gli attrezzi dove la mano se li aspetta. Questo capitolo è quel lavoro. Non contiene teoria, ed è l'unico capitolo del libro il cui contenuto invecchierà: i numeri di versione sono quelli di ottobre 2026, e prima di seguire i passi conviene controllare le pagine dei rilasci dei progetti. Tutto il resto del libro dipende dall'ultima riga del riepilogo qui sopra: il ciclo che consiste nel compilare una cartuccia, inviarla a un host e fermarla nel debugger si ripete in ogni capitolo che segue.

I comandi della Sezione 1.5 sono stati eseguiti per questo libro su macOS con PRG32-QT. Le installazioni per Windows e Linux seguono le istruzioni ufficiali dei progetti e non sono state eseguite per questo testo.

1.1 Quattro livelli: installate ciò che vi serve

Non tutto va installato, e non tutto subito. La Tabella 1.1 ordina i pezzi secondo la necessità. Il corso, e il libro, si possono seguire con i soli livelli 1 e 2: *non serve alcuna scheda.*

Il livello 1 richiede cinque minuti. Il livello 2 è un download voluminoso e richiede da mezz'ora a un'ora: conviene farlo a casa, con una buona connessione. I livelli 3 e 4 sono per i curiosi, e per il laboratorio.

Tabella 1.1. Quattro livelli di installazione.

Livello	Installate	Potete	Serve
1	PRG32-QT	eseguire cartucce, scaricarle dallo Store, usare il debugger	a tutti, subito
2	ESP-IDF e il repository di PRG32	compilare cartucce vostre da assembly o C	a tutti, prima del Capitolo 2
3	l'emulatore QEMU	eseguire il firmware vero sul portatile	facoltativo
4	il kit: una scheda ESP32-C6	eseguire sull'hardware vero	facoltativo

1.2 La piattaforma, in dettaglio

1.2.1 Un runtime, un formato di file, un insieme di strumenti

Il nome PRG32 copre quattro cose.

- Un **firmware residente**, detto anche *runtime*: il programma che possiede lo schermo, i pulsanti e il suono.
- Un **formato di cartuccia**: un gioco come un unico piccolo file, `.prg32`.
- Uno **strumento** sul portatile: il comando `python3 -m prg32`, che compila, carica e confeziona.
- Un **Cartridge Store**, dove si condividono i giochi finiti.

Il codice di una cartuccia gira come vero codice macchina RISC-V a 32 bit, RV32IMAC, e il contratto tra un gioco e il runtime sono tre funzioni: `init`, `update` e `draw`. Il runtime chiama `init` una volta e poi `update` e `draw` a ogni frame; il Capitolo 2 parte da questo contratto.

1.2.2 Da un file sorgente allo schermo

La Figura 1.1 segue un programma dall'editor allo schermo. Il sorgente, in assembly o in C, viene tradotto in codice macchina dalla toolchain RISC-V; il compilatore è `riscv32-esp-elf-gcc`, installato da ESP-IDF, il kit di sviluppo di Espressif. Il risultato viene impacchettato in una cartuccia, fatta di codice, dati e un'intestazione. La cartuccia viene inviata attraverso la rete a un *host*, dove il runtime la carica e comincia a chiamare le sue tre funzioni.

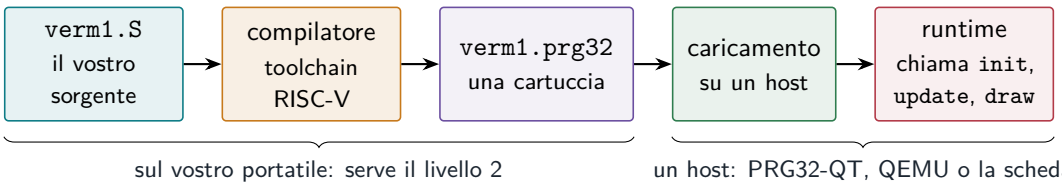


Figura 1.1. Da un file sorgente allo schermo.

Una cartuccia viene caricata all'indirizzo `0x40800000` e raggiunge il runtime soltanto attraverso una tabella di funzioni, l'*application binary interface* (ABI). È la ragione per cui lo stesso file gira su tre host diversi: non tocca mai direttamente l'hardware, e ogni host fornisce la stessa tabella.

1.2.3 Il kit: una console su breadboard

L'hardware di PRG32 (Figura 1.2) è costruito attorno a un ESP32-C6 di Espressif: un core RISC-V a 160 MHz, con Wi-Fi. Un display a colori da 2,8 pollici di 320 per 240 pixel (un controller ILI9341 su un bus SPI) mostra i giochi, che disegnano in un viewport di 320 per 200. Uno o due piccoli amplificatori audio (MAX98357A, su un bus I2S), un joystick e dei pulsanti sono collegati direttamente ai piedini del chip. Un gioco raggiunge la scheda via Wi-Fi: la scheda crea una rete chiamata `PRG32`, e la cartuccia vi viene caricata. L'elenco dei componenti e i collegamenti sono nella cartella `docs/hardware` del repository.

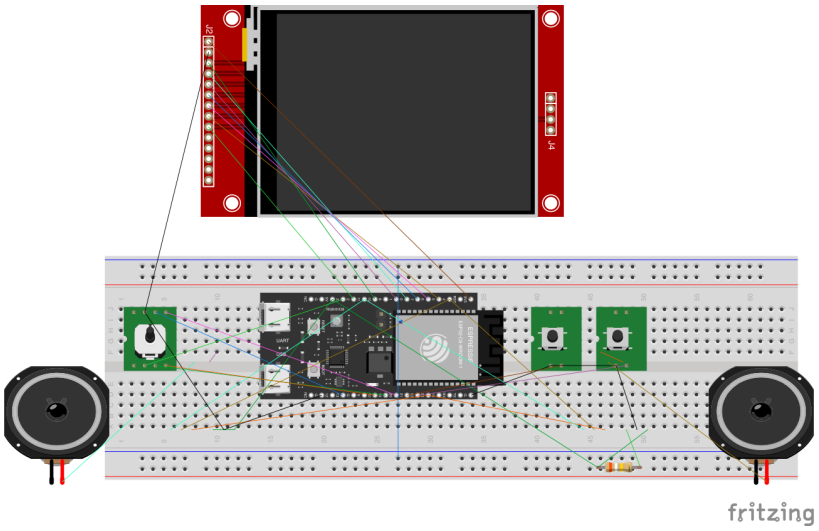


Figura 1.2. Il kit PRG32 su breadboard.

1.2.4 Tre modi di eseguire la stessa cartuccia

Il file della cartuccia è lo stesso su ogni host; cambia il modo in cui le sue istruzioni vengono eseguite e ciò che si può osservare (Tabella 1.2).

Tabella 1.2. Tre host per la stessa cartuccia.

	Scheda ESP32-C6	**QEMU di Espressif**	**PRG32-QT**
Come gira il codice	in modo nativo sul chip	in modo nativo su un ESP32-C3 emulato	guest RV32IMAC interpretato
Serve una toolchain	sì (ESP-IDF)	sì (ESP-IDF)	no, per giocare e fare debug
La cartuccia arriva con	caricamento via Wi-Fi, Store	scrittura nell'immagine della flash	importazione di file, Store, caricamento HTTP
Esecuzione passo-passo	con JTAG e GDB	con GDB	debugger integrato
Gira su	il kit hardware	Windows, Linux, macOS	desktop, Raspberry Pi, iOS, Android, TV

La regola pratica è: usate la scheda quando volete la verità sul *tempo*, e PRG32-QT quando volete la verità su un *registro*. L'emulatore modella un ESP32-C3 e non un C6 perché Espressif mantiene per quel chip un emulatore maturo con grafica; esegue le stesse istruzioni RISC-V, e la cartuccia non se ne accorge.

1.2.5 PRG32-QT: la console sul portatile

PRG32-QT è un'applicazione per Windows, macOS, Linux, Raspberry Pi, telefoni, tablet e televisori. Contiene un interprete di RV32IMAC, che esegue una cartuccia istruzione per istruzione e le offre le stesse funzioni del firmware. Ha tre velocità, scelte in *Settings*: *Accurate*, temporizzata come l'ESP32-C6, che è quella predefinita; *Optimal*, a 30 frame al secondo; e *Unlimited*, veloce quanto lo è il portatile. Il profilo accurato attribuisce a ogni istruzione e a ogni chiamata un costo in cicli di un clock a 160 MHz: non è un modello esatto al ciclo del chip, ma è abbastanza vicino da far vedere quando un frame è in ritardo. PRG32-QT contiene anche un browser dello Store e il debugger descritto nel Capitolo 0.

I comandi della console corrispondono alla tastiera come nella Tabella 1.3. Un controller di gioco USB o Bluetooth viene riconosciuto da PRG32-QT quando lo si collega.

Tabella 1.3. I comandi, su una tastiera.

Sul kit	In PRG32-QT	In QEMU
Joystick	tasti freccia, oppure **W A S D**	gli stessi, battuti nel terminale
Pulsante A	**Z** o **J**	**Z** o **J**
Pulsante B	**X** o **K**	**X** o **K**
Start / Select	**Invio** o **Spazio**	**Invio** o **Spazio**
Uscire dal gioco	**Esc**: ritorno a Setup	chiudere QEMU

Attenzione

QEMU legge la tastiera dal terminale che lo ha lanciato, non dalla finestra con l'immagine. È la classica sorpresa della prima sessione.

1.3 Passo 1: installare PRG32-QT

Cinque minuti; nessun compilatore, nessun account. Dalla pagina https://github.com/riscv-prg32/PRG32-QT, sotto *Releases*, prendete l'ultima versione del file per il vostro sistema (Tabella 1.4). Un file `SHA256SUMS-platforms.txt` sulla stessa pagina elenca le somme di controllo; lì ci sono anche le versioni per Android e per i televisori.

Tabella 1.4. Gli installer di PRG32-QT (versione 0.4.1, ottobre 2026; i nomi seguono questo schema con il numero corrente).

Sistema	File	Tipo
Windows 10 o 11	`PRG32-QT-0.4.1-Windows-X64-Setup.exe`	installer
macOS, Apple Silicon	`PRG32-QT-0.4.1-macOS-ARM64.dmg`	immagine disco
macOS, Intel	`PRG32-QT-0.4.1-macOS-Intel-X64.dmg`	immagine disco
Linux, PC a 64 bit	`PRG32-QT-0.4.1-Linux-X64.deb`	pacchetto Debian
Raspberry Pi OS, 64 bit	`PRG32-QT-0.4.1-Raspberry-Pi-OS-ARM64.deb`	pacchetto Debian

Windows. Scaricate il file che termina con `Windows-X64-Setup.exe` ed eseguitelo. Se Windows mostra un avviso blu di SmartScreen, scegliete *Ulteriori informazioni* e poi *Esegui comunque*. Accettate la cartella proposta: l'installer porta con sé le librerie Qt e il runtime

del compilatore, e non c'è altro da installare. Avviate *PRG32* dal menu Start; se il firewall lo chiede, autorizzatelo sulle reti private. La domanda compare perché PRG32-QT ascolta sulla porta 8080 le cartucce inviate dagli strumenti.

macOS. Dal menu Apple scegliete *Informazioni su questo Mac*: «Chip Apple M...» significa ARM64, «Processore Intel» significa X64. Aprite il `.dmg` e trascinate *PRG32* sulla cartella Applicazioni. Al primo avvio, se macOS dice che lo sviluppatore non può essere verificato (l'applicazione è distribuita fuori dall'App Store), fate clic con il tasto destro su *PRG32* e scegliete *Apri*, oppure andate in *Impostazioni di Sistema, Privacy e sicurezza, Apri comunque*. Accettate le connessioni in entrata e l'accesso alla rete locale quando viene chiesto: servono all'interfaccia di dispositivo sulla porta 8080 e al suo annuncio sulla rete locale.

Linux. Il pacchetto richiede Qt 6.5 o successivo, che Debian 13 e Ubuntu 25.04 forniscono; sui sistemi più vecchi (Ubuntu 24.04 ha Qt 6.4) il programma si compila da sorgente (Listato 1.1). Su Raspberry Pi OS usate il pacchetto ARM64, oppure lo script `scripts/build-raspbian.sh`.

```
# Debian 13, Ubuntu 25.04 and later, or another system with Qt 6.5 or newer
cd ~/Downloads
sudo apt install ./PRG32-QT-0.4.1-Linux-X64.deb
PRG32 &

# On older systems (Ubuntu 24.04 has Qt 6.4: too old): build from source
sudo apt install git cmake ninja-build g++ qt6-base-dev qt6-declarative-dev \
    qt6-multimedia-dev qt6-websockets-dev
git clone https://github.com/riscv-prg32/PRG32-QT
cd PRG32-QT
./scripts/build-linux.sh
```

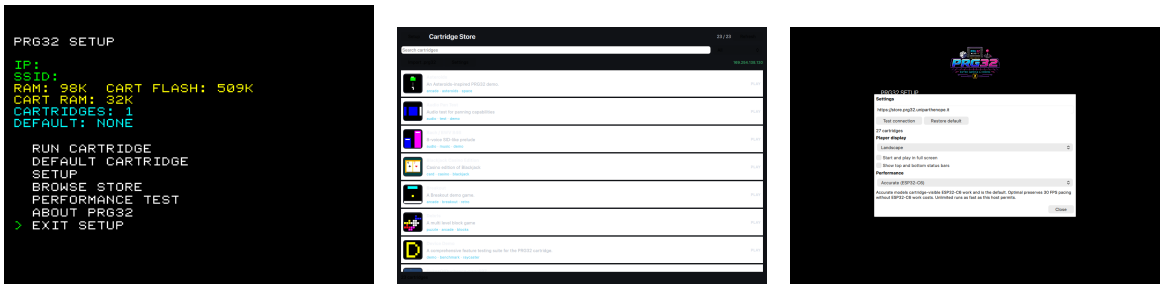
Listato 1.1. PRG32-QT su Linux.

Eseguite: la vostra prima cartuccia, dallo Store

1. Avviate PRG32-QT: compare la schermata *Setup*, la stessa che il firmware mostra sul kit.
2. Scegliete *Browse Store*: viene scaricato il catalogo del Cartridge Store.
3. Scegliete un gioco, scaricatelo, e scegliete *Run Cartridge*.
4. Giocate con i tasti freccia, **Z** e **X**. **Esc** vi riporta a Setup.
5. Aprite *Settings*: guardate i tre profili di prestazioni e le scelte di disposizione.

Risultato atteso. Un gioco dello Store gira sul vostro portatile (Figura 1.3). Questo è il livello 1: da ora potete eseguire qualunque cartuccia, e guardarci dentro. Se l'elenco dello Store resta vuoto, il portatile non raggiunge Internet oppure c'è di mezzo un proxy: *Settings* mostra l'indirizzo dello Store.

1.4 Passo 2: installare la toolchain



(a) Il menu Setup **(b)** Browse Store **(c)** Settings

Figura 1.3. Che cosa dovreste vedere: le stesse tre schermate su ogni sistema.

1.4.1 Un'idea dietro ogni installazione

Il compilatore per la famiglia ESP32 arriva con ESP-IDF, il kit di sviluppo di Espressif (Espressif Systems, 2026). PRG32 richiede ESP-IDF 5.4 o successivo, installato per due chip: `esp32c6`, il kit, ed `esp32c3`, l'emulatore. Dopo di che, un solo comando fa tutto: `python3 -m prg32`.

ESP-IDF non è attivo da solo

Ogni nuovo terminale deve accendere ESP-IDF prima che gli strumenti funzionino. Quasi ogni problema di installazione è ESP-IDF non attivo *nel terminale in cui state scrivendo*. Il sintomo è sempre lo stesso messaggio:

```
idf.py: command not found
```

(oppure `riscv32-esp-elf-gcc` non trovato). Su Windows, attivare significa aprire la *ESP-IDF PowerShell*; su macOS e Linux, eseguire `export.sh`.

1.4.2 Windows

Installate Git for Windows e Python 3, poi eseguite l'ESP-IDF Tools Installer e lavorate nella finestra di PowerShell che esso crea (Listato 1.2). Usate la ESP-IDF PowerShell, mai un terminale qualunque; su Windows il comando è `python`, non `python3`; `pip` installa nell'ambiente di ESP-IDF, ed è voluto. Evitate le cartelle con spazi o lettere accentate nel percorso. Chi preferisce un editor può usare Visual Studio Code con l'estensione Espressif ESP-IDF, che installa la stessa toolchain.

```
# 1. Install Git for Windows (git-scm.com) and Python 3 (python.org)
# 2. Run the ESP-IDF Tools Installer (dl.espressif.com/dl/esp-idf):
#      - choose an ESP-IDF release 5.4 or newer
#      - select the targets esp32c3 and esp32c6
#      - optional: tick the QEMU RISC-V emulator
# 3. Open the shortcut "ESP-IDF PowerShell" that the installer created
# 4. In that window:

cd $HOME\Documents
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
pip install pyaudio zeroconf
python -m prg32 doctor
```

Listato 1.2. La toolchain su Windows.

1.4.3 macOS

Prima viene Homebrew, perché fornisce `git`, `cmake` e `python` (Listato 1.3); `git` può chiedere, la prima volta, di installare gli strumenti a riga di comando di Xcode. Il clone ricorsivo e `install.sh` scaricano qualche gigabyte: abbiate pazienza. Il punto prima di `export.sh` conta, perché fa sì che lo script modifichi *questo* terminale. La procedura funziona su Apple Silicon e su Intel.

```
# 1. Homebrew (brew.sh), then the prerequisites
brew install git cmake ninja dfu-util ccache libusb python curl zip portaudio

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listato 1.3. La toolchain su macOS.

1.4.4 Linux (Debian, Ubuntu)

I prerequisiti sono una lunga riga di `apt`, da copiare per intero (Listato 1.4); i passi 2 e 3 sono gli stessi di macOS. Altre distribuzioni hanno gli stessi pacchetti con altri nomi. Solo per il kit, l'utente deve appartenere al gruppo `dialout`, che dà accesso alla porta seriale della scheda: `sudo usermod -aG dialout $USER`, poi si esce e si rientra nella sessione.

```
# 1. Prerequisites
sudo apt update
sudo apt install -y git wget flex bison gperf python3 python3-venv python3-pip
    \
    python3-zeroconf cmake ninja-build ccache libffi-dev libssl-dev dfu-util \
    libusb-1.0-0 curl zip portaudio19-dev

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listato 1.4. La toolchain su Linux.

1.4.5 A ogni nuovo terminale: accendere ESP-IDF

`export.sh` agisce sul terminale in cui viene eseguito, e solo su quello. Una nuova finestra è un nuovo terminale, e va attivata di nuovo. Due alias risparmiano battute (Listato 1.5); sono una comodità, non un obbligo. I comandi `prg32` si eseguono dalla cartella di PRG32.

```
# macOS and Linux: in every new terminal, before anything else
. $HOME/esp-idf/export.sh
cd $HOME/PRG32

# save typing: add these two lines to ~/.zshrc or ~/.bashrc
alias get_idf=". $HOME/esp-idf/export.sh"
alias prg="python3 -m prg32"

# then, in a new terminal
get_idf
prg doctor

# Windows: open "ESP-IDF PowerShell" from the Start menu, then
cd $HOME\Documents\PRG32
```

Listato 1.5. Attivare ESP-IDF in un nuovo terminale.

Vedete l'errore una volta, di proposito

Chiudete il terminale, apritene uno nuovo ed eseguite `python3 -m prg32 doctor` *senza* attivare ESP-IDF. Leggete il messaggio. È meglio incontrarlo qui che nel mezzo di un tutorial.

1.4.6 Il doctor

Il comando `doctor` controlla l'installazione (Listato 1.6). La riga su `idf.py` dice che ESP-IDF è attivo; quella su `riscv32-esp-elf-gcc` che il compilatore RISC-V è raggiungibile; quelle sulle partizioni che la cartella di PRG32 è completa. I quattro slot per le cartucce, da `cart0` a `cart3`, sono regioni della memoria flash della scheda, di 131 072 byte ciascuna. Ogni riga marcata `ERROR` dice che cosa fare. Se il doctor passa, il livello 2 è fatto.

```
$ python3 -m prg32 doctor
[OK] idf.py found
[OK] riscv32-esp-elf-gcc found
[OK] partitions file found: .../PRG32/profiles/partitions_prg32.csv
[OK] partition cart0 parsed (offset=0x210000, size=131072)
...

# and when ESP-IDF is not active in this terminal:
[ERROR] idf.py missing (you must source ESP-IDF export.sh)
```

Listato 1.6. Il doctor (messaggi di ottobre 2026).

1.5 Passo 3: la vostra prima compilazione

1.5.1 Compilare una cartuccia dagli esempi

Il repository contiene giochi completi. Compilarne uno dimostra che il compilatore funziona prima ancora di aver scritto una sola riga (Listato 1.7).

```
# in the PRG32 folder, ESP-IDF active
python3 -m prg32 cartridge build examples/games/pong/c/game.c \
    --portable --entry-prefix pong_c --name pong --out pong.prg32

[OK] built pong.prg32 name=pong load=0x40800000
[INFO] code=3004 mem=3028 audio=0

# what did we build?
python3 -m prg32 cartridge summary pong.prg32
```

Listato 1.7. Compilare Pong, in C, dagli esempi.

Il comando `cartridge build` prende un sorgente e produce un file `.prg32`. L'opzione `--entry-prefix` dà il nome che nel sorgente precede `_init`, `_update` e `_draw`; `--portable` chiede una cartuccia che giri su ogni host, e non su un solo firmware. Un Pong intero sono tremila byte. Nel Capitolo 2 il sorgente sarà quello del lettore, in assembly, e il comando sarà lo stesso. Su Windows scrivete `python` e mettete il comando su una sola riga.

1.5.2 Inviarla a PRG32-QT

PRG32-QT risponde sulla porta 8080 esattamente come una scheda (Listato 1.8). Il comando `runtime` stampa lo stato dell'host ed è una prova rapida della connessione; `upload-and-run` memorizza la cartuccia in uno slot e la avvia. Il comando dice `esp32c6` perché, per lo strumento, PRG32-QT *è* una scheda. Se `localhost` non risponde, leggete l'indirizzo mostrato nella schermata Setup di PRG32-QT e usate quello. Senza gli strumenti, lo stesso risultato si ottiene in PRG32-QT importando il file `pong.prg32` e scegliendo *Run Cartridge*; con `--url http://192.168.4.1` lo stesso comando invia il gioco a una scheda vera.

```
# PRG32-QT must be running on this computer

# 1. is it listening?
python3 -m prg32 runtime --url http://localhost:8080

# 2. upload the cartridge to slot cart0 and start it
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://localhost:8080 --slot cart0

{"loaded":false,"ok":true,"slot":"cart0","stored":true}
{"ok":true,"slot":"cart0"}
[OK] Finished upload and run on ESP32C6
```

Listato 1.8. Inviare la cartuccia a PRG32-QT, in esecuzione sullo stesso calcolatore.

1.5.3 Guardare dentro: il debugger

Aprite la macchina

1. In PRG32-QT aprite *Settings* e attivate *Run cartridges in RISC-V debug mode*.
2. Eseguite di nuovo la cartuccia: il gioco è a sinistra, la macchina a destra (Figura 1.4).
3. Premete **Pause** (il pulsante con due barre): il gioco si blocca, e nel listato è evidenziata un'istruzione.
4. Premete **Step** (il triangolo con una barra) qualche volta: l'evidenziazione si sposta, e alcuni registri cambiano valore.
5. Premete **Update**, poi **Draw**: il listato salta a quelle due funzioni della cartuccia.
6. Premete **Resume** (il triangolo): il gioco prosegue. Abbassate la velocità a 0.1x e guardatelo al rallentatore.

I tre pulsanti di trasporto del debugger sono disegnati come su un lettore video; questo libro li chiama **Pause**, **Resume** e **Step**. Questo è solo un primo sguardo: il prossimo capitolo spiega che cosa sono i registri. Qui importa che il debugger si apra e risponda. La modalità debug resta attiva finché non viene disattivata.

1.6 Facoltativo: QEMU e il kit

1.6.1 Livello 3: l'emulatore QEMU

QEMU emula un chip intero ed esegue il firmware vero (Listato 1.9). Il comando `qemu build` compila il firmware e la prima volta richiede parecchi minuti, perché scarica dei componenti; `qemu upload` scrive una cartuccia nell'immagine della flash, e mentre gira QEMU deve essere chiuso. Se il caricamento fallisce, eseguite QEMU una volta perché l'immagine della flash esista, chiudetelo, e caricate di nuovo. Da ottobre 2026 il firmware parte per impostazione predefinita dal menu *Setup*, su QEMU e sulla scheda: scegliete *Run Cartridge*, poi lo slot `cart0`. Questa procedura viene dalla documentazione di PRG32 (`docs/usage/qemu.md`) e non è stata eseguita per questo libro.

```
# libraries for the window of the emulator
brew install libgcrypt glib pixman sdl2 libslirp                    # macOS
sudo apt install -y libgcrypt20 libglib2.0-0 libpixman-1-0 \
    libsdl2-2.0-0 libslirp0                                          # Linux

# the emulator itself (Windows: tick it in the ESP-IDF installer)
python $IDF_PATH/tools/idf_tools.py install qemu-riscv32
. $IDF_PATH/export.sh

# build the firmware for QEMU, stage a cartridge, run
python3 -m prg32 qemu build
python3 -m prg32 qemu upload pong.prg32
python3 -m prg32 qemu run
```

Listato 1.9. Livello 3: l'emulatore QEMU.

1.6.2 Livello 4: il kit

Il firmware viene scritto sulla scheda una volta, attraverso un cavo USB; dopo di che i giochi arrivano via Wi-Fi e il cavo non serve più (Listato 1.10). Il comando di caricamento è lo stesso usato per PRG32-QT, con un altro indirizzo. Se la porta seriale non viene trovata la si indica esplicitamente: `COM`*x* su Windows, `/dev/ttyACM0` su Linux, `/dev/cu.usbmodem...` su macOS. Uno schermo nero dopo la scrittura significa di solito che è stato scritto il firmware per QEMU invece di quello per la scheda. Anche questa procedura viene dalla documentazione e non è stata eseguita per questo libro.

```
# connect the board with a USB cable; ESP-IDF active, in the PRG32 folder
python3 -m prg32 esp32c6 build-and-flash

# the board starts and creates a Wi-Fi network
#     name: PRG32     password: prg32game     address: 192.168.4.1
# join it with your laptop, then
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://192.168.4.1 --slot cart0

# or let the board join the Wi-Fi of the classroom
python3 -m prg32 wifi set --ssid "NAME" --password "PASSWORD" --mode sta
```

Listato 1.10. Livello 4: il kit.

1.7 Quando qualcosa non funziona

La Tabella 1.5 elenca i sintomi che si incontrano più spesso; la prima riga copre la maggior parte dei casi. Il file `docs/usage/troubleshooting.md` del repository contiene l'elenco completo.

Tabella 1.5. Risoluzione dei problemi.

Vedete	Significa	Fate così
`idf.py: command not found`	ESP-IDF non è attivo in questo terminale	eseguite `export.sh`, o aprite la ESP-IDF PowerShell
manca `riscv32-esp-elf-gcc`	i compilatori non sono stati installati, o non sono attivi	`./install.sh esp32c3,esp32c6`, poi `export.sh`
`No module named prg32`	non siete nella cartella di PRG32	`cd` nella cartella che avete clonato
upload: connection refused	PRG32-QT non è in esecuzione, o un firewall blocca la porta 8080	avviatelo; autorizzatelo; usate l'indirizzo della sua schermata Setup
macOS non apre PRG32	l'applicazione viene da fuori dell'App Store	clic destro, Apri; oppure Privacy e sicurezza, Apri comunque
Linux: PRG32 non parte	il Qt del sistema è più vecchio della 6.5	una distribuzione più recente, o compilate da sorgente
QEMU ignora i tasti	la tastiera va al terminale	fate clic sul terminale che ha lanciato QEMU

Come chiedere aiuto

Incollate il comando e il messaggio *intero* come testo. Non mandate mai una fotografia dello schermo.

1.7.1 Pronti per il prossimo capitolo?

Cinque controlli, e l'officina è pronta.

1. PRG32-QT parte ed esegue un gioco dello Store (passo 1).
2. In un nuovo terminale, dopo aver attivato ESP-IDF, `doctor` mostra solo `OK` (passo 2).
3. `cartridge build` produce `pong.prg32` (passo 3).
4. `upload-and-run` lo avvia in PRG32-QT (passo 3).
5. In modalità debug, **Pause** e **Step** funzionano (passo 3).

1.8 Verificate di aver capito

D1.1 In un nuovo terminale, `python3 -m prg32 doctor` dice: `idf.py missing`. Qual è la causa più probabile?

A. ESP-IDF va installato di nuovo
B. ESP-IDF non è attivo in questo terminale
C. PRG32-QT non è in esecuzione
D. la scheda non è collegata

D1.2 Quale host permette di eseguire una cartuccia e farne il debug senza installare un compilatore?

A. la scheda ESP32-C6
B. QEMU
C. PRG32-QT
D. nessuno

D1.3 Perché lo stesso `pong.prg32` gira sulla scheda, su QEMU e su PRG32-QT?

A. viene ricompilato a ogni avvio
B. contiene tre programmi
C. chiama il runtime attraverso la stessa tabella di funzioni su ogni host
D. PRG32-QT lo traduce in C

1.9 Esercizi

Esercizio 1.1 (l'officina sopravvive a un riavvio)

1. Completate i cinque controlli qui sopra sul vostro portatile.
2. Scaricate tre cartucce dallo Store, e in ciascuna trovate con il debugger la prima istruzione di `update`.
3. Compilate uno degli altri esempi in `examples/games`, ed eseguitelo.

4. Chiudete tutto, riavviate il calcolatore, e tornate a una cartuccia in esecuzione senza questo libro.
5. Facoltativo: installate QEMU ed eseguite lì la stessa cartuccia.

Risultato atteso. La vostra officina sopravvive a un riavvio, e sapete aprirla senza aiuto. Il quarto compito è la vera prova: mostra se l'attivazione di ESP-IDF è stata capita.

1.10 Glossario

Inglese	**Italiano**
toolchain	catena di strumenti di sviluppo
cartridge	cartuccia
emulator	emulatore
to build	compilare (costruire)
host	macchina ospite
firmware	firmware
runtime	ambiente di esecuzione
interpreter	interprete
to flash	scrivere nella memoria flash
breakpoint	punto di interruzione

1.11 Riepilogo e letture

PRG32 è un runtime, un formato di cartuccia, uno strumento e uno Store; una cartuccia gira senza modifiche sulla scheda, su QEMU e su PRG32-QT. PRG32-QT basta per giocare e per fare debug; ESP-IDF e il repository di PRG32 servono per compilare. ESP-IDF va attivato in ogni nuovo terminale: ricordatelo e la maggior parte dei problemi scompare. Avete compilato una cartuccia, l'avete inviata a PRG32-QT e l'avete fermata nel debugger: è il ciclo di tutto il libro.

Letture. Il `README` e la cartella `docs/usage` del repository di PRG32 (Montella et al., 2026a); il `README` e la documentazione di PRG32-QT (Montella, 2026); i capitoli sull'installazione e su PRG32-QT di *Playing with RISC-V* (Montella et al., 2026b). L'Appendice **??** di questo libro è un riferimento della piattaforma e dei suoi comandi.

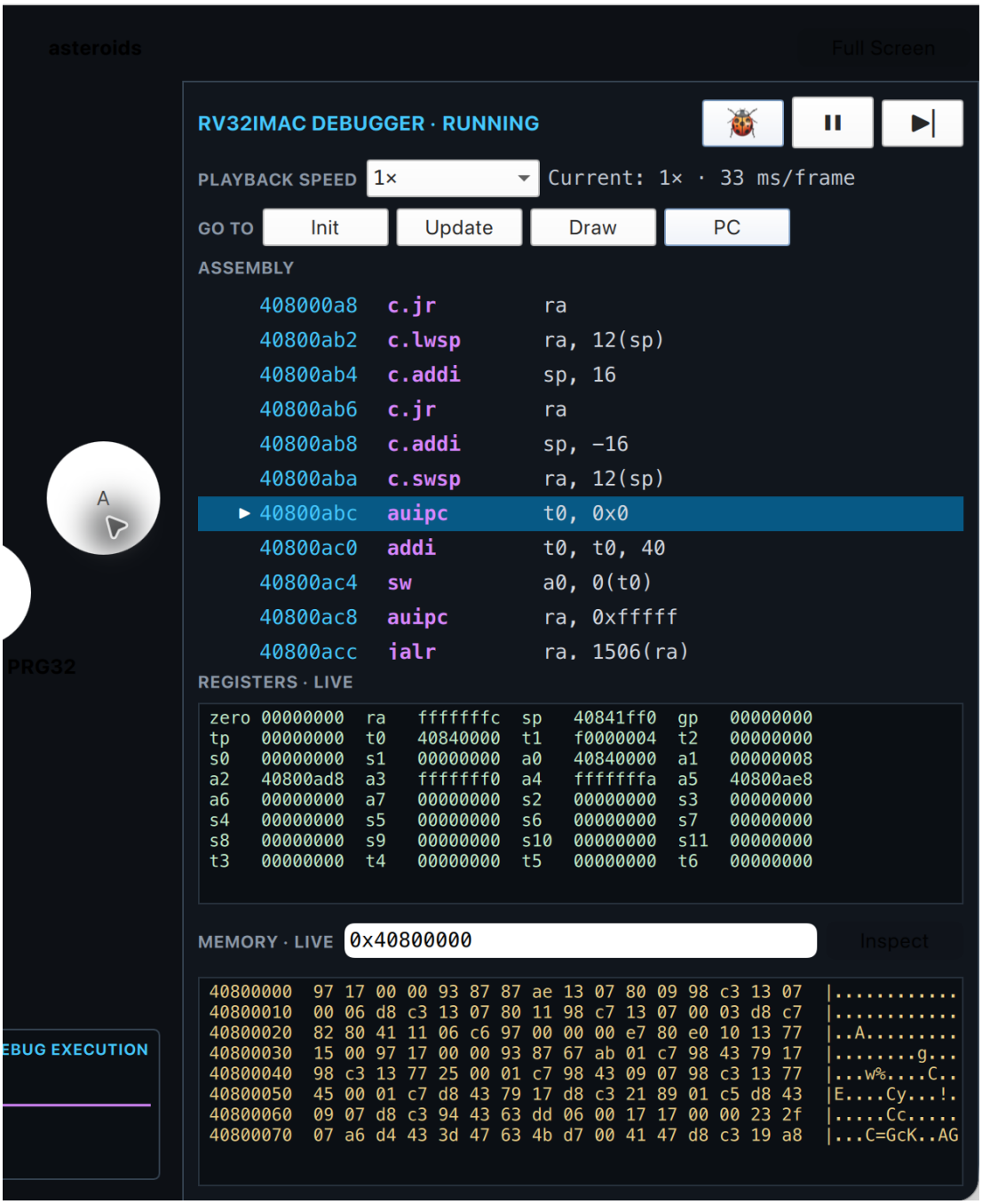


Figura 1.4. La macchina, aperta: l'assembly con l'istruzione corrente, i 32 registri, la memoria, e i contatori del frame. Se vedete questo con la vostra cartuccia, siete pronti per il resto del libro.

CAPITOLO 2

Primo contatto: assembly RISC-V su PRG32

"Talk is cheap. Show me the code."
— Linus Torvalds, mailing list del kernel Linux, 2000: le chiacchiere stanno a zero, fatemi vedere il codice

In questo capitolo

Alla fine di questo capitolo saprete

- compilare una cartuccia da un file assembly ed eseguirla su PRG32;
- dire che cos'è un registro, e nominare quelli che si usano ogni giorno;
- chiamare una funzione del framework, passando gli argomenti nei registri;
- conservare un valore in memoria da un frame al successivo: caricare, modificare, memorizzare;
- leggere un'istruzione come il numero di 32 bit che è in realtà;
- elencare le domande che questo programma solleva, e a cui risponde il resto del libro.

Questo capitolo è il giro «gioca» della spirale per tutta la prima metà del libro: ogni capitolo successivo rimanda a qualcosa che è stato scritto qui. *Non* ci si aspetta che il lettore capisca tutto ciò che contiene. Questo capitolo serve a vedere; i successivi servono a spiegare. Si scrivono quattro piccoli programmi, ciascuno completo e ciascuno funzionante; sono i passi verificati del primo laboratorio di assembly del libro di PRG32 (Montella et al., 2026b).

2.1 La macchina

2.1.1 I registri, e tre tipi di istruzione

Un processore RISC-V, visto dal programmatore, è una cosa molto piccola.

Trentadue registri.
Ogni *registro* contiene un valore di 32 bit. Il processore può calcolare soltanto su valori che stanno nei registri.

Calcolare.
Un'operazione aritmetica o logica tra registri: `add`, `sub`, `and`, `or`, `xor`, e le loro forme *immediate*, come `addi`, in cui un operando è una piccola costante scritta nell'istruzione.

Spostare.
Un trasferimento tra un registro e la memoria: `lw` carica una parola, `sw` memorizza una parola. Nient'altro tocca la memoria.

Saltare.
Un cambiamento dell'istruzione che viene eseguita dopo: salti condizionati come `beqz`, chiamate e ritorni.

Questa è quasi tutta la macchina. La ricchezza dell'informatica non viene da un grande insieme di istruzioni, ma dal combinarne uno piccolo in lunghe sequenze.

2.1.2 I registri che si usano davvero

L'hardware chiama i registri da `x0` a `x31` e, con una sola eccezione, li tratta allo stesso modo. I programmatori li chiamano con altri nomi, che ricordano il *ruolo* svolto da ciascuno (Tabella 2.1). I ruoli non sono fissati dall'hardware ma da un accordo, la *convenzione di chiamata*, parte dell'application binary interface (ABI). Rispettarla è ciò che permette a una funzione scritta in assembly e al codice C del framework di chiamarsi a vicenda.

Tabella 2.1. I registri che si usano ogni giorno. La tabella completa è nell'Appendice **??**.

Registro	Nome	Ruolo	Conservato da una chiamata?
`a0`–`a7`	argomenti	valori passati a una chiamata; `a0` porta anche il valore di ritorno	no
`t0`–`t6`	temporanei	valori di lavoro	no
`s0`–`s11`	salvati	valori che una chiamata promette di conservare	sì
`ra`	indirizzo di ritorno	dove salta `ret`	no: salvatelo voi
`sp`	puntatore allo stack	cima dello stack, allineata a 16 byte	sì
`zero`	zero	si legge sempre come 0; le scritture sono ignorate	–

2.1.3 Il contratto: tre funzioni

Un gioco PRG32 sono tre funzioni che il programmatore scrive e il firmware chiama (Figura 2.1). `init` viene chiamata una volta, quando la cartuccia parte, e prepara lo stato del gioco. `update` viene chiamata una volta per frame: legge l'ingresso e cambia lo stato. `draw` viene chiamata una volta per frame: trasforma lo stato in un'immagine. Il firmware trova le tre funzioni per nome: un prefisso scelto dal programmatore, seguito da `_init`, `_update` e `_draw`. Questo è l'intero contratto tra un gioco e la macchina.

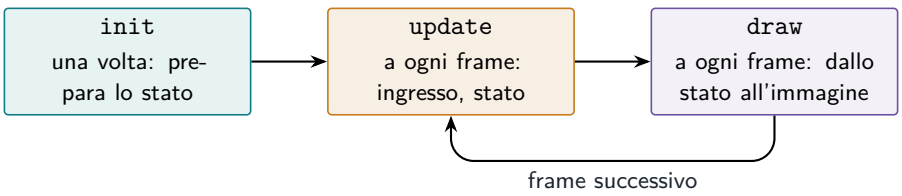


Figura 2.1. Il contratto tra una cartuccia e il runtime.

2.2 La vostra prima cartuccia

2.2.1 Passo 1: il più piccolo programma che gira

Il Listato 2.1 è una cartuccia completa. Create una cartella, scrivete il programma in un file chiamato `step1_skeleton.S`, e resistete alla tentazione di copiarlo: imparano anche le dita.

```
# Assembly Lab 1: the smallest program that runs
.option norelax

.section .text
.global first_init
.global first_update
.global first_draw

first_init:
    ret

first_update:
    ret

first_draw:
    ret
```

Listato 2.1. Passo 1: tre funzioni che non fanno nulla, correttamente.

Le righe che cominciano con un punto sono *direttive*: ordini per l'assemblatore, non istruzioni per il processore. `.option norelax` mantiene prevedibile la disposizione della cartuccia, e ogni esempio di PRG32 comincia così. `.section .text` dice che ciò che segue è codice eseguibile. `.global` rende un'*etichetta* visibile fuori dal file, così che il firmware possa trovare le funzioni per nome. Un'etichetta, un nome seguito da due punti, sta per l'indirizzo di ciò che la segue. L'unica istruzione del programma, scritta tre volte, è `ret`: ritorna al chiamante.

La cartuccia si compila e si esegue con i comandi incontrati nel Capitolo 1 (Listato 2.2); il file della cartuccia è identico byte per byte per QEMU, per la scheda e per PRG32-QT, dove l'indirizzo è `http://localhost:8080`.

```
# 1. build a portable cartridge from one assembly file
python3 -m prg32 cartridge build step1_skeleton.S --portable \
    --entry-prefix first --name step1 --out build/step1.prg32

# 2a. run it in QEMU on your laptop
python3 -m prg32 qemu upload build/step1.prg32
python3 -m prg32 qemu run

# 2b. or send it to a board, or to PRG32-QT, over the network
python3 -m prg32 esp32c6 upload-and-run build/step1.prg32 \
    --url http://192.168.4.1 --slot cart0
```

Listato 2.2. Compilare ed eseguire una cartuccia.

Eseguite

Il risultato atteso è uno schermo nero. Il risultato è reale lo stesso: il framework ha trovato le tre funzioni, le ha chiamate frame dopo frame, ed è sopravvissuto.

2.2.2 Passo 2: dire ciao

Per mettere qualcosa sullo schermo il programma chiama due funzioni del framework: `prg32_gfx_clear`, che riempie lo schermo con un colore, e `prg32_gfx_text8`, che scrive una stringa con un font di 8 per 8 pixel (Listato 2.3, Figura 2.2). In cima al file due direttive `.equ` danno un nome a due costanti: `.equ BLACK, 0x0000` e `.equ WHITE, 0xffff`.

```
first_draw:
    addi sp, sp, -16      # prologue: a 16-byte frame
    sw   ra, 12(sp)       # save the return address

    li   a0, BLACK
    call prg32_gfx_clear

    li   a0, 32           # x
    li   a1, 40           # y
    la   a2, title        # address of the string
    li   a3, WHITE        # foreground
    li   a4, BLACK        # background
    call prg32_gfx_text8

    lw   ra, 12(sp)       # epilogue: restore ra
    addi sp, sp, 16       # release the frame
    ret

.section .rodata
title:
    .asciz "HELLO PRG32"
```

Listato 2.3. Passo 2: la nuova `first_draw`.

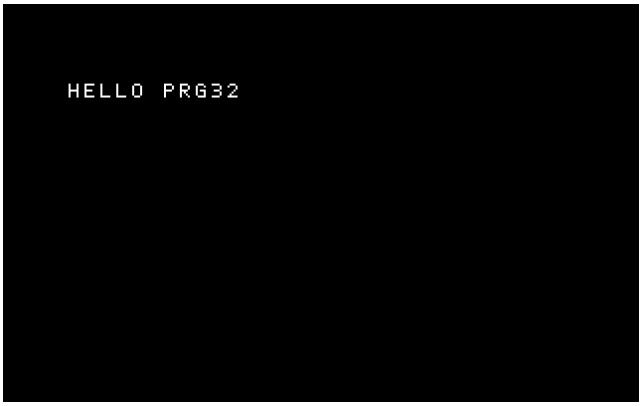


Figura 2.2. Il passo 2, compilato ed eseguito su PRG32.

La regola per passare gli argomenti è semplice: il primo argomento va in `a0`, il secondo in `a1`, e così via. `li` (*load immediate*) mette un numero in un registro; `la` (*load address*) vi

mette l'indirizzo di un'etichetta. La stringa vive nella sezione `.rodata`, *read-only data*, dati di sola lettura, e la direttiva `.asciz` ne memorizza i caratteri seguiti da un byte di valore zero che ne segna la fine.

2.2.3 Perché salvare `ra`? Lo stack frame

La prima riga e le ultime tre di `first_draw` sono nuove, e sono il prezzo da pagare per chiamare un'altra funzione. L'istruzione `call` memorizza in `ra` l'indirizzo a cui tornare. Ma `first_draw` è stata a sua volta chiamata dal firmware, e `ra` conteneva già l'indirizzo del *suo* ritorno: la prima `call` lo sovrascriverebbe, e la `ret` finale salterebbe nel posto sbagliato.

Una funzione che ne chiama un'altra salva quindi per prima cosa `ra`. Abbassa `sp` di 16 per fare spazio sullo *stack*, una regione di memoria che ogni funzione può usare per le proprie necessità temporanee, e memorizza `ra` a `12(sp)`, cioè 12 byte sopra l'indirizzo contenuto in `sp` (Figura 2.3). Queste righe sono il *prologo* della funzione. Dopo l'ultima chiamata disfa tutto in ordine inverso, nell'*epilogo*: ricarica `ra`, alza `sp` di 16, e ritorna.

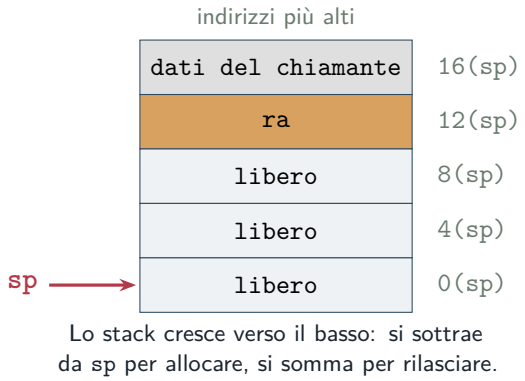


Figura 2.3. La memoria dello stack dopo il prologo di `first_draw`.

Il classico primo errore

Dimenticare la coppia `sw ra`/`lw ra` è il classico primo errore: la funzione ritorna nel posto sbagliato, e il programma va in crash o si blocca molto dopo la riga colpevole.

La convenzione richiede che `sp` resti allineato a un multiplo di 16 byte tra una chiamata e l'altra, ed è per questo che il frame è di 16 byte anche se ne vengono usati solo 4. Lo stack è trattato formalmente nei Capitoli ?? e ??.

2.2.4 Passo 3: un valore che sopravvive tra i frame

I registri non conservano i loro valori tra una chiamata di `update` e la successiva: nel frattempo li usa il firmware. Ciò che deve durare deve vivere in memoria (Listato 2.4).

```
first_update:
    # A leaf function: it calls nothing,
    # so it needs no stack frame.
    la   t0, counter      # t0 = ADDRESS of counter
    lw   t1, 0(t0)        # t1 = VALUE stored there   (load)
    addi t1, t1, 1        # one tick per frame        (modify)
    sw   t1, 0(t0)        # write it back             (store)
```

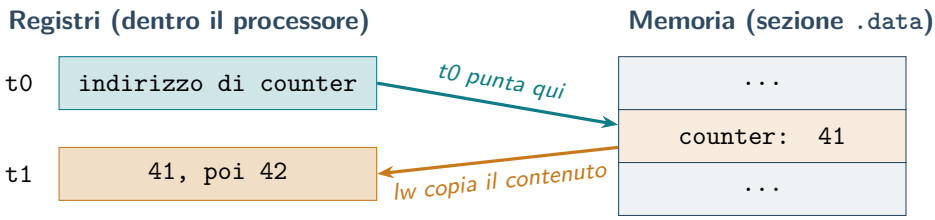
```
8       ret
9
10  .section .data
11  counter: .word 0
```

Listato 2.4. Passo 3: la nuova `first_update` e la sua variabile.

`counter` è un'etichetta: dà un nome a una locazione di memoria, nella sezione `.data`, che contiene una parola di 32 bit, inizialmente zero. `la` mette in `t0` l'*indirizzo* di quella locazione. `lw` legge in `t1` il *valore* che si trova a quell'indirizzo; `sw` lo riscrive. Poiché il processore calcola solo nei registri, ogni modifica di una variabile ha gli stessi tre movimenti: *caricare, modificare, memorizzare*. La funzione è una *foglia*: non chiama nulla, quindi non ha bisogno di uno stack frame.

Indirizzo o valore? Il punto cruciale della memoria

Un'etichetta dà un nome a una *locazione*. `la` dà la locazione; `lw` dà il contenuto (Figura 2.4). È così che in ultima analisi funziona ogni variabile di ogni programma. Usare un indirizzo dove serve un valore, o il contrario, è uno degli errori più comuni dei principianti: commentate ogni registro come ADDRESS (indirizzo) o VALUE (valore).




```
la   t0, counter    # t0 = WHERE the value lives
lw   t1, 0(t0)      # t1 = WHAT is stored there
addi t1, t1, 1      # change the copy in the register
sw   t1, 0(t0)      # memory still says 41 until this line
```

Figura 2.4. Il registro `t0` contiene un indirizzo; `t1` contiene un valore.

Dopo questo passo non cambia nulla di visibile: il contatore avanza, ma nessuno lo disegna. Come si potrebbe dimostrare che sta avanzando? Con il debugger di PRG32-QT, oppure con il passo successivo.

2.2.5 Passo 4: rendere visibile il contatore

Otto righe aggiunte a `first_draw`, dopo la chiamata che scrive il testo, disegnano una barra verde la cui larghezza è il contatore (Listato 2.5, Figura 2.5). La costante `GREEN` è definita in cima al file con `.equ GREEN, 0x07e0`.

```
1       la   t0, counter
2       lw   t1, 0(t0)
3       andi a2, t1, 255      # width = counter modulo 256
4       li   a0, 32           # x
5       li   a1, 60           # y
```

```
    li   a3, 8            # height
    li   a4, GREEN        # .equ GREEN, 0x07e0
    call prg32_gfx_rect
```

Listato 2.5. Passo 4: le righe aggiunte a `first_draw`.



Figura 2.5. Passo 4: la barra cresce da 0 a 255 pixel, poi ricomincia.

La funzione `prg32_gfx_rect` prende cinque argomenti: x, y, larghezza, altezza e colore. La larghezza è calcolata da `andi` con 255, che tiene soltanto gli otto bit bassi del contatore: la barra cresce da 0 a 255 pixel e poi ricomincia. In queste otto righe si nascondono tre capitoli futuri: l'uguaglianza 255 = `0xFF` (notazione posizionale), `andi` come maschera di bit (algebra di Boole), e `0x07E0` come colore (la codifica dell'informazione).

Il programma completo è il Listato 2.6.

```
# Assembly Lab 1: hello, state, and a counter you can see.
.option norelax
.equ BLACK, 0x0000
.equ WHITE, 0xffff
.equ GREEN, 0x07e0

.section .text
.global first_init
.global first_update
.global first_draw

first_init:
    ret

first_update:
    # A leaf function: it calls nothing, so it needs no stack frame.
    la   t0, counter          # t0 = ADDRESS of counter
    lw   t1, 0(t0)            # t1 = VALUE stored there      (load)
    addi t1, t1, 1            # one tick per frame           (modify)
    sw   t1, 0(t0)            # write it back                (store)
    ret

first_draw:
    addi sp, sp, -16          # prologue: make a 16-byte frame
    sw   ra, 12(sp)           # save the return address

```

```
27      li   a0, BLACK
28      call prg32_gfx_clear
29
30      li   a0, 32                # x
31      li   a1, 40                # y
32      la   a2, title             # address of the string
33      li   a3, WHITE             # foreground
34      li   a4, BLACK             # background
35      call prg32_gfx_text8
36
37      la   t0, counter
38      lw   t1, 0(t0)
39      andi a2, t1, 255           # width = counter modulo 256
40      li   a0, 32                # x
41      li   a1, 60                # y
42      li   a3, 8                 # height
43      li   a4, GREEN
44      call prg32_gfx_rect
45
46      lw   ra, 12(sp)            # epilogue: restore ra
47      addi sp, sp, 16            # release the frame
48      ret
49
50  .section .rodata
51  title:
52      .asciz "HELLO PRG32"
53
54  .section .data
55  counter: .word 0
```

Listato 2.6. La prima cartuccia completa (`step4_bar.S`).

2.3 Leggere ciò che avete scritto

2.3.1 Le istruzioni appena usate

La Tabella 2.2 ordina per famiglia le istruzioni incontrate finora. In tutte il risultato va nel *primo* registro scritto. L'insieme di istruzioni intero di base di RISC-V, RV32I, ha una quarantina di istruzioni: un quarto è già stato usato.

Tabella 2.2. Le istruzioni usate in questo capitolo.

Famiglia	Istruzione	Che cosa fa	Esempio
Aritmetica	`add`, `sub`	combina due registri	`add t0, t1, t2`
Logica	`and`, `or`, `xor`	operazione bit a bit su due registri	`and t0, t1, t2`
Immediato	`addi`, `andi`, `ori`, `xori`	un registro e una piccola costante	`addi t1, t1, 1`
Memoria	`lw`, `sw`	carica o memorizza una parola di 32 bit	`lw t1, 0(t0)`
Controllo	`call`, `ret`	va a una funzione e ne ritorna	`call prg32_gfx_rect`

2.3.2 Pseudo-istruzioni

Parecchi degli mnemonici usati non sono affatto istruzioni del processore. Sono *pseudo-istruzioni*: scorciatoie che l'assemblatore espande in una o due istruzioni vere (Tabella 2.3). Una pseudo-istruzione non è un'operazione hardware distinta. È il registro `zero` a renderne possibili molte: un piccolo insieme di istruzioni, riusato con cura.

Tabella 2.3. Le pseudo-istruzioni e le loro espansioni.

Voi scrivete	L'assemblatore emette	Significato
`li t0, 42`	`addi t0, zero, 42`	carica una costante
`mv t0, t1`	`addi t0, t1, 0`	copia un registro
`not t0, t1`	`xori t0, t1, -1`	complemento bit a bit
`la t0, counter`	`auipc t0, ...` + `addi t0, t0, ...`	carica l'indirizzo di un'etichetta
`ret`	`jalr zero, 0(ra)`	salta all'indirizzo contenuto in `ra`
`beqz t0, skip`	`beq t0, zero, skip`	salta se `t0` vale zero

2.3.3 Un'istruzione è un numero di 32 bit

Il processore non legge testo. L'assemblatore traduce ogni riga in una parola di 32 bit, ed è quella parola a essere memorizzata nella cartuccia ed eseguita. La Figura 2.6 seziona `addi t1, t1, 1`: dodici bit contengono la costante, cinque bit indicano il registro sorgente (`t1` è `x6`), tre bit selezionano l'operazione, cinque bit indicano il registro di destinazione e sette bit, l'*opcode*, dicono di che tipo di istruzione si tratta. Scritti in esadecimale, i trentadue bit sono `0x00130313`.

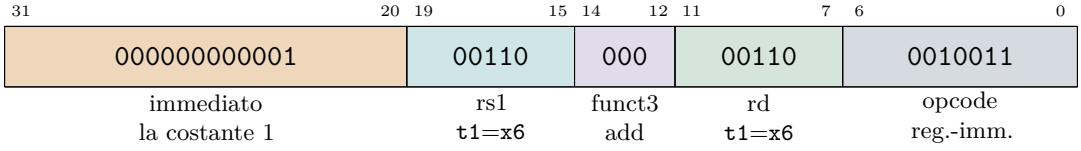


Figura 2.6. `addi t1, t1, 1` è il numero `0x00130313`.

Concetto

L'assembly è una scrittura leggibile del codice macchina. Un'istruzione è un dato: il processore ne legge i campi con circuiti logici, che verranno progettati nei capitoli sulla logica e sul processore.

2.3.4 PRG32-QT: un microscopio per i registri

PRG32-QT esegue la stessa cartuccia sul desktop un'istruzione alla volta, si ferma tra due istruzioni qualunque e mostra ogni registro.

Eseguite passo-passo `first_update`

Caricate il passo 3 in modalità debug, mettete un breakpoint su `first_update` e premete **Step** quattro volte. Prima di ogni pressione, dite che cosa conterranno `t0` e `t1`; dopo, verificate.

2.3.5 Tracciare a mano

Lo stesso esercizio si può, e si deve, fare su carta. La Tabella 2.4 segue `first_update` per alcuni frame. Al frame 256 il contatore continua a crescere, ma i suoi otto bit bassi tornano a zero, perché $256 = 1\,0000\,0000_2$: la barra ricomincia. Tracciare a mano è la più preziosa abilità di debug insegnata in questo libro.

Tabella 2.4. Tre frame, e uno in più, di `first_update`.

Frame	`lw`: `t1` diventa	`addi`: `t1` diventa	`sw`: counter diventa	Larghezza
1	0	1	1	1
2	1	2	2	2
256	255	256	256	0
257	256	257	257	1

2.4 Che cosa viene dopo

Il programma di questo capitolo ha sollevato più domande di quante ne abbia risolte. La Tabella 2.5 è la mappa in avanti del libro: ogni capitolo successivo si apre indicandone una riga.

Tabella 2.5. Le domande che il primo programma ha già sollevato.

Avete visto	La domanda	Risposta in
`0x07e0`, `0xffff`, 255	Perché i programmatori scrivono i numeri in esadecimale?	Capitoli 4, 5
una posizione sotto lo zero mostrata come `0xFFFFFFFF`	Come fa un registro a contenere un numero negativo?	Capitolo 5
una palla che si muove di meno di un pixel per frame	Come si rappresentano le frazioni senza un'unità in virgola mobile?	Capitolo 6
`andi a2, t1, 255`	Che cosa fa AND a trentadue bit tutti insieme?	Capitoli **??**, **??**
`addi t1, t1, 1`	Quale circuito somma davvero due numeri?	Capitolo **??**
un pulsante premuto è un 1	Di che cosa è fatto, fisicamente, un bit?	Capitolo **??**
`counter` sopravvive tra i frame	Come fa un circuito a ricordare un bit?	Capitoli **??**, **??**
`0x00130313`	Come fa il processore a decodificarla ed eseguirla?	Capitoli **??**, **??**
tre funzioni e un rettangolo	Se ne può fare un gioco intero?	Verm1: Capitoli **??**, **??**, **??**, **??**

2.5 Verificate di aver capito

D2.1 Dopo `la t0, counter` e `lw t1, 0(t0)`, che cosa contiene `t0`?

A. Il valore del contatore
B. L'indirizzo del contatore
C. Zero, perché `la` azzera il registro

D. L'indirizzo di ritorno

D2.2 Una funzione che chiama `prg32_gfx_clear` deve salvare un registro prima della chiamata. Quale?

A. `a0`, perché porta l'argomento
B. `sp`, perché lo stack si sposta
C. `ra`, perché `call` lo sovrascrive
D. `t0`, perché è temporaneo

D2.3 `t1` contiene 300. Che cosa lascia in `a2` l'istruzione `andi a2, t1, 255`?

A. 300
B. 255
C. 45
D. 44

2.6 Esercizi

Esercizio 2.1 (su PRG32)

1. Cambiate la stringa del titolo e il suo colore in giallo (`0xffe0`); eseguite di nuovo.
2. Fate crescere la barra a velocità doppia, poi quadrupla, cambiando una sola istruzione.
3. Fate accorciare la barra invece di farla crescere. Che cosa succede quando il contatore passa per lo zero?
4. In PRG32-QT eseguite passo-passo `first_update` e annotate `t0` e `t1` dopo ogni istruzione.
5. Disegnate una seconda barra, rossa, la cui larghezza è il contatore modulo 64.

Risultato atteso. Nel terzo compito il contatore passa da 0 a `0xFFFFFFFF`: con `andi 255` la barra salta a 255 e poi si accorcia. Conservate l'osservazione: la spiega il Capitolo 5.

2.7 Glossario

Inglese	Italiano
register	registro
load / store	caricare / memorizzare
address	indirizzo
immediate	immediato (costante)
branch	salto condizionato
label	etichetta
stack	pila (stack)
stack pointer	puntatore alla pila
return address	indirizzo di ritorno
calling convention	convenzione di chiamata
machine code	codice macchina
cartridge	cartuccia

2.8 Riepilogo e letture

Un processore RISC-V sono trentadue registri e un piccolo insieme di istruzioni che calcolano, spostano e saltano. Un gioco PRG32 sono tre funzioni; gli argomenti viaggiano nei registri da `a0` ad `a7`; una funzione che ne chiama un'altra salva `ra`. Le variabili vivono in memoria: `la` per l'indirizzo, `lw` e `sw` per il valore. Un'istruzione è un numero di 32 bit; l'assembly ne è la scrittura leggibile.

Letture. *Playing with RISC-V*, capitolo 5, «First Contact» (Montella et al., 2026b); Patterson e Hennessy (2020), capitolo 2; il tutorial `docs/learn/tutorial_ascii_game.md` del repository di PRG32.

Parte II

Informazione

CAPITOLO 3

Storia della computazione

"printf("Hello, world!");"
— Brian Kernighan e Dennis Ritchie, *The C Programming Language*, 1978

In questo capitolo

Alla fine di questo capitolo saprete

- definire l'informatica e dire perché non è la scienza dei calcolatori;
- collocare su una linea del tempo le macchine e le persone chiave, dall'abaco al calcolatore a programma memorizzato;
- nominare le quattro unità dell'architettura di von Neumann e ritrovarle nella macchina programmata nel Capitolo 2;
- spiegare che cosa significa RISC e perché RISC-V è un insieme di istruzioni aperto;
- descrivere dove si colloca l'informatica tra scienza, ingegneria ed economia.

Il lettore ha già eseguito un programma RISC-V. Questo capitolo racconta da dove viene quella macchina. Non è una cronaca: delle migliaia di macchine e di persone che la storia della computazione potrebbe elencare, conserva quelle che hanno aggiunto un'idea che si trova ancora dentro l'ESP32-C6 sulla breadboard. Il filo da seguire è che, a ogni tappa, qualcuno ha trovato un modo nuovo di fare una di due cose: *rappresentare* l'informazione in un oggetto fisico, oppure *prescrivere* una sequenza di operazioni da eseguire senza doverci più pensare.

3.1 Che cos'è l'informatica?

Definizione 3.1 (Informatica)

L'informatica è la scienza applicata che si occupa del trattamento dell'informazione mediante procedure automatizzate.

La parola *informatica* viene dal francese *informatique*, contrazione di *information automatique*. L'inglese dice *computer science* e, sempre più spesso, *informatics*. Lo strumento della disciplina si chiama *elaboratore* o *calcolatore* in italiano, *ordinateur* in francese, *computer*

in inglese. Il nome inglese della disciplina ha il difetto di mettere lo strumento al posto dell'oggetto di studio, un difetto che una frase attribuita a Edsger W. Dijkstra corregge:

> *L'informatica non riguarda i calcolatori più di quanto l'astronomia riguardi i telescopi.*

La macchina è lo strumento, non l'oggetto di studio. In questo libro lo strumento viene studiato da vicino proprio perché smetta di essere un mistero.

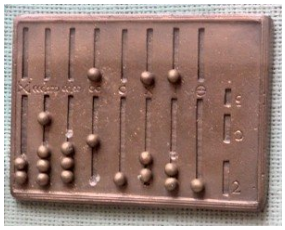
La disciplina ha due anime. L'*informatica teorica* studia i fondamenti dell'informazione e della computazione a livello logico: che cosa si può calcolare, e a quale costo. Il suo oggetto centrale è l'*algoritmo*, un insieme finito di operazioni che descrive la trasformazione di un dato A in un risultato B. L'*informatica applicata* studia le tecnologie abilitanti e i sistemi informativi costruiti con esse. È pervasiva, è un motore dello sviluppo economico e dell'intero settore delle tecnologie dell'informazione e della comunicazione, è una disciplina STEM al centro di una rivoluzione industriale; e ha un costo sociale che va governato, il *divario digitale* tra chi ha accesso a queste tecnologie e chi non lo ha.

3.2 Prima dell'elettricità

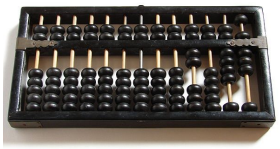
Da tremila anni si costruiscono macchine che aiutano a calcolare.

3.2.1 Le origini: l'abaco e il meccanismo di Anticitera

Babilonesi, Cinesi, Greci e Romani usavano tutti dei pallottolieri (Figura 3.1). In un abaco un numero *è* una posizione di palline, e un'addizione *è* un movimento delle dita: il dispositivo è una rappresentazione fisica di dati e di operazioni. Il meccanismo di Anticitera, recuperato da un relitto e costruito tra il secondo e il primo secolo avanti Cristo (la maggior parte delle stime cade tra il 205 e il 60 a.C. circa), è il più antico calcolatore a ingranaggi conosciuto: i suoi quadranti calcolavano posizioni astronomiche ed eclissi, e in esso un numero è l'angolo di una ruota dentata.



(a) Abaco romano tascabile



(b) *Suanpan* cinese



(c) Meccanismo di Anticitera



(d) Una delle sue lastre

Figura 3.1. Le origini: un numero diventa una posizione di palline o l'angolo di un ingranaggio.

Concetto

La rappresentazione viene prima del calcolo. Prima di poter calcolare qualcosa, un oggetto fisico deve stare per un numero.

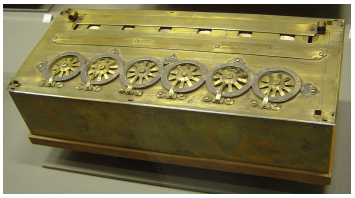
3.2.2 Il mondo islamico medievale

Tra il nono e il tredicesimo secolo gli studiosi del mondo islamico hanno conservato la scienza dell'antichità e vi hanno aggiunto tre cose che riguardano questa storia. La prima è una famiglia di strumenti astronomici, l'astrolabio meccanico (Figura 3.2a) e il torquetum, che sono calcolatori analogici per le posizioni degli astri. La seconda è la crittoanalisi: nel nono secolo al-Kindi descrisse l'analisi delle frequenze, il metodo per violare un cifrario contando quante volte compare ciascun simbolo. La terza è la macchina programmabile: il suonatore di flauto automatico dei fratelli Banū Mūsā, e gli orologi e gli automi a pesi di al-Jazari (1206), il cui comportamento si poteva cambiare spostando dei pioli.

La stessa parola *algoritmo* viene dal nome del matematico al-Khwārizmī, come *algebra* viene dal titolo del suo libro.



(a) Un astrolabio di ottone



(b) Una Pascalina



(c) La calcolatrice di Leibniz

Figura 3.2. Strumenti e calcolatrici prima del 1700.

3.2.3 Dal 1600 al 1800: calcolatrici meccaniche e logica

Il Seicento costruì le prime calcolatrici meccaniche. Le tavole dei logaritmi di Nepero (1614) trasformarono le moltiplicazioni in addizioni. La *Pascalina* di Blaise Pascal (1642) sommava e sottraeva, e la sua vera invenzione fu il *riporto* automatico: ogni ruota è una cifra decimale (Figura 3.2b), e una ruota che passa da 9 a 0 spinge avanti di uno scatto la vicina, esattamente come farà con i bit il sommatore completo del Capitolo **??**. La calcolatrice a cilindri scalati di Leibniz (1673) eseguiva le quattro operazioni.

Lo stesso Leibniz descrisse l'aritmetica binaria nel 1703, e un secolo e mezzo più tardi George Boole, in *The Laws of Thought* (1854), mostrò che la logica si può trattare come un'algebra (Boole, 1854): l'argomento dei Capitoli **??** e **??**.

Nel frattempo una macchina che non aveva nulla a che fare con i numeri introdusse il programma memorizzato. Nel telaio di Jacquard (1804) il disegno del tessuto è registrato su una catena di schede perforate: si cambiano le schede, e lo stesso telaio tesse un disegno diverso.

3.2.4 L'Ottocento: Charles Babbage e Ada Lovelace

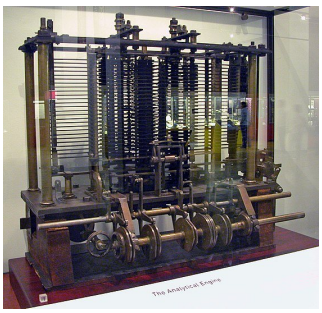
Charles Babbage unì le due linee (Figura 3.3). La sua *Difference Engine*, la macchina alle differenze, calcolava automaticamente tavole matematiche. La sua *Analytical Engine*, la macchina analitica, progettata ma mai completata, era un calcolatore di uso generale in tutto tranne che nella tecnologia: aveva un *mill*, il mulino, cioè l'unità aritmetica; un'unità

di controllo; uno *store*, il magazzino, cioè la memoria; e un dispositivo di uscita. Programmi e dati dovevano essere forniti su schede perforate, prese in prestito dal telaio di Jacquard.

Ada Lovelace ideò un metodo per programmare la macchina analitica. Le sue note del 1843 contengono quello che è considerato il primo programma pubblicato, per il calcolo dei numeri di Bernoulli, e lei è considerata la prima programmatrice della storia.



(a) Difference Engine



(b) Analytical Engine



(c) Schede perforate

Figura 3.3. Le macchine di Charles Babbage.

3.3 Il Novecento

Il Novecento ha prodotto l'idea di una macchina universale, e poi le macchine che l'hanno resa reale. La teoria è venuta prima.

3.3.1 Turing, Shannon e Nakajima

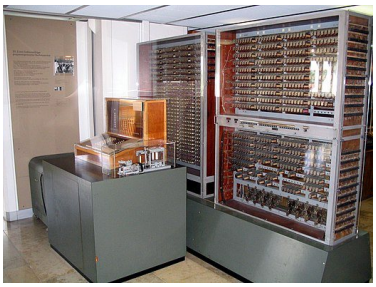
Nel 1936 Alan Turing definì una macchina astratta universale, e con essa la nozione di ciò che si può calcolare (Turing, 1936). Durante la guerra lavorò a Bletchley Park alla violazione del cifrario tedesco Enigma (Figura 3.4a).

Negli stessi anni Akira Nakajima in Giappone e Claude Shannon negli Stati Uniti fondarono la teoria dei circuiti a commutazione: l'algebra di Boole descrive i circuiti fatti di interruttori (Shannon, 1938). È il principio su cui poggia tutta la progettazione digitale, e il ponte tra la logica dei Capitoli **??**–**??** e i circuiti dei Capitoli **??**–**??**.

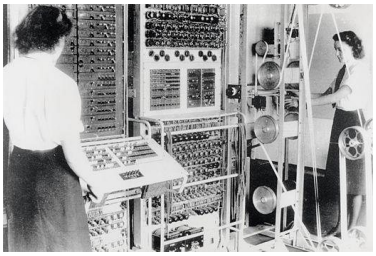
In Germania Konrad Zuse costruì lo Z3 (1941), il primo calcolatore funzionante controllato da un programma, di cui nel 1998 è stata dimostrata la completezza secondo Turing; il suo successore, lo Z4, fu uno dei primi calcolatori venduti commercialmente.



(a) Una macchina Enigma



(b) Zuse Z3 (replica)



(c) Colossus

Figura 3.4. Macchine degli anni Quaranta.

3.3.2 John von Neumann: il calcolatore a programma memorizzato

Nel 1945 il rapporto sull'EDVAC, firmato da John von Neumann, descrisse un'architettura hardware per calcolatori digitali programmabili (von Neumann, 1945). Il suo cuore è un'idea sola.

Il programma memorizzato

Programma e dati condividono la stessa memoria: *un programma è un dato*. Le istruzioni sono numeri conservati in memoria, prelevati ed eseguiti uno dopo l'altro.

L'architettura ha quattro unità essenziali (Figura 3.5): un'*unità di controllo*, che governa la sequenza delle operazioni; un'*unità aritmetico-logica* (ALU), con il suo registro accumulatore; una *memoria*; e dispositivi di *ingresso* e di *uscita*.

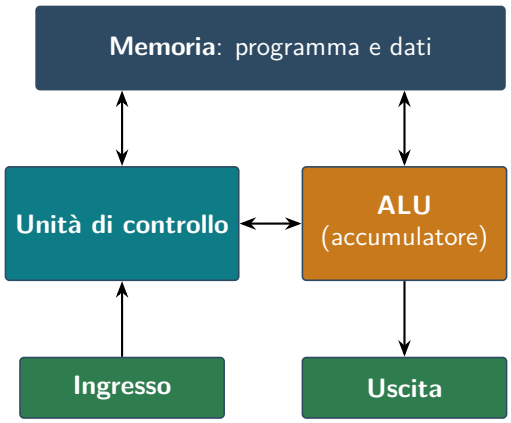


Figura 3.5. Le quattro unità dell'architettura di von Neumann.

Ritorno alla macchina

L'ESP32-C6 è ancora una macchina di von Neumann, e le sue quattro unità erano tutte al lavoro nel Capitolo 2. I registri e la ALU hanno eseguito `addi t1, t1, 1`; l'unità di controllo ha deciso che l'istruzione successiva era `sw`; la memoria conteneva la variabile `counter` *e le istruzioni stesse*, tra cui il numero `0x00130313`; il joystick e il display sono ingresso e uscita.

3.3.3 Otto decenni di calcolatori

Da allora ogni decennio ha reso il calcolatore più piccolo, più economico e più vicino alle persone (Tabella 3.1, Figura 3.6); nessuno ha cambiato l'idea del programma memorizzato.

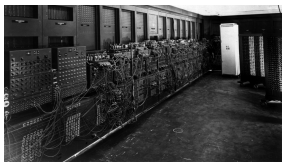
3.4 Dagli home computer a RISC-V

3.4.1 I microprocessori: da CISC a RISC a uno standard aperto

Il processore del kit PRG32 è fatto com'è per una sequenza di decisioni prese nell'arco di cinquant'anni (Tabella 3.2). I primi microprocessori sono cresciuti per accumulo: ogni generazione aggiungeva istruzioni, spesso complicate, per facilitare il lavoro dei compilatori e dei programmatori in assembly. Il risultato si chiama *complex instruction set computer*

Tabella 3.1. Otto decenni di calcolatori.

Decennio	Epoca	Che cosa è cambiato
Anni ’40	La guerra	ENIAC (USA) e Colossus (Regno Unito), 1943–45
Anni ’50	Militari e governi	EDVAC, 1949: il programma memorizzato
Anni ’60	I mainframe	una macchina, molti utenti
Anni ’70	I minicomputer	un calcolatore per reparto
Anni ’80	Il personal computer	un calcolatore per persona
Anni ’90	Internet	i calcolatori si collegano
Anni 2000	Smartphone e tablet	un calcolatore in tasca
Anni 2010	Il cloud computing	il calcolo come servizio
Anni 2020	Internet delle cose e IA	calcolatori ovunque



(a) ENIAC



(b) Un mainframe



(c) Un minicomputer



(d) Commodore 64

Figura 3.6. Da una stanza a una scrivania in quarant’anni.

(CISC), e la famiglia x86, nata nel 1978, porta ancora questa eredità nei personal computer di oggi.

All’inizio degli anni Ottanta due progetti universitari, a Berkeley e a Stanford, misurarono che cosa fanno davvero i programmi, e scoprirono che un processore con istruzioni poche, semplici e regolari può essere reso più veloce di uno con molte istruzioni complicate. È l’idea del *reduced instruction set computer* (RISC), per la quale David Patterson e John Hennessy hanno ricevuto il premio Turing nel 2017. ARM l’ha applicata al basso consumo, e i core ARM sono oggi in quasi ogni telefono. Nel 2010 il quinto progetto RISC di Berkeley è diventato qualcosa di nuovo: un insieme di istruzioni pubblicato come *standard aperto*.

Definizione 3.2 (RISC-V)

RISC-V è una *instruction set architecture*, un’architettura dell’insieme di istruzioni, a standard aperto: la specifica delle istruzioni, dei registri e della loro codifica è pubblica, e chiunque può progettare, costruire e vendere un processore che la realizza senza pagare una licenza per l’architettura.

RISC-V quindi non è un processore, e non è stato il primo RISC: il RISC di Berkeley, MIPS e ARM sono venuti prima. Ciò che lo distingue da x86 e da ARM è l’apertura.

3.4.2 La legge di Moore in un numero

L’Intel 4004 del 1971 aveva 2 300 transistor. Un processore per portatili di oggi ne ha decine di miliardi: una crescita di un fattore di circa dieci milioni. Per cinquant’anni il numero di transistor su un chip è raddoppiato circa ogni due anni, un’osservazione fatta da Gordon

Tabella 3.2. I microprocessori: da CISC a RISC a uno standard aperto.

Anno	Tappa	Perché conta
1971	Intel 4004	il primo microprocessore commerciale: 2 300 transistor su un chip
1975	MOS 6502	una CPU a 8 bit economica: Apple II, Commodore 64, l'era degli home computer
1978	Intel 8086	l'inizio di x86, un insieme di istruzioni complesso (CISC) ancora nei PC di oggi
1980–82	RISC di Berkeley e MIPS di Stanford	istruzioni poche, semplici e regolari vanno più veloci: l'idea RISC
1985	ARM1	RISC per il basso consumo: oggi in quasi ogni telefono
2010	RISC-V nasce a UC Berkeley	il quinto progetto RISC di Berkeley diventa un insieme di istruzioni aperto
2015–20	RISC-V Foundation, poi RISC-V International	una specifica condivisa, governata dalla Svizzera
2023–25	European Chips Act; il progetto DARE	RISC-V come strategia per la sovranità tecnologica europea

Moore nel 1965 e rivista nel 1975 (Moore, 1965). È un'osservazione su un'industria, non una legge di natura, e i suoi limiti fisici sono ormai vicini. Il progresso viene sempre più da architetture migliori, da acceleratori specializzati e da progetti aperti, ed è per questo che l'architettura, l'argomento di questo libro, conta più che mai. Il Capitolo **??** riprende la storia da qui.

3.4.3 Lo spirito degli home computer, su una CPU moderna e aperta

Gli home computer degli anni Ottanta avevano una qualità che le macchine successive hanno perso: una persona poteva capirne uno per intero. PRG32 è stato progettato per restituire quella qualità, sull'architettura che gli studenti incontreranno nell'industria (Tabella 3.3). Lo schermo ha la stessa dimensione, e la stessa immediatezza, a quarant'anni di distanza.

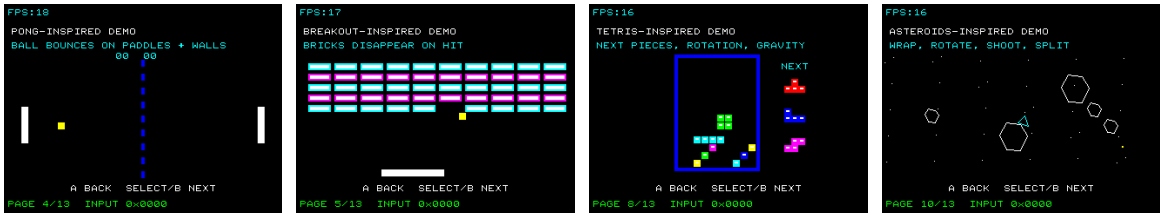
Tabella 3.3. Un Commodore 64 e PRG32.

	Commodore 64 (1982), MOS 6510	**PRG32 su ESP32-C6, RISC-V**
Dimensione della parola	8 bit	32 bit
Registri	A, X, Y (più puntatore allo stack, stato, PC)	32 registri di uso generale, da `x0` a `x31`
Clock	circa 1 MHz	160 MHz
Spazio di indirizzamento	64 KB	4 GB
Insieme di istruzioni	56 istruzioni, proprietario	una quarantina di istruzioni di base più le estensioni, standard aperto
Schermo	320 per 200 pixel, 16 colori	viewport di 320 per 200, palette RGB565 di 256 voci
Si programma	scrivendo sulla macchina stessa	compilando una cartuccia e caricandola

Il 6502 è ancora un piacere da esplorare, per esempio con il simulatore su https://skilldrick.github.io/easy6502.

3.4.4 I giochi rétro come esemplari

Un biologo studia il moscerino della frutta non perché i moscerini siano importanti, ma perché un moscerino è abbastanza piccolo da essere capito per intero e abbastanza ricco da mostrare come funziona l'ereditarietà. Un gioco rétro svolge qui la stessa parte (Figura 3.7): è abbastanza piccolo da tenerlo in mente e abbastanza ricco da mettere in esercizio stato, ingresso, aritmetica, decisioni e un ciclo, cioè l'intero corso. I quattro giochi della figura sono cartucce di esempio del repository di PRG32 (`examples/games`).



(a) Pong **(b)** Breakout **(c)** Tetris **(d)** Asteroids

Figura 3.7. I giochi rétro come esemplari.

3.5 L'informatica come scienza applicata

Dove si colloca l'informatica tra le scienze? Un esempio dal Dipartimento in cui questo libro è stato scritto rende concreta la risposta (Figura 3.8). Sul mare si forma una tempesta: un fenomeno naturale. La fisica e la matematica, le scienze di base, lo descrivono con delle equazioni. L'informatica, una scienza applicata, trasforma le equazioni in algoritmi e simulazioni. L'ingegneria costruisce i supercalcolatori, le reti e i sensori che le eseguono in modo affidabile. E l'economia riceve previsioni e servizi che raggiungono le persone e creano valore. Le previsioni meteorologiche e marine di https://meteo.uniparthenope.it sono calcolate ogni giorno in questo modo, sui sistemi ad alte prestazioni del Dipartimento.

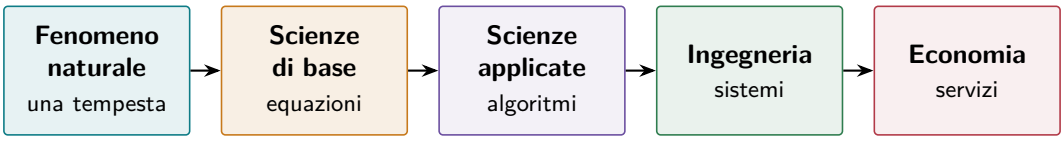


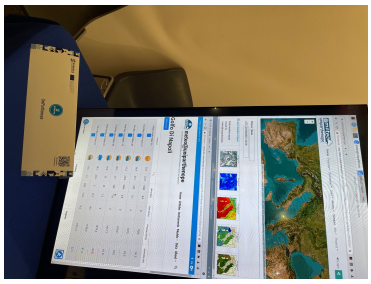
Figura 3.8. Dove si colloca l'informatica.

Le stesse idee che in questo libro si esercitano su un microcontrollore a 160 MHz scalano fino ai cluster (Figura 3.9): registri, memoria, istruzioni, solo in numero molto maggiore.

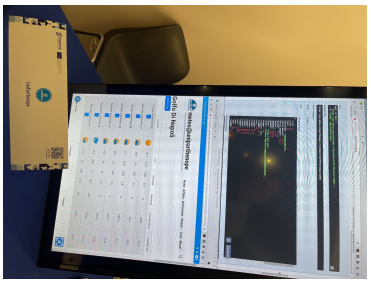
3.6 Verificate di aver capito

D3.1 Quale idea è il cuore dell'architettura di von Neumann?

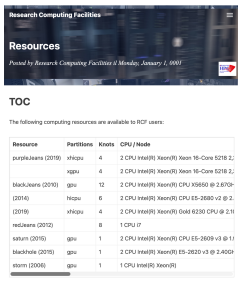
A. Il processore ha esattamente quattro registri
B. Programma e dati sono memorizzati nella stessa memoria
C. Ogni istruzione richiede un ciclo di clock



(a) Un cluster didattico



(b) Una simulazione parallela



(c) rcf.uniparthenope.it

Figura 3.9. Il calcolo ad alte prestazioni alla Parthenope: mini PC con GPU su una rete a 10 Gbit, e le Research Computing Facilities.

D. Ingresso e uscita sono facoltativi

D3.2 Che cosa rende RISC-V diverso da x86 e ARM?

A. È l'insieme di istruzioni più veloce
B. È stata la prima architettura RISC
C. Il suo insieme di istruzioni è uno standard aperto, libero da realizzare
D. Funziona solo sui microcontrollori

D3.3 Chi è considerato il primo programmatore della storia?

A. Charles Babbage
B. Alan Turing
C. Ada Lovelace
D. Konrad Zuse

3.7 Esercizi

Esercizio 3.1 (prima su carta, poi su PRG32)

1. Disegnate le quattro unità dell'architettura di von Neumann e scrivete, dentro ciascuna, una cosa del programma del Capitolo 2 che vi risiede o vi accade.
2. Per ciascuna delle seguenti cose, dite se è un modo nuovo di *rappresentare* l'informazione o un modo nuovo di *prescrivere* operazioni: l'abaco, il telaio di Jacquard, la Pascalina, l'algebra di Boole, le schede perforate della macchina analitica, il programma memorizzato.
3. Scaricate dallo Store due dei quattro giochi della Figura 3.7, oppure compilateli da `examples/games`. Con i contatori del debugger, trovate quante istruzioni esegue ciascuno per frame, e confrontate i due giochi.

Risultato atteso. Registri e ALU: lì è stata eseguita `addi`. Memoria: il contatore, e le istruzioni stesse. Unità di controllo: il prelievo di ogni istruzione. Ingresso/uscita: la barra sullo schermo. Il programma memorizzato appartiene a entrambe le classi: un programma è esso stesso informazione rappresentata.

3.8 Glossario

Inglese	**Italiano**
computer science / informatics	informatica
computer	calcolatore, elaboratore
punched card	scheda perforata
loom	telaio
mainframe	mainframe, grande elaboratore
stored program	programma memorizzato
control unit	unità di controllo
arithmetic logic unit	unità aritmetico-logica
instruction set	insieme (set) di istruzioni
high-performance computing	calcolo ad alte prestazioni

3.9 Conclusioni e letture

La computazione è più antica dell'elettronica: rappresentazione e procedura sono venute prima. Il calcolatore a programma memorizzato del 1945 è ancora la macchina che si programma oggi. RISC ha reso i processori abbastanza semplici da essere capiti per intero; RISC-V li ha resi aperti. L'informatica è pervasiva: l'informazione digitale appartiene a ogni campo, i dati ne sono la moneta, e gli algoritmi trasformano i dati in decisioni.

Letture. Patterson e Hennessy (2020), capitolo 1; *Playing with RISC-V*, capitolo 1, «Why a Game, Why RISC-V, Why Now» (Montella et al., 2026b); Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*; per la storia in profondità, Ceruzzi (2003).

CAPITOLO 4

Rappresentare l'informazione digitale

"printf("Hello, world!");"
— Brian Kernighan e Dennis Ritchie, *The C Programming Language*, 1978

In questo capitolo

Alla fine di questo capitolo saprete

- spiegare la differenza tra informazione analogica e digitale;
- leggere una configurazione di bit come numero naturale, in binario e in esadecimale;
- decodificare un colore di PRG32 come `0x07E0` nei suoi campi rosso, verde e blu;
- dire come caratteri e stringhe sono memorizzati, in ASCII e in UTF-8;
- descrivere come un vettore vive in memoria e come l'assembly ne raggiunge un elemento;
- distinguere un algoritmo deterministico da uno probabilistico, e collocare tra essi l'apprendimento automatico.

Nel Capitolo 2 il lettore ha scritto `0x07e0` e ha ottenuto il verde, ha scritto 255 e ha ottenuto una barra che ricomincia, ha scritto `.asciz` e ha ottenuto un saluto, ha scritto `.word` e ha ottenuto un contatore. Questo capitolo dà a quelle cose il loro nome. La sua tesi sta in una frase, che il resto del capitolo illustra quattro volte: *nella macchina tutto è una configurazione di bit, e il significato di una configurazione è una convenzione.* I due capitoli successivi trattano formalmente i numeri.

4.1 L'informazione

4.1.1 Il flusso dei dati

Ogni uso di un calcolatore ha la stessa forma. Dei *dati* entrano dal mondo: un tasto, un sensore, un file. Un *programma*, l'incarnazione di un algoritmo, li trasforma. Dei *risultati* escono: un'immagine, un suono, un numero. E qualcuno riceve *conoscenza*, cioè informazione in base alla quale è possibile agire. Su PRG32 le quattro fasi sono, nell'ordine, la funzione `prg32_input_read`, la `update` del programmatore, la `draw` del programmatore, e il giocatore che reagisce allo schermo.

L'ultima fase ha richiesto, finora, una persona. L'intelligenza artificiale, a cui torna la fine di questo capitolo, è un'intelligenza di un tipo suo proprio: un potente riconoscimento di schemi, senza autocoscienza e senza esperienza soggettiva.

4.1.2 Informazione analogica e digitale

Definizione 4.1 (Analogico e digitale)

Una grandezza *analogica* varia con continuità: una tensione, una pressione sonora, le lancette di un orologio. Una grandezza *digitale* è una sequenza di simboli presi da un alfabeto finito: le cifre di un display.

Digitalizzare una grandezza analogica significa farle due cose (Figura 4.1): *campionarla* nel tempo, leggendone il valore a istanti regolari, e *quantizzarla* nel valore, sostituendo ogni lettura con il più vicino tra un numero finito di livelli.

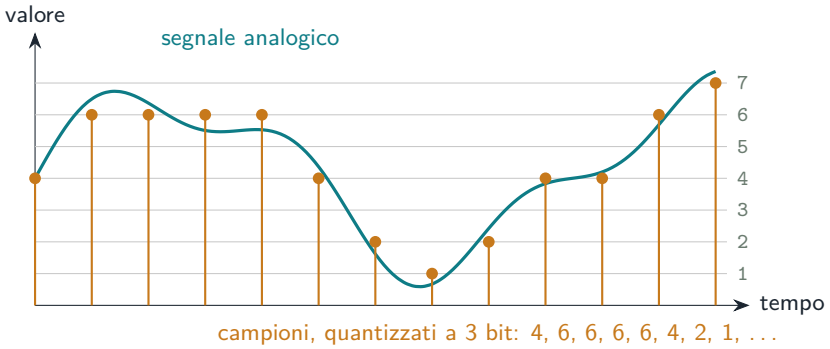


Figura 4.1. Digitalizzare: campionare nel tempo e quantizzare nel valore.

In entrambi i passaggi si perde qualcosa, e si guadagna qualcosa di decisivo. Un numero finito di livelli può essere *contato*, *copiato esattamente* e *controllato per rilevare errori*; una grandezza continua si può soltanto misurare, e ogni copia la degrada. Con due livelli il simbolo è il *bit*, che vale 0 oppure 1. Due livelli sono facili da realizzare in modo affidabile con i transistor, che sono spenti o accesi (Capitolo **??**), ed è per questo che i calcolatori sono binari. L'algebra di Boole (Capitolo **??**) è la matematica dei due livelli.

4.1.3 Bit, byte, parola

I bit si trattano a gruppi di dimensioni standard (Tabella 4.1). Il fatto fondamentale che li riguarda è un conteggio.

Teorema 4.1 (Numero di configurazioni)

Con n bit si possono scrivere esattamente 2^n configurazioni distinte.

Dimostrazione. Per induzione su n. Un bit ha $2 = 2^1$ configurazioni. Se n bit hanno 2^n configurazioni, ciascuna può essere seguita da uno 0 o da un 1, il che dà $2 \cdot 2^n = 2^{n+1}$ configurazioni di $n + 1$ bit. □

Tabella 4.1. Unità di informazione.

Unità	Dimensione	Configurazioni	Dove la si è incontrata
bit	1 cifra binaria	2	un pulsante del joystick
nibble	4 bit	16	una cifra esadecimale
byte	8 bit	256	un carattere ASCII; la larghezza della barra (`andi 255`)
mezza parola	16 bit	65 536	un colore RGB565
parola	32 bit	4 294 967 296	un registro RISC-V; un'istruzione; `.word`
KiB, MiB, GiB	2^{10}, 2^{20}, 2^{30} byte	–	dimensioni di memoria: 1 KiB = 1024 byte

Concetto

Il significato di una configurazione è una convenzione, non una proprietà dei bit. Che cosa significa una configurazione non c'è scritto dentro: è un accordo tra chi la scrive e chi la legge.

La dimensione di una parola dipende dall'architettura: su RV32 una parola è di 32 bit per definizione.

4.1.4 Contare in base 2 e in base 16

La Tabella 4.2 elenca le sedici configurazioni di quattro bit con i loro valori in decimale e le loro cifre esadecimali. Le quattro colonne di un numero binario pesano 8, 4, 2 e 1. Questa tabella va imparata a memoria: è la tavola pitagorica di questo libro.

Tabella 4.2. Le sedici configurazioni di quattro bit.

Dec	Binario	Esa	Dec	Binario	Esa
0	`0000`	`0`	8	`1000`	`8`
1	`0001`	`1`	9	`1001`	`9`
2	`0010`	`2`	10	`1010`	`A`
3	`0011`	`3`	11	`1011`	`B`
4	`0100`	`4`	12	`1100`	`C`
5	`0101`	`5`	13	`1101`	`D`
6	`0110`	`6`	14	`1110`	`E`
7	`0111`	`7`	15	`1111`	`F`

4.1.5 La notazione posizionale: che cosa significa `11011001`

In un sistema *posizionale* il valore di una cifra dipende dalla sua posizione: ogni posizione pesa una potenza della base. La base dieci usa le potenze di dieci, la base due le potenze di due, la base sedici le potenze di sedici. La regola è la stessa; cambia solo la base. Il byte `11011001` si legge come segue.

peso	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1
bit	**1**	**1**	0	**1**	**1**	0	0	**1**
contributo	128	64	0	16	8	0	0	1

$$128 + 64 + 16 + 8 + 1 = 217$$

Il Capitolo 5 dà la formula generale e gli algoritmi di conversione. Qui lo scopo è soltanto leggere un byte.

4.1.6 Perché i programmatori scrivono in esadecimale

Una cifra esadecimale corrisponde esattamente a quattro bit, perché $16 = 2^4$. Convertire tra binario ed esadecimale è quindi una consultazione di tabella, cifra per cifra, senza aritmetica:

$$\underbrace{\texttt{0000}}_{\texttt{0}}\ \underbrace{\texttt{0111}}_{\texttt{7}}\ \underbrace{\texttt{1110}}_{\texttt{E}}\ \underbrace{\texttt{0000}}_{\texttt{0}} \quad \Longrightarrow \quad \texttt{0x07E0}.$$

Un registro di 32 bit sono otto cifre esadecimali: `0x00130313` è leggibile, trentadue zeri e uni non lo sono. Il prefisso `0x` indica l'esadecimale in assembly e in C, e il debugger di PRG32-QT mostra i registri in questo modo.

Il valore decimale di `0x07E0` è $7 \cdot 256 + 14 \cdot 16 = 2016$, e non serve assolutamente a nulla: ciò che conta in questo numero è la *configurazione di bit*, come mostra la prossima sezione.

4.1.7 Un colore è un numero: RGB565

Su PRG32 un colore sono sedici bit divisi in tre *campi*: cinque bit di rosso, sei di verde, cinque di blu (Figura 4.2). Il formato si chiama RGB565. La costante `.equ GREEN, 0x07e0` è la configurazione in cui il verde è al massimo, 63 su 63, e rosso e blu sono a zero. L'occhio è più sensibile al verde, ed è per questo che il verde riceve il bit in più.

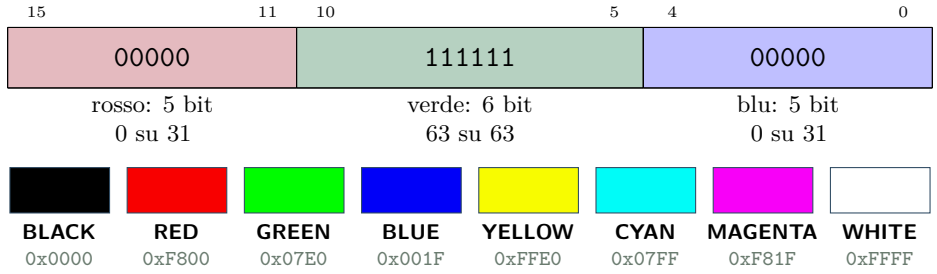


Figura 4.2. RGB565: `0x07E0` è il verde. Sotto, le costanti di colore di `prg32.h`.

Sedici bit, tre campi. Gli stessi sedici bit potrebbero essere un numero, due caratteri o mezza istruzione: il significato sta nel modo in cui il programma li usa.

4.1.8 I numeri: naturali, interi, reali

Gli stessi numeri sono rappresentati con tre convenzioni diverse, che i due capitoli successivi sviluppano.

- I **numeri naturali** sono la configurazione posizionale stessa: n bit coprono l'intervallo da 0 a $2^n - 1$.

- Gli **interi con segno** si scrivono in *complemento a due*: un registro di 32 bit copre da circa -2 miliardi a circa $+2$ miliardi (Capitolo 5).
- I **numeri reali** si scrivono in *virgola fissa* oppure nella *virgola mobile* dello standard IEEE 754, in cui il valore è $(-1)^s \times 1{,}f \times 2^{e-bias}$ (Capitolo 6). La precisione singola usa 32 bit, dà circa 7 cifre decimali e arriva a circa $3{,}4 \times 10^{38}$; la precisione doppia usa 64 bit, dà circa 16 cifre e arriva a circa $1{,}8 \times 10^{308}$.

4.2 Il testo

Anche i caratteri sono numeri.

4.2.1 ASCII: un carattere, un byte

Il codice ASCII (1963) assegna un numero tra 0 e 127, sette bit, a ciascuno di 128 caratteri: le lettere dell'inglese, le cifre, la punteggiatura e alcuni codici di controllo (Tabella 4.3).

Tabella 4.3. Alcuni caratteri ASCII.

Carattere	Decimale	Esa	Binario	Nota
`A`	65	`0x41`	`0100 0001`	le maiuscole cominciano da 65
`a`	97	`0x61`	`0110 0001`	come `A` con un bit in più: $97 - 65 = 32$
`0`	48	`0x30`	`0011 0000`	il carattere zero non è il numero zero
spazio	32	`0x20`	`0010 0000`	un carattere come un altro
a capo	10	`0x0A`	`0000 1010`	un carattere di controllo
NUL	0	`0x00`	`0000 0000`	segna la fine di una stringa C

La tabella è stata progettata con cura. Maiuscole e minuscole differiscono solo nel bit 5, così che `xori` con `0x20` scambia maiuscola e minuscola: un primo assaggio di algebra di Boole applicata al testo. E i caratteri delle cifre sono consecutivi, così che sottraendo 48 dal carattere di una cifra si ottiene il valore della cifra.

4.2.2 Stringhe: una sequenza di caratteri in memoria

Una *stringa* è una sequenza di caratteri memorizzati in byte consecutivi. L'unica questione è come fa chi legge la stringa a sapere dove finisce, e ci sono due risposte classiche (Figura 4.3). La *stringa C* segna la fine con un byte di valore zero. La *stringa Pascal* mette prima la lunghezza.

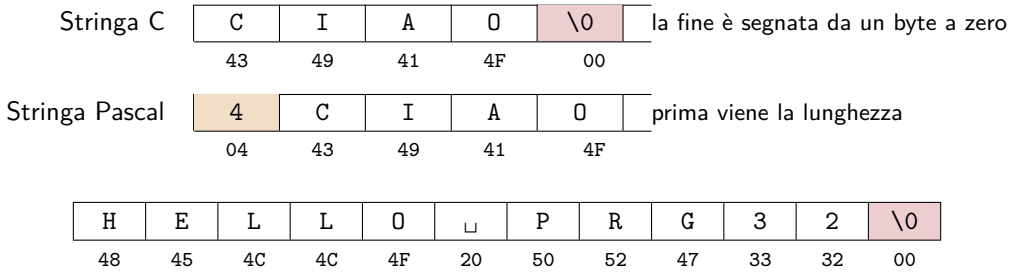


Figura 4.3. Stringhe in memoria; sotto ogni cella, il codice ASCII in esadecimale. Sotto: `.asciz "HELLO PRG32"`.

La `z` di `.asciz` significa *zero-terminated*, terminata da zero: per gli undici caratteri della stringa del Capitolo 2 l'assemblatore memorizza dodici byte. La funzione `prg32_gfx_text8` riceve in `a2` soltanto l'indirizzo del primo byte, e legge finché non trova lo zero. Se lo zero mancasse, la funzione continuerebbe a leggere ciò che segue in memoria, qualunque cosa sia.

4.2.3 Unicode e UTF-8: ogni sistema di scrittura

Centoventotto caratteri bastano per l'inglese e per nient'altro. *Unicode* assegna un numero, detto *code point*, a ogni carattere di ogni scrittura; *UTF-8* memorizza quel numero in un numero di byte da uno a quattro (Tabella 4.4). La codifica è compatibile con ASCII: un testo ASCII puro è già UTF-8 valido. È la codifica del web e dei file sorgente che il lettore sta scrivendo.

Tabella 4.4. Code point Unicode e loro codifica UTF-8.

Carattere	Code point	Byte UTF-8	Lunghezza
A	U+0041	`41`	1 byte: identico ad ASCII
è	U+00E8	`C3 A8`	2 byte
€	U+20AC	`E2 82 AC`	3 byte
l'ideogramma per «sole», «giorno»	U+65E5	`E6 97 A5`	3 byte
l'emoji della «faccina sorridente»	U+1F600	`F0 9F 98 80`	4 byte

Attenzione

In UTF-8 la lunghezza di una stringa in byte non è la sua lunghezza in caratteri.

4.2.4 L'ordine dei byte: little-endian

Una parola sono quattro byte, e la memoria è indirizzata byte per byte. In che ordine vanno i quattro byte di una parola? La direttiva `.word 0x12345678` si può memorizzare in due modi (Figura 4.4).

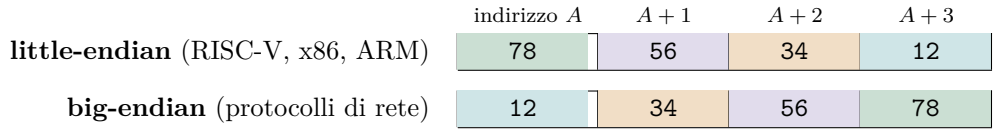


Figura 4.4. I due ordini dei byte della parola `0x12345678`.

Definizione 4.2 (Little-endian e big-endian)

Nell'ordine *little-endian* il byte meno significativo di una parola è memorizzato all'indirizzo più basso; nell'ordine *big-endian*, il byte più significativo.

RISC-V è little-endian: è ciò che fanno `lw` e `sw`. L'ordine dei byte si nota in due circostanze: quando si esamina la memoria in un debugger, e quando si scambiano dati con un'altra macchina. I nomi vengono dai *Viaggi di Gulliver*, dove due popoli si fanno la guerra per decidere da quale estremità vada rotto un uovo.

4.3 Strutture dati

Dai singoli valori si passa alle collezioni organizzate. Cinque modi di organizzare i dati ricorrono in tutta l'informatica.

Vettore (array).
Elementi dello stesso tipo, uno dopo l'altro, raggiunti tramite un indice: 32, 11, 34, 22.

Lista.
Elementi collegati ciascuno al successivo: economica per inserire e togliere, lenta da raggiungere per posizione.

Pila (stack).
L'ultimo a entrare è il primo a uscire: *push* aggiunge in cima, *pop* toglie dalla cima. Il frame che ha salvato `ra` nel Capitolo 2 stava su uno stack.

Dizionario.
Coppie di chiave e valore: `NA` dà `NAPOLI`, `PA` dà `PALERMO`, `MI` dà `MILANO`.

Albero.
Una gerarchia: un animale cammina o nuota, ha polmoni o branchie, due zampe o quattro.

Nella macchina sono tutti la stessa cosa: byte in memoria, più una regola per trovare l'elemento successivo.

4.3.1 Un vettore in memoria, un elemento in un registro

Per il vettore la regola è aritmetica (Listato 4.1). Un vettore è un indirizzo di base seguito da elementi di uguale dimensione; l'elemento i di un vettore di parole sta all'indirizzo $base + 4i$, perché una parola sono quattro byte.

```
.section .data
scores: .word 32, 11, 34, 22     # four 32-bit words, 16 bytes

.section .text
    la   t0, scores      # t0 = ADDRESS of element 0
    li   t1, 2           # index i = 2
    slli t1, t1, 2       # offset = i * 4 bytes (shift left by 2)
    add  t0, t0, t1      # ADDRESS of element i
    lw   a0, 0(t0)       # a0 = VALUE of scores[2] = 34
```

Listato 4.1. Leggere `scores[2]`.

L'istruzione `slli` fa scorrere a sinistra i bit di un registro; uno scorrimento di due posizioni moltiplica per quattro, come in decimale uno scorrimento di una posizione moltiplica per dieci. (La proprietà è dimostrata nel Capitolo 5.) L'espressione `scores[i]` di un linguaggio di alto livello è esattamente queste quattro istruzioni.

Attenzione

Che cosa legge `lw` se $i = 4$? Ciò che segue il vettore in memoria, qualunque cosa sia. La macchina non controlla i limiti: lo fa il programmatore.

4.4 Dati e algoritmi

Il capitolo si chiude allargando lo sguardo, dai bit di un programma ai dati del mondo e alle procedure che lavorano su di essi.

4.4.1 Dati, big data e data science

Definizione 4.3 (Big data)

I *big data* sono dati che superano i limiti degli strumenti tradizionali delle basi di dati.

Sono strutturati e non strutturati: l'economia, la geolocalizzazione, le serie temporali, le reti sociali. La loro dimensione si misura in petabyte e zettabyte, cioè in miliardi di terabyte. Vengono da quattro tipi di sorgente: i *dati da sensori* (l'ambiente, le posizioni GPS, l'Internet delle cose: ogni dispositivo misura qualcosa); le *transazioni* (gli acquisti al supermercato, le carte di credito, gli ordini online); i *dati pubblici* (atti amministrativi, territorio, consumi: gli open data delle istituzioni); e i *social media* (preferenze, politica, opinioni: ciò che le persone scrivono e condividono). La *data science* è l'analisi di grandi quantità di informazione, e le tecnologie che estraggono conoscenza e valore dai dati sono il calcolo parallelo e le GPU.

Definizione 4.4 (Data mining)

Il *data mining* è l'insieme delle tecniche e delle metodologie per estrarre informazione o conoscenza da grandi quantità di dati.

Tra le sue tecniche ci sono gli alberi di decisione e la classificazione, l'apprendimento di regole e i metodi bayesiani, le reti neurali e il riconoscimento di schemi.

4.4.2 GPGPU: il calcolo generico sui processori grafici

Un processore grafico (GPU) ha migliaia di core semplici costruiti per fare la stessa operazione su molti dati insieme. La somma di N elementi, che richiede N passi uno dopo l'altro, ha bisogno di soli $\log_2 N$ passi paralleli (Figura 4.5): gli elementi si sommano a coppie, poi le somme parziali a coppie, e così via. Lo stesso hardware che disegna i videogiochi oggi addestra le reti neurali. Sotto ogni core ci sono sempre registri, istruzioni e bit; il Capitolo **??** torna sull'argomento.

4.4.3 Algoritmi e calcolabilità

Un problema è *calcolabile* se può essere risolto da un algoritmo. La *macchina di Turing* è un calcolatore ideale con un programma, espresso matematicamente tramite insiemi, relazioni e funzioni. La *macchina di von Neumann* è l'architettura di calcolo capace di eseguire un programma che realizza un algoritmo per la macchina di Turing. Un problema che nessun algoritmo su una macchina di Turing può risolvere non può essere risolto da alcun calcolatore: questo enunciato, nella sua versione informale, è la tesi di Church–Turing. Non ogni problema ha un algoritmo; l'esempio classico di problema non calcolabile è il *problema della fermata*, quello di decidere se un programma arbitrario prima o poi si fermerà.

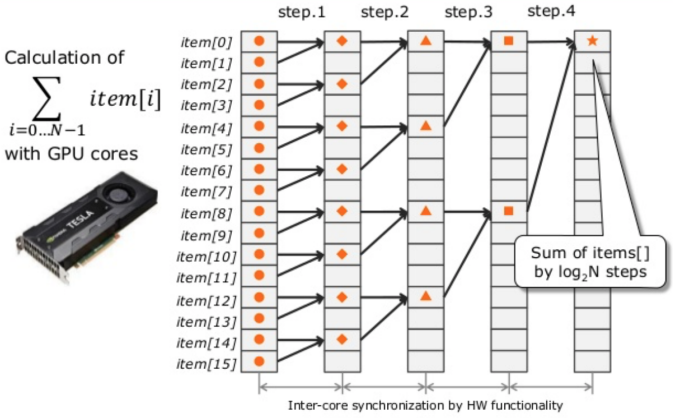


Figura 4.5. Una somma parallela sui core di una GPU: $\log_2 N$ passi.

Definizione 4.5 (Algoritmo)

Un algoritmo è una procedura con cinque proprietà. È **atomico**: i suoi passi sono elementari e non ulteriormente scomponibili. È **non ambiguo**: ogni passo è interpretato in un solo modo diretto dall'esecutore, umano o macchina. È **finito**: ha un numero finito di passi e una quantità finita di dati in ingresso. È **terminante**: la sua esecuzione finisce dopo un tempo finito. È **effettivo**: la sua esecuzione porta a un risultato unico e ben definito.

Un'istruzione RISC-V è il modello di un passo atomico e non ambiguo: il suo significato è fissato dalla specifica, bit per bit.

4.4.4 Algoritmi deterministici e non deterministici

Un algoritmo è *deterministico* se per ogni istruzione esiste, a parità di ingresso, esattamente un passo successivo: il percorso dell'algoritmo esiste ed è unico, $y = f(x)$. Convertire un'unità di misura è deterministico; lo è anche `first_update`, che somma sempre uno.

Un algoritmo è *non deterministico* se, a parità di ingresso, può comportarsi in modo diverso a ogni esecuzione. Gli algoritmi *probabilistici* usano un generatore di numeri casuali; gli algoritmi *concorrenti* possono dipendere dai tempi (le cosiddette race condition). Un gioco che colloca un nemico chiamando `prg32_random_number` è non deterministico.

Il Listato 4.2 mostra due algoritmi probabilistici in pseudo-codice. Il primo verifica se un numero è primo: la risposta «composto» è sempre giusta, la risposta «probabilmente primo» è giusta con probabilità schiacciante. Scambiare una minuscola possibilità di errore con un enorme guadagno di velocità è il modo in cui la crittografia reale verifica i numeri primi. (Il test di Fermat mostrato è il più semplice; le librerie reali usano il test di Miller–Rabin, che ne chiude le falle.) Il secondo è una passeggiata aleatoria, che dà un risultato diverso a ogni esecuzione: le simulazioni ne fanno la media su molte.

```
is_probably_prime(n):
    repeat 30 times:
        a = random(2, n - 2)
        if a^(n-1) mod n != 1:
            return COMPOSITE          # certain
```

```
    return PROBABLY PRIME            # almost certain

random_walk():
    while t < t_final:
        r = random(-1, 1)
        x = x + r * u                # a different path every run
        y = y + r * v
        t = t + 1
```

Listato 4.2. Due algoritmi probabilistici.

4.4.5 L'apprendimento automatico

L'*apprendimento automatico* (machine learning) è un metodo di analisi dei dati che automatizza la costruzione di un modello analitico: il calcolatore trova schemi nascosti senza essere stato esplicitamente programmato su dove cercare. In un programma classico la funzione $y = f(x)$ è scritta da una persona; nell'apprendimento automatico f viene appresa dagli esempi. I modelli generativi di oggi sono la stessa idea su scala enorme: un grande modello linguistico predice il prossimo pezzo di testo a partire da miliardi di numeri appresi.

Quei numeri sono bit in memoria, moltiplicati e sommati da processori: l'argomento di questo libro. E un modello può sbagliare con grande sicurezza, quindi per controllarlo servono i fondamenti. È la ragione della regola enunciata nel Capitolo 0: l'esame verifica che i fondamenti stiano nella testa dello studente.

4.5 Verificate di aver capito

D4.1 Quanti valori diversi può rappresentare un byte?

A. 8
B. 16
C. 255
D. 256

D4.2 Il colore PRG32 `0xF81F` è:

A. rosso
B. magenta
C. ciano
D. giallo

D4.3 Quanti byte occupa in memoria `.asciz "CIAO"`?

A. 3
B. 4
C. 5
D. 8

D4.4 Su RISC-V, `.word 0x0000ABCD` è memorizzata all'indirizzo A. Quale byte si trova all'indirizzo A?

A. `0x00`
B. `0xAB`
C. `0xCD`
D. `0xDC`

4.6 Esercizi

Esercizio 4.1 (su PRG32)

1. Cambiate la barra del Capitolo 2 in magenta, poi in un grigio scuro a vostra scelta: calcolate prima a mano il valore RGB565.
2. Scrivete il vostro nome con `prg32_gfx_text8`. Quanti byte occupa la stringa?
3. Dichiarate un vettore di quattro colori con `.word` e disegnate quattro barre, una per colore, leggendo ogni colore con `lw`.
4. In PRG32-QT aprite la vista della memoria sulla vostra stringa e sul vostro vettore: trovate lo zero finale e verificate l'ordine dei byte.

Risultato atteso. Quattro barre in quattro colori; nella vista della memoria la stringa finisce con `00` e ogni `.word` mostra per primo il byte meno significativo. Un grigio scuro, per confronto: rosso 8 su 31, verde 16 su 63, blu 8 su 31 è `01000 010000 01000`, cioè `0x4208`.

4.7 Glossario

Inglese	Italiano
bit / byte / word	bit / byte / parola
positional notation	notazione posizionale
hexadecimal	esadecimale
sampling	campionamento
quantisation	quantizzazione
byte order (endianness)	ordinamento dei byte
string	stringa
array	vettore (array)
linked list	lista concatenata
stack	pila
tree	albero
machine learning	apprendimento automatico

4.8 Conclusioni e letture

L'informazione digitale è una configurazione di bit più una convenzione per leggerla. Gli stessi sedici bit possono essere un numero, un colore o due caratteri: lo decide il programma. L'esadecimale è una scrittura compatta del binario, quattro bit per cifra. Il testo è fatto di numeri (ASCII, UTF-8); le strutture sono byte più una regola per trovare l'elemento successivo. I dati sono la moneta di questo regno, e gli algoritmi trasformano i dati in informazione adatta a decidere.

Letture. Patterson e Hennessy (2020), paragrafi 2.4 e 2.9; Charles Petzold, *Code*, i capitoli su bit, byte e testo; per l'apprendimento automatico, Sebastian Raschka, *Machine Learning with Python*.

CAPITOLO 5

Rappresentazione dei numeri: gli interi

"Conterò poco, è vero: — diceva l'Uno ar Zero — ma tu che vali? Gnente: propio gnente."
— Trilussa, *Nummeri*, 1944

In questo capitolo

Alla fine di questo capitolo saprete

- convertire interi e frazioni tra le basi 2, 8, 10 e 16;
- dire quali numeri naturali stanno in n bit e riconoscere un overflow;
- codificare e decodificare interi con segno in segno e modulo e in complemento a due;
- sommare, sottrarre e cambiare di segno in complemento a due, e rilevare l'overflow;
- spiegare perché $0 - 1$ appare come `0xFFFFFFFF` in un registro RISC-V;
- scegliere l'istruzione RISC-V giusta per dati con segno e senza segno.

5.1 Riscaldamento sulla macchina

Un marcatore sta sullo schermo, alla posizione 2 di una linea orizzontale. Il programma legge il joystick e, finché **LEFT** è premuto, sottrae uno dalla posizione a ogni frame (Listato 5.1); la funzione `draw` stampa la posizione in esadecimale con `prg32_console_hex32`.

```
steer_update:
    ...
    call prg32_input_read      # a0 = button mask
    mv   t0, a0

    la   t1, marker_x
    lw   t2, 0(t1)             # t2 = marker_x, starts at 2
    andi t3, t0, BTN_LEFT      # is LEFT pressed?
    beqz t3, 1f
    addi t2, t2, -1            # one step to the left
1:  sw   t2, 0(t1)
    ...
# draw prints marker_x with prg32_console_hex32
```

Listato 5.1. Tenete premuto **LEFT** e guardate il numero (adattato dal laboratorio su stato e ingresso del libro di PRG32).

Provate voi

Tenete premuto **LEFT**. Il valore mostrato fa `00000002`, `00000001`, `00000000` … e poi `FFFFFFFF`, e il marcatore scompare. (Sulla scheda e in QEMU la console è disegnata sullo schermo; PRG32-QT la raccoglie nel suo log.) È un errore? `0xFFFFFFFF` è il numero 4 294 967 295 oppure il numero -1?

La risposta è: entrambi. Dipende da come si leggono i bit, e questo capitolo dà le regole.

5.2 Basi e conversioni

5.2.1 Sistemi posizionali

Definizione 5.1 (Rappresentazione posizionale)

Sia $b \geq 2$ un intero, la *base*. Una stringa di cifre $d_{n-1}\, d_{n-2} \cdots d_1\, d_0$, con $0 \leq d_i \leq b-1$ per ogni i, rappresenta in base b il numero naturale

$$N \;=\; \sum_{i=0}^{n-1} d_i\, b^{\,i} \;=\; d_{n-1} b^{n-1} + \cdots + d_1 b + d_0. \tag{5.1}$$

La cifra d_{n-1} è la *più significativa*, d_0 la *meno significativa*.

In informatica si usano quattro basi (Tabella 5.1). Ogni numero naturale ha esattamente una rappresentazione in ciascuna base, a parte gli zeri aggiunti a sinistra.

Tabella 5.1. Le quattro basi dell'informatica.

Base b	Cifre	Nome	Tredici si scrive
2	0, 1	binaria	`1101`
8	0, 1, …, 7	ottale	`15`
10	0, 1, …, 9	decimale	`13`
16	0, …, 9, A, …, F	esadecimale	`D`

5.2.2 Le basi che sono potenze di due condividono gli stessi bit

Se ogni cifra è scritta in binario puro, i bit di un numero sono gli stessi in ogni base $b = 2^k$: basta raggrupparli da destra, k alla volta, completando con zeri a sinistra. Per il numero 127:

$$\texttt{01111111}_2 \;=\; \underbrace{\texttt{0111}}_{7}\,\underbrace{\texttt{1111}}_{F} \;=\; \texttt{0x7F} \qquad \text{e} \qquad \texttt{01111111}_2 \;=\; \underbrace{\texttt{01}}_{1}\,\underbrace{\texttt{111}}_{7}\,\underbrace{\texttt{111}}_{7} \;=\; 177_8.$$

La ragione è che nella (5.1) con $b = 2$ i termini si possono raccogliere k alla volta, e ogni gruppo è una cifra moltiplicata per una potenza di 2^k.

5.2.3 Convertire un intero: divisioni successive

Per scrivere un numero naturale N in base r: si divide N per r; il resto è la cifra successiva, a partire dalla meno significativa; si ripete sul quoziente finché non è zero. I resti si leggono poi dall'ultimo al primo (Tabella 5.2).

Tabella 5.2. Il numero 127 convertito per divisioni successive.

$r = 8$

N	$N/8$	$N \bmod 8$
127	15	7
15	1	7
1	0	1

$127 = 177_8$

$r = 16$

N	$N/16$	$N \bmod 16$
127	7	$15 = \mathrm{F}$
7	0	7

$127 = \texttt{0x7F}$

$r = 2$

N	$N/2$	$N \bmod 2$
127	63	1
63	31	1
31	15	1
15	7	1
7	3	1
3	1	1
1	0	1

$127 = 1111111_2$

Il metodo è corretto perché, per la (5.1), $N = r\,(d_{n-1} r^{\,n-2} + \cdots + d_1) + d_0$: il resto della divisione di N per r è d_0, e il quoziente è il numero le cui cifre sono $d_{n-1} \cdots d_1$, a cui si applica lo stesso ragionamento. In C le due operazioni sono `/` e `%`; in RISC-V, con l'estensione M, sono `divu` e `remu`, e il Capitolo `??` usa esattamente questo algoritmo per stampare un punteggio.

5.2.4 Convertire una frazione: moltiplicazioni successive

Un numero $0 \le N < 1$ si scrive in base r come $N = d_{-1} r^{-1} + d_{-2} r^{-2} + \cdots$. Moltiplicando per r si ottiene $d_{-1} + (d_{-2} r^{-1} + \cdots)$: la parte intera del prodotto è la cifra successiva, a partire dalla più significativa, e il procedimento si ripete sulla parte frazionaria (Tabella 5.3).

Tabella 5.3. Due frazioni convertite in base 2.

0,875: esatta

Prodotto	Frazione	Cifra
$0{,}875 \times 2$	0,75	1
$0{,}75 \times 2$	0,5	1
$0{,}5 \times 2$	0	1

$0{,}875 = 0{,}111_2$

0,8: periodica

Prodotto	Frazione	Cifra
$0{,}8 \times 2$	0,6	1
$0{,}6 \times 2$	0,2	1
$0{,}2 \times 2$	0,4	0
$0{,}4 \times 2$	0,8	0
$0{,}8 \times 2$	0,6	1 …

$0{,}8 = 0{,}\overline{1100}_2$

Attenzione

Una frazione finita in base 10 può essere periodica in base 2. Con un numero finito di bit va troncata: da qui vengono gli errori di approssimazione dei numeri reali (Capitolo 6).

5.3 Numeri naturali e overflow

5.3.1 Strategie di codifica

Definizione 5.2 (Rappresentazione)

Una *rappresentazione* è una legge $X = R(x)$ che associa a un numero x una stringa di bit X. Qui x è il numero che si intende; X è la configurazione memorizzata nel registro.

Il numero di bit dipende da quanti numeri si vogliono rappresentare; di solito è un multiplo di 8, tipicamente 16, 32 o 64. La legge R si sceglie secondo il tipo di numero, naturale, con segno o razionale, e una buona legge è quella che rende semplici da costruire come circuiti le operazioni aritmetiche e i confronti.

5.3.2 Numeri naturali su n bit

Con n bit ci sono 2^n configurazioni, e la scelta naturale è associare a ogni numero naturale da 0 a $2^n - 1$ la stringa di bit che lo rappresenta nel sistema posizionale binario. Con $n = 4$ l'intervallo è [0,15]. Le dimensioni trattate da RISC-V sono nella Tabella 5.4.

Tabella 5.4. Numeri naturali su 8, 16 e 32 bit.

Bit	Intervallo	In RISC-V	Massimo
8	$0 \dots 2^8 - 1$	byte (`lbu`)	255
16	$0 \dots 2^{16} - 1$	mezza parola (`lhu`)	65 535
32	$0 \dots 2^{32} - 1$	parola, registro	4 294 967 295

Il numero 4 294 967 295 è la lettura di `0xFFFFFFFF` come numero naturale: la prima metà della risposta al riscaldamento.

5.3.3 Proprietà delle stringhe binarie

Dalla (5.1) seguono subito tre proprietà.

1. *L'estensione a sinistra con zeri non cambia il valore*: $5 = 101_2 = 00101_2$.
2. *Moltiplicare per 2^k è uno scorrimento a sinistra di k posizioni*: $00101_2 \times 2 = 01010_2$, cinque diventa dieci. Ogni cifra d_i passa alla posizione $i + k$, dove pesa 2^k volte tanto.
3. *La divisione intera per 2^k è uno scorrimento a destra di k posizioni*: $00101_2/2 = 00010_2$, cinque diventa due, e il resto va perso.

Gli scorrimenti costano molto meno delle moltiplicazioni, e i compilatori li usano ogni volta che possono (Listato 5.2). L'istruzione `slli t1, t1, 2` del Capitolo 4 moltiplicava così un indice per quattro. Gli scorrimenti sono anche la base dell'aritmetica in virgola fissa (Capitolo 6).

```
# multiply by 4
slli t0, t0, 2

# divide by 2 (unsigned)
srli t0, t0, 1

```

```
# x * 10 = x*8 + x*2
slli t1, t0, 3
slli t2, t0, 1
add  t0, t1, t2
```

Listato 5.2. Gli scorrimenti come moltiplicazioni e divisioni.

5.3.4 Overflow

Definizione 5.3 (Overflow)

Si ha un *overflow* quando il risultato di un'operazione non è contenuto nel sottoinsieme finito di numeri che i bit disponibili possono rappresentare.

I registri hanno una dimensione finita, e ce n'è un numero finito. Gli insiemi numerici sono infiniti, e di ciascuno si può rappresentare solo un sottoinsieme finito. Gli operatori aritmetici sono chiusi sull'intero insieme, non sul sottoinsieme finito. Con 7 bit si codifica l'intervallo [0,127]: 100 e 38 sono rappresentabili, la loro somma 138 no.

5.3.5 Aritmetica su quattro bit

L'aritmetica binaria segue le stesse regole colonna per colonna dell'aritmetica decimale, con l'unica tabellina $0+0=0$, $0+1=1$, $1+1=0$ con riporto di 1.

Somma		Somma con overflow		Differenza		Prodotto	
6 +	0110 +	14 +	1110 +	11 −	1011 −	5 x	0101 x
8 =	1000 =	3 =	0011 =	5 =	0101 =	3 =	0011 =
14	1110	17	1 0001	6	0110	15	000 1111

Nella seconda somma serve un quinto bit: il registro di quattro bit conserva `0001` e il riporto va perso. I processori segnalano un riporto perso in modi diversi: molti hanno un *flag di riporto* (carry flag); RISC-V non ne ha e lascia che sia il programma a calcolarlo con un confronto, come mostra la Sezione 5.5. Il prodotto di due numeri di n bit richiede fino a $2n$ bit: su RV32 l'istruzione `mul` dà i 32 bit bassi e `mulhu` i 32 alti.

5.4 Interi con segno

Ci sono tre modi di scrivere un segno meno con i bit. Due sono presentati qui e il terzo, con una variante, nel Capitolo 6.

5.4.1 Segno e modulo

Il bit più significativo è il segno (1 per i negativi) e gli altri $n-1$ bit sono il modulo:

$$X = |x| + 2^{n-1}s, \qquad s = \begin{cases} 1 & x < 0 \\ 0 & x \geq 0. \end{cases}$$

L'intervallo va da $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$. Lo zero ha due rappresentazioni, $+0$ e -0, quindi si rappresentano solo $2^n - 1$ numeri. E l'aritmetica deve lavorare separatamente su

segno e modulo: per sommare due numeri di segno opposto bisogna trovare il modulo maggiore, sottrarne il minore, e dare al risultato il segno dell'operando con il modulo maggiore. La rappresentazione è semplice per le persone e scomoda per i circuiti. Sopravvive nella mantissa dei numeri in virgola mobile (Capitolo 6).

5.4.2 Complemento a due

Definizione 5.4 (Complemento a due)

Su n bit, la rappresentazione in *complemento a due* di un intero x con $-2^{n-1} \le x \le 2^{n-1} - 1$ è il numero naturale

$$X = \begin{cases} x & \text{se } 0 \le x < 2^{n-1}, \\ 2^n - |x| & \text{se } -2^{n-1} \le x < 0, \end{cases}$$

scritto in binario su n bit. In modo equivalente, $X = x \bmod 2^n$.

L'intervallo va da -2^{n-1} a $+2^{n-1} - 1$, e lo zero ha una sola rappresentazione. Il bit più significativo vale 1 se e solo se il numero è negativo, perché i numeri negativi occupano le configurazioni da 2^{n-1} a $2^n - 1$. È la rappresentazione usata da ogni processore moderno, RISC-V compreso: un registro di 32 bit letto come intero con segno copre $-2^{31} \dots 2^{31} - 1$, da circa meno due miliardi a circa più due miliardi. L'intervallo è asimmetrico, con un numero negativo in più rispetto ai positivi, e questo ha una conseguenza che comparirà tra poco.

5.4.3 Quattro bit, tre letture

La Tabella 5.5 legge ciascuna delle sedici configurazioni di quattro bit nei tre modi visti finora. Quando il bit più significativo vale 0 le tre letture coincidono; quando vale 1 differiscono.

Tabella 5.5. Quattro bit, tre letture.

Bit	Senza segno	Segno-mod.	Compl. a 2	Bit	Senza segno	Segno-mod.	Compl. a 2
0000	0	+0	0	1000	8	−0	−8
0001	1	+1	+1	1001	9	−1	−7
0010	2	+2	+2	1010	10	−2	−6
0011	3	+3	+3	1011	11	−3	−5
0100	4	+4	+4	1100	12	−4	−4
0101	5	+5	+5	1101	13	−5	−3
0110	6	+6	+6	1110	14	−6	−2
0111	7	+7	+7	1111	15	−7	−1

Concetto

I bit non dicono quale sia la colonna giusta. Lo dice il programma, scegliendo le sue istruzioni.

5.4.4 La ruota dei numeri

Con n bit l'aritmetica è *modulo* 2^n: la retta dei numeri si chiude in un cerchio (Figura 5.1). Sommare 1 sposta di un passo in senso orario, qualunque sia la lettura. Le due letture differiscono solo nel punto in cui la ruota «si spezza». Nella lettura senza segno si spezza tra `1111` e `0000`, e attraversare quel punto è un riporto: $15 + 1 = 0$. Nella lettura in complemento a due si spezza tra `0111` e `1000`, e attraversare quel punto è un overflow con segno: $7 + 1 = -8$. Un passo indietro da `0000` è `1111`: ecco perché $0 - 1$ dà tutti uni.

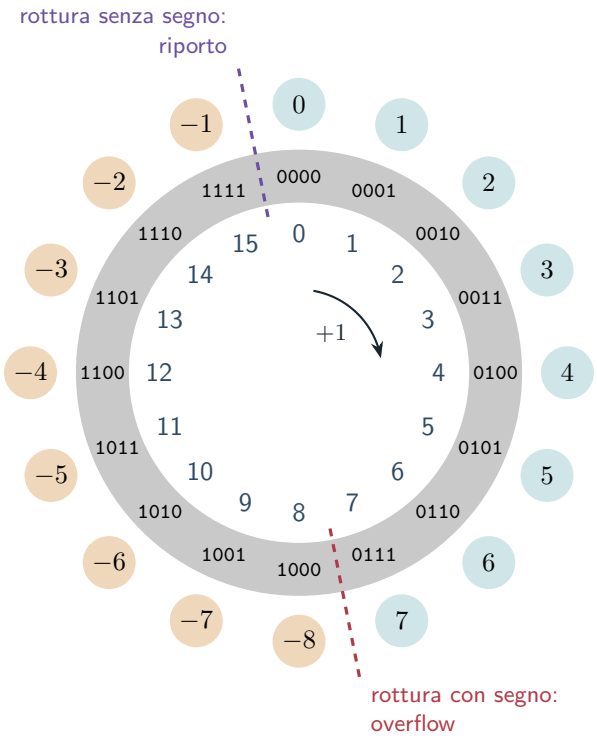


Figura 5.1. La ruota dei numeri per quattro bit. Numeri interni: valore senza segno; anello: bit; dischi esterni: valore in complemento a due.

Lo stesso sommatore serve a entrambe le letture; cambia solo il punto in cui la ruota si spezza.

5.4.5 Aritmetica in complemento a due

Teorema 5.1 (Addizione in complemento a due)

Siano x, y e $x + y$ tutti in $[-2^{n-1}, 2^{n-1} - 1]$, e siano X, Y le rappresentazioni di x e y. Allora la rappresentazione di $x+y$ è $(X+Y) \bmod 2^n$: la somma delle due configurazioni come numeri naturali, scartando il riporto uscente dal bit più significativo.

Dimostrazione. $X \equiv x$ e $Y \equiv y \pmod{2^n}$ per definizione, quindi $X + Y \equiv x + y \pmod{2^n}$; e la rappresentazione di $x + y$ è, per definizione, $(x + y) \bmod 2^n$. □

L'addizione non ha quindi bisogno di conoscere i segni: il sommatore per i numeri naturali dà già la risposta giusta.

Segni opposti		Due negativi		Prodotto	
2 +	0010 +	-2 +	1110 +	2 x	0010 x
-6 =	1010 =	-3 =	1101 =	-3 =	1101 =
-4	1100	-5	1 1011	-6	001 1010

Lo stesso ragionamento vale per la moltiplicazione: $XY \equiv xy \pmod{2^n}$, quindi gli n bit bassi di un prodotto sono corretti qualunque siano i segni. Nel terzo esempio $2 \times 13 = 26 = 1\,1010_2$, e i quattro bit bassi `1010` si leggono -6. Per questo RISC-V ha una sola `mul` per operandi con segno e senza segno.

5.4.6 Cambiare di segno un numero

Per definizione $-x$ è rappresentato da $2^n - X$. Due metodi lo calcolano senza una sottrazione.

Metodo 1: complementare ogni bit, poi sommare 1.
$6 =$ `0110`, complemento `1001`, più uno: `1010` $= -6$. Funziona perché il complemento di ogni bit di X è il numero $(2^n - 1) - X$, una sottrazione da tutti uni in cui non può esserci alcun prestito; sommando 1 si ottiene $2^n - X$.

Metodo 2: scorrere da destra.
Partendo da destra, si copiano i bit fino al primo 1 compreso, e si complementano tutti gli altri. $5 =$ `0101` dà `1011` $= -5$; $1 =$ `0001` dà `1111` $= -1$. È il metodo 1 con l'addizione svolta a mente: il $+1$ finale si propaga attraverso gli uni finali del complemento, che sono gli zeri finali dell'originale.

Cambiare di segno due volte restituisce il numero originale, con un'eccezione: il numero più negativo, -2^{n-1}, non ha un compagno positivo, e il suo cambio di segno va in overflow e restituisce il numero stesso.

```
# neg t0, t1 is a pseudo-instruction:
sub  t0, zero, t1

# the same thing by hand:
not  t0, t1        # complement
addi t0, t0, 1     # plus one
```

Listato 5.3. Il cambio di segno in RISC-V.

La sottrazione è un'addizione

$y - x = y + (2^n - x) \pmod{2^n}$. Un sommatore e un complementatore sono gli unici componenti necessari per costruire addizione, sottrazione, moltiplicazione e divisione.

È la ragione ingegneristica del complemento a due: un solo circuito, il sommatore del Capitolo **??**, fa tutto il lavoro. Nel datapath di RISC-V `sub` è `add` con il secondo operando complementato e un riporto entrante di 1.

5.4.7 Rilevare l'overflow

Senza segno, 8 bit		Con segno, 8 bit	
128 +	1000 0000 +	127 +	0111 1111 +
130 =	1000 0010 =	127 =	0111 1111 =
258	1 0000 0010	254	1111 1110

Nella prima somma c'è un riporto uscente, e il registro contiene 2, la somma modulo 256. Nella seconda non c'è riporto uscente, ma il risultato si legge -2.

Teorema 5.2 (Condizioni di overflow)

In un'addizione su n bit:

1. si ha un overflow *senza segno* se e solo se c'è un riporto uscente dal bit più significativo;
2. si ha un overflow *con segno* se e solo se due operandi con lo stesso segno danno una somma di segno opposto: due positivi danno un negativo, o due negativi danno un positivo.

La somma di due numeri di segno opposto non va mai in overflow.

L'ultima affermazione vale perché una tale somma è compresa tra i due operandi, entrambi rappresentabili. Le due condizioni sono indipendenti: la stessa addizione può far scattare l'una, entrambe o nessuna. In termini di circuiti la condizione con segno equivale a questa: il riporto *entrante* nel bit più significativo è diverso dal riporto *uscente* da esso.

5.5 Ritorno alla macchina

5.5.1 Il riscaldamento, spiegato

La variabile `marker_x` conteneva 0. L'istruzione `addi t2, t2, -1` ha fatto un passo indietro su una ruota con 2^{32} posizioni, e il registro ora contiene trentadue uni: `0xFFFFFFFF`. Letto senza segno è 4 294 967 295; letto in complemento a due è -1. Il sommatore ha fatto la stessa cosa in entrambi i casi. La funzione `prg32_console_hex32` stampa la configurazione, non un'interpretazione: non ha né ragione né torto. E il marcatore scompare perché a `prg32_gfx_rect` si chiede di disegnare a una x enorme, oppure a -1: fuori dallo schermo in ogni caso (Figura 5.2). Non è un errore del processore; è una decisione che manca nel programma.

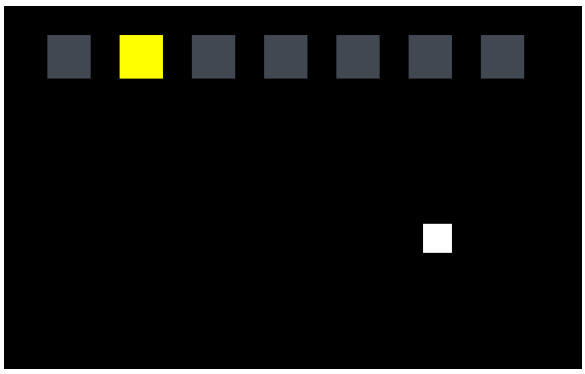


Figura 5.2. Il programma del laboratorio: sette lampade per i bit dei pulsanti e un marcatore.

5.5.2 Con segno o senza segno? Sceglie l'istruzione

I registri di RISC-V non hanno un tipo. Gli stessi 32 bit sono con segno o senza segno solo agli occhi dell'istruzione che li legge (Tabella 5.6).

Tabella 5.6. Istruzioni con segno e senza segno.

Operazione	Con segno	Senza segno	Perché differiscono
somma, sottrazione	`add`, `addi`, `sub`	le stesse	complemento a due: un sommatore per entrambi
prodotto (parola bassa)	`mul`	la stessa	i 32 bit bassi non dipendono dal segno
confronto	`slt`, `slti`	`sltu`, `sltiu`	`0xFFFFFFFF` è minore di 0? sì, se con segno
salto condizionato	`blt`, `bge`	`bltu`, `bgeu`	la stessa domanda, come salto
scorrimento a destra	`sra`, `srai`	`srl`, `srli`	copia il bit di segno, o inserisce zeri
caricamento di un byte	`lb`	`lbu`	estensione del segno o con zeri
divisione, resto	`div`, `rem`	`divu`, `remu`	i moduli si leggono in modo diverso

5.5.3 Tenere il marcatore sullo schermo

La decisione che mancava è presa nel Listato 5.4: dopo lo spostamento, la posizione viene *limitata* (clamp) all'intervallo $[0, 304]$.

```
    andi t3, t0, BTN_LEFT
    beqz t3, 1f
    addi t2, t2, -SPEED
1:  andi t3, t0, BTN_RIGHT
    beqz t3, 2f
    addi t2, t2, SPEED
2:
    bgez t2, 3f             # x < 0 ?  clamp to 0
    li   t2, 0
3:  li   t3, MAX_X          # .equ MAX_X, 304
    ble  t2, t3, 4f         # x > 304 ?  clamp to 304
    mv   t2, t3
4:
    sw   t2, 0(t1)          # one store, after all adjustments
```

Listato 5.4. Limitare la posizione (laboratorio su stato e ingresso, passo 3).

`bgez` legge `t2` come numero con segno: esamina il bit di segno. Anche `ble` è un confronto con segno: -1 è minore di 304, mentre con `bleu` la configurazione `0xFFFFFFFF` sarebbe di gran lunga maggiore. La costante `-SPEED` è un immediato: dodici bit in complemento a due, da -2048 a $+2047$.

Concetto

Un gioco non rileva l'overflow a cose fatte: mantiene i suoi valori dentro un intervallo in cui l'overflow non può avvenire.

5.5.4 Rilevare l'overflow senza flag

RISC-V non ha un flag di riporto né un flag di overflow: meno stati nascosti, un processore più semplice. Quando un programma ha davvero bisogno di saperlo, lo chiede con un confronto (Listato 5.5). Per i numeri senza segno, una somma che ha fatto il giro è sempre minore di ciascuno dei due operandi. Per i numeri con segno, sommare un numero negativo deve rendere la somma più piccola, e sommarne uno non negativo no. Sulla ruota, `sltu` rileva l'attraversamento della cima e il test con segno l'attraversamento del fondo.

```
# unsigned: did the sum wrap around?
    add  t2, t0, t1        # t2 = t0 + t1   (modulo 2^32)
    sltu t3, t2, t0        # t3 = 1 if the sum is smaller than
                           #        an operand: a carry was lost

# signed: did the sign come out wrong?
    add  t2, t0, t1
    slti t3, t1, 0         # t3 = 1 if t1 is negative
    slt  t4, t2, t0        # t4 = 1 if the sum is less than t0
    bne  t3, t4, overflow  # they must agree
```

Listato 5.5. I due test di overflow suggeriti dalla specifica RISC-V.

5.5.5 Estensione del segno: lo stesso numero su più bit

Teorema 5.3 (Estensione del segno)

Un numero in complemento a due su n bit si scrive su $m > n$ bit copiando il suo bit di segno in tutte le nuove posizioni a sinistra.

Dimostrazione. Se $x \geq 0$ il bit di segno è 0 e l'enunciato è l'estensione con zeri dei numeri naturali. Se $x < 0$ la sua configurazione su n bit è $X = 2^n - |x|$ e la configurazione voluta è $2^m - |x| = X + (2^m - 2^n)$; il numero $2^m - 2^n$ è esattamente $m - n$ uni seguiti da n zeri. □

-1 su 4 bit	`1111`	`= -1`
-1 su 8 bit	`1111 1111`	`= -1`
$+5$ su 4 bit	`0101`	`= +5`
$+5$ su 8 bit	`0000 0101`	`= +5`

L'estensione con zeri trasformerebbe `1111` (-1) in `0000 1111` ($+15$). L'istruzione `addi t2, t2, -1` memorizza -1 in dodici bit, e il processore ne estende il segno a trentadue prima di sommare. `lb` estende il segno del byte che carica, `lbu` riempie con zeri: la scelta dipende da che cosa significa il byte.

5.6 Verificate di aver capito

D5.1 Quale numero rappresenta in complemento a due la configurazione di 8 bit `1111 1100`?

A. 252
B. -4

C. -3
D. -124

D5.2 Su 8 bit, quale addizione produce un overflow con segno?

A. $100 + 27$
B. $100 + 28$
C. $-100 + 28$
D. $-1 + 1$

D5.3 `t0` contiene `0xFFFFFFFF` e `t1` contiene 1. Dopo `slt t2, t0, t1` e `sltu t3, t0, t1`:

A. `t2` $= 1$, `t3` $= 1$
B. `t2` $= 0$, `t3` $= 0$
C. `t2` $= 1$, `t3` $= 0$
D. `t2` $= 0$, `t3` $= 1$

D5.4 Come si scrive 0,625 in binario?

A. 0,101
B. 0,110
C. 0,625
D. 0,011

5.7 Esercizi

Esercizio 5.1 (prima su carta, poi su PRG32)

1. Convertite 2026 in binario, ottale ed esadecimale; convertite 0,1 in binario fino a otto bit frazionari.
2. Scrivete -100 su 8 bit in segno e modulo e in complemento a due.
3. Calcolate su 8 bit in complemento a due: $90 + 60$, $-90 - 60$, $90 - 60$. Quali vanno in overflow?
4. Nel codice del clamp sostituite `bgez` e `ble` con le loro versioni senza segno, prevedete il comportamento, poi eseguite.
5. Scrivete una `update` che somma `0x10000000` a un contatore a ogni frame e accende un rettangolo rosso quando la somma con segno va in overflow.

Risultato atteso. Su carta: $2026 =$ `0x7EA` $= 111\,1110\,1010_2$. Su PRG32: il rettangolo rosso lampeggia una volta ogni sedici frame, quando il contatore passa da `0x7...` a `0x8...`.

5.8 Glossario

Inglese	**Italiano**
base (radix)	base
natural number	numero naturale
signed integer	numero relativo (intero con segno)
most significant bit	bit più significativo
remainder	resto
shift	scorrimento (shift)
two's complement	complemento a due (alla base)
sign and magnitude	segno e modulo
overflow	overflow (traboccamento)
carry / borrow	riporto / prestito
sign extension	estensione del segno
to clamp	limitare a un intervallo

5.9 Conclusioni e letture

Ogni base ha i suoi vantaggi: la base 2 per i circuiti, la base 16 per le persone che leggono i bit. n bit rappresentano 2^n numeri: quali, dipende dalla legge di codifica. Il complemento a due trasforma la sottrazione in addizione, così che un solo sommatore fa tutto. L'overflow è il prezzo dei registri finiti: lo si rileva, oppure lo si evita per progetto. In RISC-V i bit non hanno un tipo: è l'istruzione a scegliere tra con segno e senza segno.

Letture. Patterson e Hennessy (2020), paragrafi 2.4 e 3.1–3.2; *Playing with RISC-V*, capitolo 6, «State and Input» (Montella et al., 2026b); Stallings (2021), i capitoli sui sistemi di numerazione e sull'aritmetica del calcolatore.

CAPITOLO 6

Rappresentazione dei numeri: i reali

"Conterò poco, è vero: — diceva l'Uno ar Zero — ma tu che vali? Gnente: propio gnente."
— Trilussa, *Nummeri*, 1944

In questo capitolo

Alla fine di questo capitolo saprete

- codificare interi con segno in complemento a uno e in eccesso-k, e dire a che cosa serve ciascuna codifica;
- rappresentare una grandezza frazionaria in virgola fissa e calcolare con essa usando istruzioni intere;
- spiegare mantissa, esponente e normalizzazione nella virgola mobile;
- codificare e decodificare un numero in IEEE 754 a precisione singola;
- riconoscere zero, numeri denormalizzati, infinito e NaN;
- dire perché l'ESP32-C6 non ha istruzioni in virgola mobile e che cosa fanno invece i giochi PRG32.

6.1 Riscaldamento sulla macchina

Una palla deve attraversare lentamente lo schermo. Un pixel per frame è troppo veloce; zero pixel per frame non è un movimento. Alla palla serve una via di mezzo: 0,375 pixel per frame. Ma il processore del kit è RV32IMAC: ha solo istruzioni intere, e nessuna virgola mobile. Il Listato 6.1 risolve il problema con un trucco.

```
.equ SPEED_Q8, 96            # 96 / 256 = 0.375 pixel per frame

ball_update:
    la   t0, ball_x_q8
    lw   t1, 0(t0)           # position, in 1/256 of a pixel
    addi t1, t1, SPEED_Q8    # add a fraction of a pixel
    sw   t1, 0(t0)
    ret

ball_draw:
    ...
```

```
12      la   t0, ball_x_q8
13      lw   t1, 0(t0)
14      srai a0, t1, 8          # x in whole pixels: drop 8 bits
15      ...
16      call prg32_gfx_rect
```

Listato 6.1. Tre ottavi di pixel per frame, con i soli interi.

La posizione è memorizzata *moltiplicata per 256*, e viene divisa per 256 solo quando la palla viene disegnata. In unità di 1/256 di pixel la velocità è $0{,}375 \times 256 = 96$, un onesto intero.

Provate voi

Eseguite il programma con `SPEED_Q8` uguale a 96, poi 16, poi 1.

Il trucco ha un nome, *virgola fissa*. Questo capitolo dice da dove viene e quando non basta, nel qual caso subentra la risposta generale, la *virgola mobile*. Prima dell'una e dell'altra servono altre due codifiche degli interi con segno, perché la virgola mobile è costruita con esse.

6.2 Altre codifiche per gli interi con segno

6.2.1 Complemento a uno

Definizione 6.1 (Complemento a uno)

Su n bit, un numero non negativo $0 \le x < 2^{n-1}$ è rappresentato da sé stesso, $X = x$; un numero negativo è rappresentato da $X = 2^n - 1 - |x|$.

L'intervallo va da $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$. Lo zero ha due rappresentazioni, `0000` e `1111`, quindi si rappresentano $2^n - 1$ numeri, come in segno e modulo. La ragione per usarlo è che il cambio di segno è banale: poiché $2^n - 1$ è una stringa di uni, sottrarre da esso complementa ogni bit,

$$-x \;=\; \texttt{not}\; x.$$

Quindi $6 = \texttt{0110}$ e $-6 = \texttt{1001}$; $5 = \texttt{0101}$ e $-5 = \texttt{1010}$; $1 = \texttt{0001}$ e $-1 = \texttt{1110}$. La `not` bit a bit di RISC-V calcola esattamente questo.

L'aritmetica in complemento a uno lavora modulo $2^n - 1$ invece che modulo 2^n, e questo costa una correzione.

Nessuna correzione		Due negativi		Risultato positivo	
3 +	0011 +	−2 +	1101 +	5 +	0101 +
−4 =	1011 =	−3 =	1100 =	−2 =	1101 =
−1	1110		1 1001 + 1 =		1 0010 + 1 =
		−5	1010	3	0011

Ogni volta che c'è un riporto uscente dal bit più significativo, al risultato va sommato 1: il *riporto circolare* (end-around carry). La correzione serve quando entrambi gli operandi sono negativi, oppure quando gli operandi hanno segni opposti e la somma è positiva. Un

sommatore e un complementatore bastano ancora; ma il passo in più e il doppio zero hanno fatto vincere il complemento a due.

6.2.2 Rappresentazione in eccesso-k

Definizione 6.2 (Eccesso-k)

Nella rappresentazione in *eccesso-k* ogni numero è traslato verso l'alto di una costante k, il *bias*: $X = x + k$. Tipicamente $k = 2^{n-1}$.

Su quattro bit con $k = 8$ la configurazione `0000` significa -8, `1000` significa 0 e `1111` significa $+7$. La proprietà decisiva è che l'ordine è conservato: se $x_1 > x_2$ allora $X_1 > X_2$, quindi i numeri con segno si confrontano come se fossero naturali. Il prezzo si paga nell'aritmetica: una somma va corretta sottraendo k, perché $X_1 + X_2 = (x_1 + x_2) + 2k$, e moltiplicazione e divisione sono molto più scomode che con i complementi. L'eccesso-k è quindi preferito dove servono solo somme e confronti, ed è esattamente la situazione dell'esponente di un numero in virgola mobile.

6.2.3 Quattro bit, cinque letture

La Tabella 6.1 completa la tabella del Capitolo 5. Segno e modulo e complemento a uno hanno due zeri. Complemento a due ed eccesso-8 coprono lo stesso intervallo, da -8 a $+7$, con un solo zero; differiscono soltanto nel bit più significativo, che è invertito. Solo l'eccesso-k mantiene le configurazioni nello stesso ordine dei numeri.

Tabella 6.1. Quattro bit, cinque letture. S-M: segno e modulo; C1, C2: complemento a uno e a due; Ecc-8: eccesso-8.

Bit	Nat.	S-M	C1	C2	Ecc-8	Bit	Nat.	S-M	C1	C2	Ecc-8
`0000`	0	+0	+0	0	−8	`1000`	8	−0	−7	−8	0
`0001`	1	+1	+1	+1	−7	`1001`	9	−1	−6	−7	+1
`0010`	2	+2	+2	+2	−6	`1010`	10	−2	−5	−6	+2
`0011`	3	+3	+3	+3	−5	`1011`	11	−3	−4	−5	+3
`0100`	4	+4	+4	+4	−4	`1100`	12	−4	−3	−4	+4
`0101`	5	+5	+5	+5	−3	`1101`	13	−5	−2	−3	+5
`0110`	6	+6	+6	+6	−2	`1110`	14	−6	−1	−2	+6
`0111`	7	+7	+7	+7	−1	`1111`	15	−7	−0	−1	+7

6.3 Virgola fissa

6.3.1 Numeri in virgola fissa

In virgola fissa la parte intera e la parte frazionaria di un numero sono memorizzate una accanto all'altra in una sola parola. La posizione della virgola è fissa e implicita: non è memorizzata, la conosce il programma.

Definizione 6.3 (Virgola fissa)

Con f bit frazionari un numero x è memorizzato come l'intero $X = x \cdot 2^f$ (arrotondato). La *risoluzione* è 2^{-f}. La notazione $Qm.f$ indica un formato con m bit interi e f bit frazionari.

Di solito l'intera parola è in complemento a due. La virgola fissa è il modo in cui giochi, audio e software di controllo trattano le frazioni sui processori privi di un'unità in virgola mobile. È anche il modo in cui andrebbe memorizzato il denaro: come numero intero di centesimi.

6.3.2 La palla, bit per bit: Q24.8

La palla del riscaldamento è nel formato Q24.8 (Figura 6.1). Quando la sua variabile contiene `0x00002A60` = 10848, la palla si trova a 10848/256 = 42,375 pixel: i 24 bit alti, `0x00002A`, sono i 42 pixel interi e gli 8 bassi, `0x60`, sono 96/256 = 0,375.

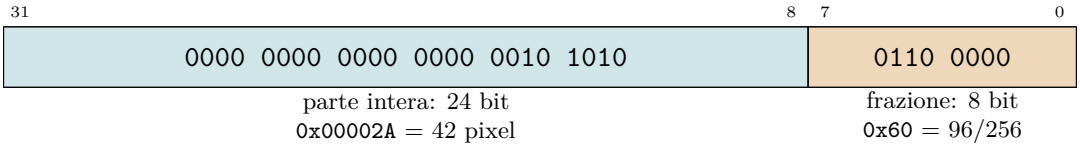


Figura 6.1. `ball_x_q8 = 0x00002A60`: la palla si trova a 42,375 pixel.

Dopo un altro frame il valore è `0x2AC0` (42,75), poi `0x2B20` (43,125): la palla attraversa il confine di un pixel ogni due o tre frame, e in media avanza di 0,375.

L'aritmetica in virgola fissa è aritmetica intera con un po' di attenzione (Tabella 6.2). Somme e differenze non ne richiedono, perché le virgole dei due operandi sono allineate. Il prodotto di due numeri scalati di 2^f è scalato di 2^{2f}, e va riportato indietro con uno scorrimento di f. Lo scorrimento che riporta ai pixel interi è quello *aritmetico*, `srai`, che conserva il segno (Capitolo 5).

Tabella 6.2. Le operazioni in virgola fissa.

Operazione	In virgola fissa	RISC-V
somma, sottrazione	come tra interi: le virgole sono allineate	`add`, `sub`, `addi`
ai pixel interi	scorrimento a destra di f, conservando il segno	`srai a0, t1, 8`
prodotto	moltiplicare, poi scorrere a destra di f	`mul`, poi `srai`

6.3.3 Numeri razionali: l'errore di approssimazione

I numeri reali sono densi: anche in un intervallo limitato, non ogni numero si può rappresentare con un numero finito di bit. A ogni numero si associa quindi il più vicino tra quelli rappresentabili. In Q24.8 il numero 0,1 diventa 26/256 = 0,1015625, perché $0{,}1 \times 256 = 25{,}6$ viene arrotondato a 26: un errore di circa l'1,6%.

L'errore di approssimazione è al massimo metà della risoluzione: 1/512 con otto bit frazionari. L'*underflow* è il caso di un numero vicino a zero, ma non zero, che viene rappresentato da zero: con una velocità di 0,001 pixel per frame la costante verrebbe arrotondata a 0, e la palla non si muoverebbe mai.

Concetto

In virgola fissa l'errore *assoluto* è lo stesso ovunque sulla retta dei numeri. La risoluzione di otto bit frazionari è 1/256: nulla di più piccolo si distingue da zero.

6.4 Virgola mobile

La virgola fissa spende tutti i suoi bit su un unico intervallo fissato. La virgola mobile ne spende alcuni per dire *dove* si trova l'intervallo: scambia precisione con ampiezza dell'intervallo.

6.4.1 Numeri reali in virgola mobile

Definizione 6.4 (Virgola mobile)

Un numero reale x è rappresentato in virgola mobile da una coppia (m,e) tale che

$$x = m \cdot b^{e},$$

dove m è la *mantissa*, e l'*esponente* e b la base.

La virgola è «mobile» perché è l'esponente a dire dove si trova. È la notazione scientifica del laboratorio di fisica. Infinite coppie rappresentano lo stesso numero,

$$346{,}09801 \;=\; (346{,}09801;\,0) \;=\; (346098{,}01;\,-3) \;=\; (0{,}034609801;\,4),$$

e la *normalizzazione* sceglie un unico rappresentante: la virgola va subito dopo la prima cifra significativa, (3,4609801; 2).

6.4.2 L'intervallo di un piccolo formato decimale

Si consideri un formato in base 10 con quattro cifre per la mantissa e due per l'esponente (Figura 6.2). Rappresenta i numeri da $-9{,}999 \times 10^{99}$ a $-1{,}000 \times 10^{-99}$, lo zero, e i numeri da $+1{,}000 \times 10^{-99}$ a $+9{,}999 \times 10^{99}$. Si ha un *overflow* quando il modulo è troppo grande per l'esponente; un *underflow* quando è troppo piccolo, e il numero collassa a zero.

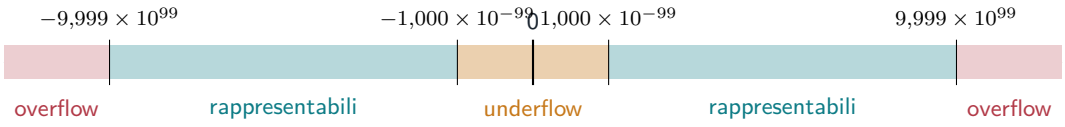


Figura 6.2. Un formato decimale con quattro cifre di mantissa e due di esponente (non in scala).

6.4.3 Approssimazione: errore assoluto ed errore relativo

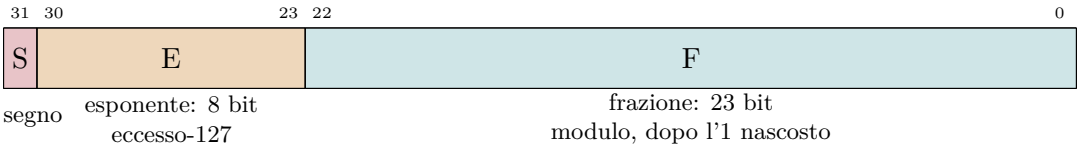
In un formato del genere l'errore assoluto non è costante. Vicino a zero la distanza tra due numeri consecutivi è $1{,}001 \times 10^{-99} - 1{,}000 \times 10^{-99} = 0{,}001 \times 10^{-99}$; vicino al limite superiore è $9{,}999 \times 10^{99} - 9{,}998 \times 10^{99} = 0{,}001 \times 10^{99}$. L'errore assoluto è piccolo sui numeri piccoli e grande sui numeri grandi. Ciò che è costante lungo tutto l'asse è l'errore *relativo*, perché il numero di cifre significative, quattro, è lo stesso ovunque.

Concetto

Virgola fissa: errore assoluto costante, errore relativo che esplode vicino a zero. Virgola mobile: errore relativo costante, errore assoluto che cresce con il numero.

6.4.4 IEEE 754 a precisione singola

Fino al 1985 ogni costruttore aveva un proprio formato in virgola mobile, e i programmi davano risultati diversi su macchine diverse. Lo standard IEEE 754 (1985, rivisto nel 2008 e nel 2019) vi ha posto fine (IEEE, 2019). Il suo formato a precisione singola divide 32 bit in tre campi (Figura 6.3).



$$\text{valore} \;=\; (-1)^{S} \times 1{,}F \times 2^{\,E-127}$$

Figura 6.3. IEEE 754 a precisione singola.

- La mantissa è in segno e modulo: un bit di segno e 23 bit di modulo.
- Una mantissa binaria normalizzata comincia sempre con 1. Quel bit è *nascosto*: non viene memorizzato, e si guadagna un bit di precisione.
- L'esponente è in eccesso-127 (non 128): gli esponenti veri vanno da -126 a $+127$. Grazie alla codifica in eccesso, e alla posizione dell'esponente prima della frazione, due numeri positivi in virgola mobile si possono confrontare come se fossero naturali.
- Le configurazioni $E = 0$ ed $E = 255$ sono riservate: zero e numeri denormalizzati, infinito e NaN.

Esempio 6.1 ($-5{,}125$ in IEEE 754)

1. Si convertono le due parti: $5 = 101_2$ e $0{,}125 = 1/8 = 0{,}001_2$, quindi $5{,}125 = 101{,}001_2$.
2. Si normalizza: $101{,}001_2 = 1{,}01001_2 \times 2^2$.
3. Segno $S = 1$; esponente $E = 2 + 127 = 129 = \texttt{1000 0001}$.
4. Frazione $F = \texttt{01001}$ seguita da zeri: l'1 iniziale è nascosto.

$$\underbrace{\texttt{1}}_{S}\;\underbrace{\texttt{1000 0001}}_{E}\;\underbrace{\texttt{0100 1000 0000 0000 0000 000}}_{F}$$

Raggruppato a nibble, `1100 0000 1010 0100 0000 ...` è `0xC0A40000`.

6.4.5 IEEE 754 in sintesi

La Tabella 6.3 confronta i due formati di uso comune, e la Tabella 6.4 elenca il significato di tutte le configurazioni.

I numeri *denormalizzati* (o subnormali) riempiono il vuoto dell'underflow attorno allo zero con una precisione che diminuisce gradualmente. La divisione $1/0$ dà infinito; $0/0$ e la radice quadrata di -1 danno NaN. NaN non è uguale a nulla, nemmeno a sé stesso.

Tabella 6.3. IEEE 754 a precisione singola e doppia.

	Precisione singola	**Precisione doppia**
Bit totali	32	64
Bit di segno	1	1
Bit di esponente	8	11
Bit di frazione	23	52
Codifica dell'esponente	eccesso-127	eccesso-1023
Intervallo dell'esponente	$-126 \ldots +127$	$-1022 \ldots +1023$
Minimo normalizzato	$2^{-126} \approx 1{,}2 \times 10^{-38}$	$2^{-1022} \approx 2{,}2 \times 10^{-308}$
Massimo	circa $2^{128} \approx 3{,}4 \times 10^{38}$	circa $2^{1024} \approx 1{,}8 \times 10^{308}$
Minimo denormalizzato	circa $1{,}4 \times 10^{-45}$	circa $4{,}9 \times 10^{-324}$
Cifre decimali	circa 7	circa 16

Tabella 6.4. IEEE 754: che cosa significano le configurazioni.

Tipo	**Segno**	**Esponente** E	**Frazione** F	**Valore**
Normalizzato	0 o 1	$0 < E < \max$	qualunque	$(-1)^S \times 1{,}F \times 2^{E-bias}$
Denormalizzato	0 o 1	tutti zeri	non zero	$(-1)^S \times 0{,}F \times 2^{1-bias}$
Zero	0 o 1	tutti zeri	tutti zeri	$+0$ o -0
Infinito	0 o 1	tutti uni	tutti zeri	$+\infty$ o $-\infty$
Not a Number	0 o 1	tutti uni	non zero	NaN: un risultato non valido

6.4.6 La virgola mobile non è l'insieme dei numeri reali

Un numero in virgola mobile è un numero razionale scelto da un insieme finito, e l'aritmetica su tali numeri obbedisce a leggi che non sono quelle del campo reale (Goldberg, 1991).

- 0,1 non ha un'espansione binaria finita: è memorizzato in modo approssimato, come 0,8 nel Capitolo 5.
- Due risultati in virgola mobile non vanno mai confrontati per uguaglianza: se ne confronta la differenza con una tolleranza (Listato 6.2).
- L'addizione non è associativa: $(a + b) + c$ può differire da $a + (b + c)$.
- Sommare un numero molto piccolo a uno molto grande può lasciare invariato quello grande.
- Per il denaro e per le posizioni di un gioco, gli interi scalati sono spesso lo strumento giusto.

```
double a = 0.1 + 0.2;

// a == 0.3 ?   false
// a = 0.30000000000000004

// better:
// fabs(a - 0.3) < 1e-9
```

Listato 6.2. La somma più famosa dell'informatica.

6.4.7 Oltre la singola e la doppia

La scelta di una rappresentazione è una decisione ingegneristica che si prende ancora oggi. La *mezza precisione*, aggiunta allo standard nel 2008, ha sedici bit: 1 di segno, 5 di esponente, 10 di frazione; dà circa tre cifre decimali in metà della memoria della precisione singola. Conta perché le reti neurali contengono miliardi di numeri: dimezzare ciascuno dimezza memoria, banda ed energia. Molti modelli vanno oltre e vengono memorizzati, per *quantizzazione*, come interi a 8 bit con un fattore di scala: di nuovo virgola fissa, su scala industriale. I bit sono un bilancio; intervallo, precisione e costo tirano in direzioni diverse, su una GPU come su una palla in Q24.8.

6.5 Ritorno alla macchina

6.5.1 La virgola mobile su RISC-V, e sul kit

RISC-V è modulare: una piccola base obbligatoria più estensioni facoltative (Tabella 6.5). Un processore RISC-V con le estensioni F e D ha altri trentadue registri, da `f0` a `f31`, per i valori in virgola mobile, e istruzioni come `fadd.s`, `fmul.s`, `flw` e `fsw` che lavorano su numeri IEEE 754; `fcvt.w.s` converte in intero e `fcvt.s.w` da intero. Processori così si trovano in portatili, server e supercalcolatori.

L'ESP32-C6 è RV32IMAC: non ha né F né D, e quindi nessuna istruzione in virgola mobile. Un programma C può comunque usare il tipo `float`: il compilatore chiama routine software come `__addsf3`, che realizzano esattamente l'algoritmo di questo capitolo con scorrimenti, maschere e somme intere, al costo di decine o centinaia di istruzioni intere per operazione. I giochi PRG32 usano interi e virgola fissa: veloci, abbastanza esatti, e visibili nel debugger.

Tabella 6.5. Leggere il nome RV32IMAC.

Lettera	Estensione	Che cosa aggiunge	ESP32-C6
RV32I	insieme intero di base	registri a 32 bit, somma, logica, load e store, salti	sì
M	moltiplicazione e divisione	`mul`, `mulh`, `div`, `rem` e le loro forme senza segno	sì
A	atomiche	istruzioni per la condivisione sicura tra task	sì
C	compresse	codifiche a 16 bit delle istruzioni più comuni	sì
F	precisione singola	da `f0` a `f31` e istruzioni IEEE 754 a precisione singola	no
D	precisione doppia	istruzioni IEEE 754 a precisione doppia	no

Un microcontrollore sceglie le estensioni che gli servono. È una delle ragioni per cui la stessa architettura scala da un chip che costa due euro a un supercalcolatore.

6.6 Verificate di aver capito

D6.1 Quale codifica con segno mantiene le configurazioni di bit nello stesso ordine dei numeri?

A. Segno e modulo
B. Complemento a uno

C. Complemento a due
D. Eccesso-k

D6.2 In virgola fissa Q24.8, quale numero reale rappresenta la parola `0x00000180`?

A. 384
B. 1,5
C. 1,8
D. 0,375

D6.3 Quale numero reale è la configurazione IEEE 754 a precisione singola `0x40400000`?

A. 2,0
B. 3,0
C. 4,0
D. 0,75

D6.4 In IEEE 754, un campo esponente di tutti uni con una frazione di tutti zeri significa:

A. zero
B. un numero denormalizzato
C. infinito
D. NaN

6.7 Esercizi

Esercizio 6.1 (prima su carta, poi su PRG32)

1. Scrivete -6 su quattro bit in segno e modulo, complemento a uno, complemento a due ed eccesso-8.
2. Codificate 10,5 e 0,15625 in IEEE 754 a precisione singola; date il risultato in esadecimale.
3. Decodificate `0xBF800000`.
4. Eseguite la palla con `SPEED_Q8` uguale a 96, 16 e 1. Dopo quanti frame si sposta di un pixel in ciascun caso?
5. Date alla palla anche una velocità verticale, e fatela rimbalzare: cambiate di segno la velocità con `neg` quando raggiunge un bordo.

Risultato atteso. Su carta: `1110`, `1001`, `1010`, `0010`; `0x41280000` e `0x3E200000`; $-1{,}0$. Su PRG32: la palla avanza di un pixel ogni 2,67, 16 e 256 frame; con il rimbalzo si muove in diagonale e resta sullo schermo.

6.8 Glossario

Inglese	**Italiano**
one's complement	complemento a uno (alla base diminuita)
excess-k (biased)	eccesso-k
fixed point	virgola fissa
floating point	virgola mobile
single / double precision	singola / doppia precisione
rounding	arrotondamento
mantissa / fraction	mantissa
exponent	esponente
normalisation	normalizzazione
hidden bit	bit nascosto
denormalised	denormalizzato
Not a Number (NaN)	non un numero (NaN)

6.9 Conclusioni e letture

Gli interi hanno più rappresentazioni: 8, 16, 32 o 64 bit, con segno o senza, complementi o eccesso. La virgola fissa memorizza una frazione come intero scalato: errore assoluto costante, la velocità degli interi. La virgola mobile memorizza una mantissa e un esponente: un intervallo enorme, errore relativo costante. IEEE 754 è lo standard universale per la virgola mobile, con configurazioni speciali per zero, infinito e NaN. Su un processore senza l'estensione F, la virgola fissa è la scelta naturale.

Letture. Patterson e Hennessy (2020), paragrafo 3.5; Goldberg (1991); *Playing with RISC-V*, l'appendice sull'interfaccia del runtime, per playfield e parallasse in Q8 (Montella et al., 2026b).

Il primo blocco è completo. Il prossimo capitolo mette al lavoro le sue quattro lezioni in un programma vero: la prima parte di Verm1.

Parte III

Logica

Parte IV

Stato e memoria

Parte V

La macchina

Parte VI

Pubblicazione e prospettive

Parte VII

Appendici

Bibliografia

George Boole. *An Investigation of the Laws of Thought*. Walton and Maberly, London, 1854.

Giacomo Bucci. *Calcolatori elettronici: architettura e organizzazione*. McGraw-Hill Education, 4 edizione, 2017.

Paul E. Ceruzzi. *A History of Modern Computing*. MIT Press, 2 edizione, 2003.

Espressif Systems. ESP-IDF programming guide, 2026. https://docs.espressif.com/projects/esp-idf/.

EuroHPC Joint Undertaking. Advancing European Sovereignty in HPC with RISC-V: the DARE Project. Press release, European High-Performance Computing Joint Undertaking, 2025. https://eurohpc-ju.europa.eu/.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, Institute of Electrical and Electronics Engineers, 2019.

Raffaele Montella. PRG32-QT: the portable C++20/Qt 6 host and debugger for PRG32 cartridges. https://github.com/riscv-prg32/PRG32-QT, 2026. Version 0.4.1, MIT License.

Raffaele Montella, Ivan Cafiero, e Simone Boscaglia. PRG32: Platform for RISC-V Gaming Development (32-bit). Software repository, University of Naples “Parthenope”, 2026a. https://github.com/riscv-prg32/PRG32.

Raffaele Montella, Ivan Cafiero, e Simone Boscaglia. *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform*. University of Naples “Parthenope”, 2026b. https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook.

Gordon E. Moore. Cramming more components onto integrated circuits. *Electronics*, 38(8): 114–117, 1965.

David A. Patterson e John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2 edizione, 2020.

RISC-V International. The RISC-V Instruction Set Manual. Technical specification, RISC-V International, 2024. https://riscv.org/technical/specifications/.

Claude E. Shannon. A symbolic analysis of relay and switching circuits. *Transactions of the American Institute of Electrical Engineers*, 57(12):713–723, 1938.

William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 11 edizione, 2021.

Andrew S. Tanenbaum e Todd Austin. *Structured Computer Organization*. Pearson, 6 edizione, 2013.

Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936.

John von Neumann. First draft of a report on the EDVAC. Rapporto tecnico, Moore School of Electrical Engineering, University of Pennsylvania, 1945.